

# 算法设计与分析 课程笔记

25-26 学年第 2 学期·蒋婷婷老师

AutoPku

2026-05-18

# Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>算法设计与分析—课程笔记</b>                           | <b>5</b>  |
| 0. 课程脉络                                       | 5         |
| 1. 课程概览                                       | 5         |
| 2. 算法地图                                       | 7         |
| 3. 中英文术语对照                                    | 8         |
| 4. 复习指引                                       | 8         |
| <b>Lecture 01 —引言与数学基础</b>                    | <b>9</b>  |
| 0. Motivation                                 | 9         |
| 1. 算法与时间复杂度                                   | 9         |
| 2. 函数渐进的界                                     | 10        |
| 3. 常用数学工具                                     | 11        |
| 4. 多项式 vs 指数时间                                | 12        |
| 5. 算法分析示例—插入排序                                | 13        |
| 6. 复杂度类层次                                     | 14        |
| 概念关系                                          | 15        |
| 记号速查                                          | 15        |
| <b>Lecture 02 —递推方程的求解</b>                    | <b>17</b> |
| 0. 概念                                         | 17        |
| 1. 迭代法                                        | 17        |
| 2. 递归树法                                       | 18        |
| 3. 尝试法 (代入法)                                  | 19        |
| 4. 主定理 (Master Theorem)                       | 19        |
| 5. 综合实例: 二分归并排序                               | 21        |
| 6. 综合实例: 快速排序                                 | 21        |
| 概念关系                                          | 21        |
| 记号速查                                          | 22        |
| <b>Lecture 03 —分治算法 (Divide and Conquer)</b>  | <b>23</b> |
| 0. 基本思想与算法框架                                  | 23        |
| 1. 提高效率的两类途径                                  | 23        |
| 2. 经典实例                                       | 23        |
| 概念关系                                          | 27        |
| 记号速查                                          | 27        |
| 易错点汇总                                         | 27        |
| <b>Lecture 04 —动态规划 (Dynamic Programming)</b> | <b>28</b> |
| 0. 适用条件                                       | 28        |
| 1. 设计步骤                                       | 28        |
| 2. 实例 1: 矩阵链乘法                                | 28        |
| 3. 实例 2: 最长公共子序列 (LCS)                        | 30        |
| 4. 实例 3: 投资问题                                 | 30        |
| 5. 实例 4: 0/1 背包                               | 31        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 6. 实例 5: 最大子段和 . . . . .                                      | 31        |
| 7. 实例 6: 最优二叉检索树 (OBST) . . . . .                             | 31        |
| 8. 实例 7: 编辑距离 (Levenshtein) . . . . .                         | 32        |
| 9. DP vs 分治 . . . . .                                         | 32        |
| 概念关系 . . . . .                                                | 32        |
| 记号速查 . . . . .                                                | 32        |
| 易错点汇总 . . . . .                                               | 33        |
| <b>Lecture 05 一贪心算法 (Greedy)</b>                              | <b>34</b> |
| 0. 贪心 vs DP . . . . .                                         | 34        |
| 1. 活动选择问题 . . . . .                                           | 34        |
| 2. 最优装载 . . . . .                                             | 35        |
| 3. 最小延迟调度 . . . . .                                           | 35        |
| 4. Huffman 编码 . . . . .                                       | 35        |
| 5. 最小生成树 . . . . .                                            | 36        |
| 6. Dijkstra 单源最短路径 . . . . .                                  | 37        |
| 7. 单段调度问题 (Single-machine scheduling) . . . . .               | 37        |
| 概念关系 . . . . .                                                | 37        |
| 记号速查 . . . . .                                                | 38        |
| 易错点汇总 . . . . .                                               | 38        |
| <b>Lecture 06 一回溯 (Backtracking) 与分支限界 (Branch and Bound)</b> | <b>39</b> |
| 0. 基本思想 . . . . .                                             | 39        |
| 1. 通用回溯框架 . . . . .                                           | 39        |
| 2. 经典实例 . . . . .                                             | 40        |
| 3. 搜索树规模估计—Monte Carlo . . . . .                              | 41        |
| 4. 分支限界 (Branch and Bound, BB) . . . . .                      | 41        |
| 5. 代价函数 (圆排列 BB) . . . . .                                    | 42        |
| 概念关系 . . . . .                                                | 42        |
| 记号速查 . . . . .                                                | 42        |
| 易错点汇总 . . . . .                                               | 42        |
| <b>Lecture 07 一线性规划 (Linear Programming)</b>                  | <b>43</b> |
| 0. 线性规划模型 . . . . .                                           | 43        |
| 1. 标准形 . . . . .                                              | 44        |
| 2. 基本可行解 (BFS) . . . . .                                      | 44        |
| 3. 单纯形法 (Simplex) . . . . .                                   | 45        |
| 4. 对偶规划 . . . . .                                             | 46        |
| 5. 整数线性规划 (ILP) 与分支限界 . . . . .                               | 47        |
| 概念关系 . . . . .                                                | 48        |
| 记号速查 . . . . .                                                | 48        |
| 易错点汇总 . . . . .                                               | 49        |
| <b>Lecture 08 一平摊分析 (Amortized Analysis)</b>                  | <b>50</b> |
| 0. 概念 . . . . .                                               | 50        |
| 1. 聚集分析—栈操作 . . . . .                                         | 50        |
| 2. 聚集分析—二进制计数器 . . . . .                                      | 50        |
| 3. 记账法 . . . . .                                              | 51        |
| 4. 势能法 (Potential Method) . . . . .                           | 51        |
| 5. 动态表 (Dynamic Table) . . . . .                              | 52        |
| 6. 其他经典平摊数据结构 . . . . .                                       | 53        |
| 概念关系 . . . . .                                                | 53        |
| 记号速查 . . . . .                                                | 53        |
| 易错点汇总 . . . . .                                               | 54        |
| 练习题 . . . . .                                                 | 54        |
| <b>Lecture 09 一网络流算法 (1)</b>                                  | <b>55</b> |

|                                              |           |
|----------------------------------------------|-----------|
| 0. 容量网络                                      | 55        |
| 1. 可行流与最大流                                   | 55        |
| 2. 割 (Cut)                                   | 56        |
| 3. 三大引理                                      | 56        |
| 4. 增广链与最大流性质                                 | 56        |
| 5. Ford-Fulkerson 标号法                        | 57        |
| 6. 提高效率的两条路径                                 | 58        |
| 7. 辅助网络 $N(f)$                               | 58        |
| 概念关系                                         | 59        |
| 记号速查                                         | 59        |
| 易错点汇总                                        | 59        |
| <b>Lecture 10 —网络流算法 (2): Dinic、最小费用流、应用</b> | <b>60</b> |
| 0. 回顾                                        | 60        |
| 1. 分层辅助网络                                    | 60        |
| 2. Dinic 算法                                  | 60        |
| 3. 最小费用流                                     | 61        |
| 4. 网络流应用                                     | 62        |
| 概念关系                                         | 63        |
| 记号速查                                         | 64        |
| 易错点汇总                                        | 64        |
| <b>Lecture 11 —问题复杂度 (1): 下界与决策树</b>         | <b>65</b> |
| 0. 算法评价                                      | 65        |
| 1. 算法最优性                                     | 65        |
| 2. 平凡下界                                      | 66        |
| 3. 决策树 (Decision Tree)                       | 66        |
| 4. 检索问题复杂度                                   | 67        |
| 5. 排序问题复杂度下界                                 | 67        |
| 6. 构造最坏输入 (Adversary Argument)               | 67        |
| 7. 时间复杂度分析—排序与选择算法的优化                        | 67        |
| 概念关系                                         | 68        |
| 记号速查                                         | 68        |
| 易错点汇总                                        | 68        |
| <b>Lecture 12 —问题复杂度 (2): 排序问题</b>           | <b>69</b> |
| 1. 冒泡排序                                      | 69        |
| 2. 堆 (Heap)                                  | 70        |
| 3. 排序问题复杂度下界                                 | 71        |
| 4. 决策树示例—冒泡排序 $n = 3$                        | 72        |
| 概念关系                                         | 72        |
| 记号速查                                         | 72        |
| 易错点汇总                                        | 73        |
| <b>Lecture 13 —问题复杂度 (3): 选择问题 + NP 完全理论</b> | <b>74</b> |
| 1. 选择问题下界—找最大与最小                             | 74        |
| 2. 选择问题下界—找第二大                               | 75        |
| 3. 选择算法复杂度表                                  | 75        |
| 4. 计算复杂性理论—P, NP, NPC                        | 76        |
| 5. 经典 NP-完全问题                                | 76        |
| 6. NP-完全问题求解策略                               | 77        |
| 7. 当前已知                                      | 78        |
| 概念关系                                         | 78        |
| 记号速查                                         | 78        |
| 易错点汇总                                        | 78        |
| <b>Lecture 14 —NP 完全 (NP-Completeness)</b>   | <b>79</b> |

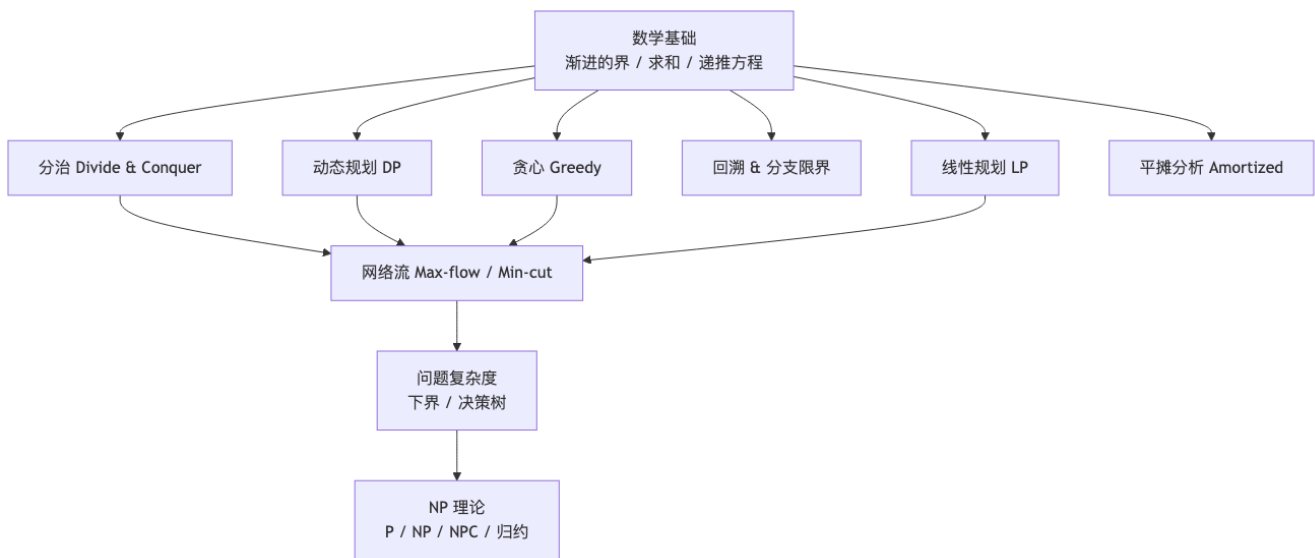
|                                                    |            |
|----------------------------------------------------|------------|
| 1. 核心问题与动机 (Motivation)                            | 79         |
| 2. 复习: 判定问题、P、NP                                   | 79         |
| 3. 多项式时间变换 ( $\leq_p$ )                            | 80         |
| 4. NP-完全 (NPC) 与 NP-难 (NP-Hard)                    | 81         |
| 5. Cook-Levin 定理 (SAT 是 NPC)                       | 82         |
| 6. 3-SAT 是 NPC                                     | 83         |
| 7. MAX-SAT 是 NPC (限制法)                             | 84         |
| 8. 图论 NPC 三件套: CLIQUE / IS / VC                    | 85         |
| 9. 哈密顿回路 (HC) 与货郎问题 (TSP)                          | 86         |
| 10. 数论类 NPC: SUBSET-SUM / PARTITION / KNAPSACK     | 86         |
| 11. 其他 NPC 问题                                      | 87         |
| 12. NP-完全归约图                                       | 88         |
| 13. NP-Hard vs NPC 区分                              | 88         |
| 14. coNP、PSPACE 与复杂度阶梯                             | 88         |
| 15. 处理 NPC 的实践策略                                   | 89         |
| 16. 概念关系                                           | 90         |
| 17. 关键定理一览                                         | 90         |
| 18. 记号速查                                           | 91         |
| 19. 易错点汇总                                          | 91         |
| <b>Lecture 15 —NP 完全 (2): 归约链、子问题分析与 Turing 归约</b> | <b>93</b>  |
| 1. 全图: NPC 归约谱系                                    | 93         |
| 2. 3-SAT $\leq_p$ 顶点覆盖 (VC): 构件设计法范例               | 94         |
| 3. 其他基本 NPC 问题                                     | 97         |
| 4. 应用 1: 用 NPC 理论分析子问题谱                            | 98         |
| 5. 应用 2: 把 NPC 理论推广到搜索 / 优化问题                      | 99         |
| 6. 一图总结: NPC 理论的”研究操作手册”                           | 102        |
| 7. 记号速查                                            | 103        |
| 8. 期末复习要点                                          | 103        |
| 9. Anki 卡片 (建议导出)                                  | 103        |
| <b>期中复习 (Midterm Review)</b>                       | <b>105</b> |
| 0. 考试范围                                            | 105        |
| 1. 数学基础                                            | 105        |
| 2. 分治策略                                            | 106        |
| 3. 动态规划                                            | 107        |
| 4. 贪心算法                                            | 108        |
| 5. 回溯 / 分支限界                                       | 108        |
| 6. 线性规划                                            | 109        |
| 7. 平摊分析                                            | 109        |
| 8. 解题模板                                            | 110        |
| 9. 常见易错点综合                                         | 111        |
| 10. 复习路线建议                                         | 112        |
| <b>2015 年算分期中试题—完整解答与复盘</b>                        | <b>113</b> |
| 第一题 (15 分) —阶排序                                    | 113        |
| 第二题 (10 分) —递推方程                                   | 114        |
| 第三题 (15 分) —递归算法分析                                 | 115        |
| 第四题 (15 分) —近似中值                                   | 115        |
| 第五题 (15 分) —文件副本放置 (DP)                            | 116        |
| 第六题 (15 分) —软件许可证购买 (贪心)                           | 117        |
| 第七题 (15 分) —会议室无线路由器布置 (回溯)                        | 118        |
| 总结表                                                | 119        |
| 备考建议                                               | 119        |

# 算法设计与分析—课程笔记

## [TIP] 课程定位

本课程是 PKU 信息科学技术学院计算机方向核心课, 既讲算法设计 (How to solve), 也讲算法分析与问题复杂度 (How hard). 期中考分治/动态规划/贪心/回溯/线性规划/平摊; 期末覆盖网络流 + NP 完全理论.

## 0. 课程脉络

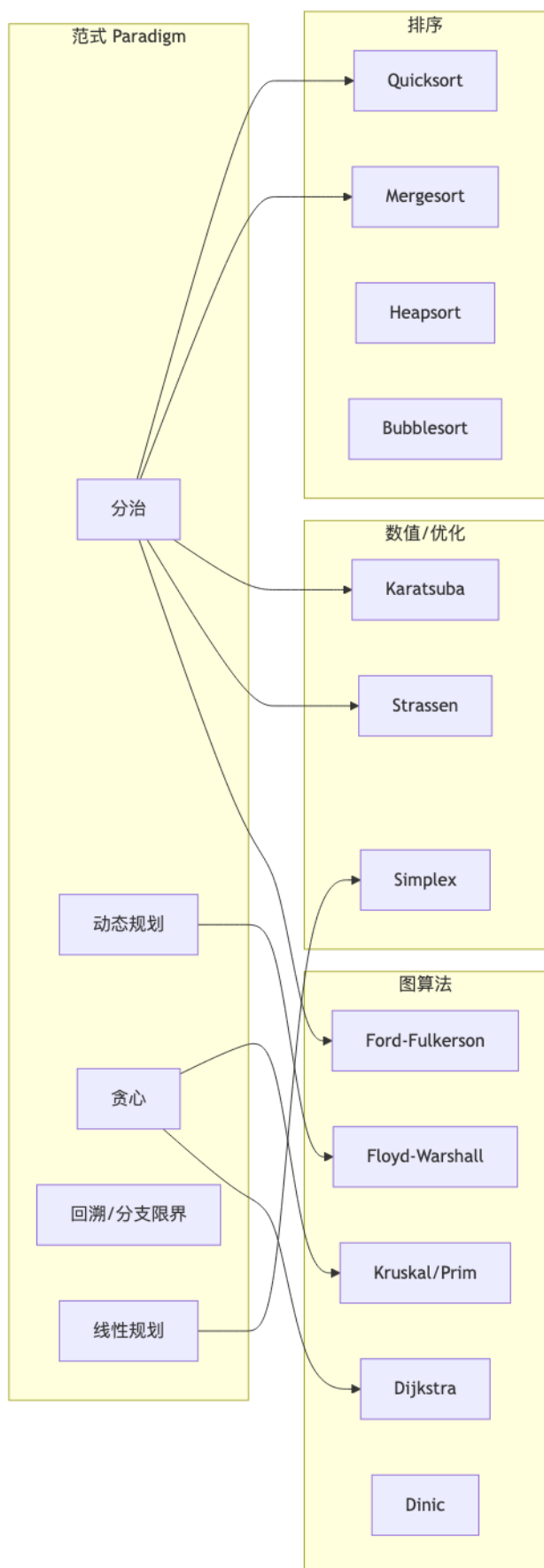


## 1. 课程概览

| 周   | 讲次         | 主题        | 关键算法/概念                            |
|-----|------------|-----------|------------------------------------|
| 1   | Lecture 01 | 引言 + 数学基础 | $O, \Omega, \Theta$ 、对数/阶乘/取整、求和   |
| 2   | Lecture 02 | 递推方程求解    | 迭代/递归树/Master 定理                   |
| 3   | Lecture 03 | 分治算法      | Karatsuba、Strassen、最近点对、Select     |
| 4-5 | Lecture 04 | 动态规划      | 矩阵链、LCS、投资、0/1 背包、OBST             |
| 6   | Lecture 05 | 贪心算法      | 活动选择、Huffman、Kruskal/Prim、Dijkstra |
| 7   | Lecture 06 | 回溯 & 分支限界 | n 后、子集和、TSP、最大团                    |
| 8   | Lecture 07 | 线性规划      | 标准形、单纯形、对偶、ILP 分支限界                |
| 9   | Lecture 08 | 平摊分析      | 聚集 / 记账 / 势能法、动态表                  |
| 10  | Lecture 09 | 网络流 (1)   | 最大流-最小割、Ford-Fulkerson             |
| 11  | Lecture 10 | 网络流 (2)   | Dinic、最小费用流、应用                     |
| 12  | Lecture 11 | 问题复杂度 (1) | 下界、决策树、检索                          |

| 周  | 讲次                         | 主题             | 关键算法/概念                                                    |
|----|----------------------------|----------------|------------------------------------------------------------|
| 13 | <a href="#">Lecture 12</a> | 问题复杂度 (2)      | 排序下界、堆排序、决策树                                               |
| 14 | <a href="#">Lecture 13</a> | 问题复杂度 (3) + NP | 选择下界、P/NP/NPC                                              |
| 15 | <a href="#">Lecture 14</a> | NP 完全理论 (1)    | $\leq_p$ 、Cook-Levin、<br>3-SAT/IS/VC/HC/TSP/SUBSET-SUM 归约链 |
| 16 | <a href="#">Lecture 15</a> | NP 完全理论 (2)    | 三大归约手段、VC 构件、子问题谱、Turing 归约、NP-equivalent                  |
| —  | <a href="#">期中复习</a>       | 期中复习           | Lec 1-8 全要点                                                |
| —  | <a href="#">2015 真题</a>    | 2015 期中真题      | 7 题完整解答                                                    |

## 2. 算法地图



### 3. 中英文术语对照

| 中文      | English                   | 备注                                  |
|---------|---------------------------|-------------------------------------|
| 渐进的界    | asymptotic bound          | $O, \Omega, \Theta, o, \omega$      |
| 主定理     | Master theorem            | $T(n) = aT(n/b) + f(n)$             |
| 分治      | divide and conquer        |                                     |
| 动态规划    | dynamic programming       | 自底向上 + 备忘录                          |
| 优化原则    | principle of optimality   | Bellman                             |
| 贪心选择性性质 | greedy-choice property    |                                     |
| 多米诺性质   | domino property           | 回溯前缀闭包                              |
| 单纯形法    | Simplex method            | LP                                  |
| 对偶规划    | dual linear program       |                                     |
| 松弛      | relaxation                | ILP→LP                              |
| 平摊分析    | amortized analysis        | aggregate / accounting / potential  |
| 势函数     | potential function $\Phi$ |                                     |
| 容量网络    | capacity network          | $N = \langle V, E, c, s, t \rangle$ |
| 增广链     | augmenting path           | F-F 算法核心                            |
| 最小割     | min cut                   | max-flow min-cut                    |
| 决策树     | decision tree             | 排序/检索下界                             |
| 多项式时间归约 | polynomial-time reduction | $\leq_p$                            |
| NP 完全   | NP-complete               | SAT, 3-SAT, ...                     |

### 4. 复习指引

- 第 1 阶段: 数学基础与递推方程 (Lec 1-2) — 必须会
- 第 2 阶段: 四大设计范式 (Lec 3-6) — 占期中考 ~70 %
- 第 3 阶段: LP + 平摊 (Lec 7-8) — 一期中
- 第 4 阶段: 网络流 (Lec 9-10) — 一期末重头
- 第 5 阶段: 复杂度 + NP (Lec 11-15) — 一期末重头, Lec 14-15 专门展开 NPC 归约链 (3-SAT → IS / VC / CLIQUE → HC → TSP / 0-1 KNAPSACK) 及搜索/优化版的 Turing 归约

#### [WARN] 应试注意

1. 算法题先用一段文字描述思想, 再给递推/伪码.
2. 动态规划: 状态/边界/转移/复杂度/标记函数 5 件套.
3. 贪心: 必须给出正确性证明 (归纳法或交换论证).
4. 回溯: 给出解向量、搜索树、约束条件.

# Lecture 01 —引言与数学基础

## [TIP] 学习指南

本节核心: 用渐近界  $O, \Omega, \Theta, o, \omega$  描述算法时间复杂度, 掌握求和、对数、阶乘、取整等基本工具, 为后面所有递推分析做准备. 前置知识: 离散数学 (集合、函数、归纳法), 微积分 (极限、定积分作和的近似). 重点关注: 渐近界的定义、定理 1.1 的极限判别、Stirling 公式、调和级数.

## 0. Motivation

我们关心两个层次的问题:

1. **可计算性** — 是否存在算法 (Turing 可计算).
2. **现实可计算性** — 是否存在 **多项式时间** 算法 (Cobham-Edmonds 论题).

## [EX] 实例对比 (假设 $10^9$ 次/秒)

– 快速排序  $10^5$  个数:  $\approx 1.7 \times 10^6$  步 = 1.7 ms – Dijkstra  $10^4$  顶点:  $\approx 10^8$  步 = 0.1 s – 回溯解 100 顶点最大团:  $\approx 1.8 \times 10^{32}$  步  $\approx 5.7 \times 10^{15}$  年

可见, **算法效率** 比 **机器速度** 更决定可解性. 提速 100 倍后, 对  $2^n$  算法仅多解 6.6 个单位规模; 对  $n$  算法可多解 100 倍.

## 1. 算法与时间复杂度

### 1.1 算法 (informal)

- 有限条指令序列;
- 输入  $\geq 0$  个, 输出  $\geq 1$  个;
- 对所有合法输入终止 (停机).

形式定义: 对所有有效输入停机的 Turing 机.

### 1.2 时间复杂度

最坏情况  $W(n)$ : 输入规模  $n$  的实例所需最长时间. 平均情况  $A(n)$ :

$$A(n) = \sum_{I \in S_n} p_I \cdot t_I,$$

其中  $S_n$  是规模为  $n$  的实例集,  $p_I$  为  $I$  的概率,  $t_I$  为基本运算次数.

## [EX] 顺序搜索的平均情况

假设  $x$  在  $L$  中的概率为  $p$ , 且各位置等概率. 则

$$A(n) = \sum_{i=1}^n \frac{p}{n} \cdot i + (1-p) \cdot n = \frac{p(n+1)}{2} + (1-p)n.$$

## 2. 函数渐进的界

### 2.1 五个符号

设  $f, g: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ .

| 符号              | 定义                                                                | 直观  |
|-----------------|-------------------------------------------------------------------|-----|
| $f = O(g)$      | $\exists c > 0, n_0, \forall n \geq n_0: 0 \leq f(n) \leq c g(n)$ | 上界  |
| $f = \Omega(g)$ | $\exists c > 0, n_0, \forall n \geq n_0: 0 \leq c g(n) \leq f(n)$ | 下界  |
| $f = \Theta(g)$ | $f = O(g) \wedge f = \Omega(g)$                                   | 同阶  |
| $f = o(g)$      | $\forall c > 0, \exists n_0: f(n) < c g(n)$                       | 严格小 |
| $f = \omega(g)$ | $\forall c > 0, \exists n_0: f(n) > c g(n)$                       | 严格大 |

### 2.2 定理 1.1 (极限判别)

设  $L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ :

- $0 < L < +\infty \Rightarrow f = \Theta(g)$ ;
- $L = 0 \Rightarrow f = o(g) \subseteq O(g)$ ;
- $L = +\infty \Rightarrow f = \omega(g) \subseteq \Omega(g)$ .

证明 (1): 取  $\varepsilon = L/2$ , 当  $n \geq n_0$  有  $|f/g - L| < L/2$ , 即  $L/2 g \leq f \leq 3L/2 g$ . 两端给出  $\Omega, O$ .

### 2.3 性质

- 传递性:  $f = O(g), g = O(h) \Rightarrow f = O(h)$  (对  $\Omega, \Theta$  同样);
- 加法吸收:  $f = O(h), g = O(h) \Rightarrow f + g = O(h)$ ;
- 推论:  $g = O(f) \Rightarrow f + g = \Theta(f)$ .

## [WARN] 易错点

-  $\Theta$  描述 同阶, 不允许在不同阶之间抖动. 例如  $f(n) = n$  若  $n$  奇数,  $f(n) = n^2$  若  $n$  偶数, 则  $f$  既不是  $\Theta(n)$  也不是  $\Theta(n^2)$ . - 阶反映  $n \rightarrow \infty$  大趋势, 可以忽略有限项. -  $o$  比  $O$  强, 但不是  $O$  的“严格”取反;  $f = O(g)$  不蕴含  $f = o(g)$ .

## 2.4 基本函数类 (阶由低到高)

$$1 \prec \log \log n \prec \log n \prec (\log n)^c \prec n^{1/k} \prec n \prec n \log n \prec n^c \prec 2^n \prec 3^n \prec n! \prec n^n.$$

[EX] 例 1

设  $f(n) = \frac{1}{2}n^2 - 3n$ . 取  $\lim f(n)/n^2 = 1/2$ , 由定理 1.1,  $f = \Theta(n^2)$ .

[EX] 例 2 (PrimalityTest)

试除法循环  $j = 2, \dots, \sqrt{n}$ . 当  $n$  为大素数时基本运算次数  $= \lfloor \sqrt{n} \rfloor - 1$ , 因此  $W(n) = O(\sqrt{n})$ . 但若  $n$  是  $2^k$  则只需 1 次便返回 false, 所以 **不能** 写  $W(n) = \Theta(\sqrt{n})$ .

## 3. 常用数学工具

### 3.1 对数函数

性质 (3.1):

$$\log_b(xy) = \log_b x + \log_b y, \quad \log_b x^\alpha = \alpha \log_b x, \quad x^{\log_b y} = y^{\log_b x}, \quad \log_a x = \frac{\log_b x}{\log_b a}.$$

记号约定:  $\log n$  默认  $\log_2 n$ ,  $\lg n = \log_{10} n$ ,  $\ln n = \log_e n$ .

阶:  $(\log n)^k = o(n^\alpha)$  对任意  $\alpha > 0$  成立; 即对数多项式 (polylogarithmic) 始终被多项式压制.

### 3.2 阶乘

Stirling 公式:

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + \Theta(1/n)). \quad (1)$$

由此:

- $n! = o(n^n)$ ;
- $n! = \omega(2^n)$ ;
- $\log(n!) = \Theta(n \log n)$ .

证明  $\log(n!) = \Theta(n \log n)$ :

$$\int_1^n \log x \, dx \leq \sum_{k=1}^n \log k \leq \int_1^{n+1} \log x \, dx,$$

而  $\int_1^n \log x \, dx = n \ln n - n + 1 = \Omega(n \log n)$ , 而  $\int_1^{n+1} \log x \, dx = O(n \log n)$ .

### 3.3 取整函数

记  $\lfloor x \rfloor$  为不超过  $x$  的最大整数;  $\lceil x \rceil$  为不小于  $x$  的最小整数.

性质:

$$1. \quad x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1;$$

2.  $\lfloor x + n \rfloor = \lfloor x \rfloor + n$  (整数  $n$ );
3.  $\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$ ;
4.  $\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/(ab) \rfloor$ .

### 3.4 求和公式

等差:  $\sum_{k=1}^n k = n(n+1)/2$

等比:  $\sum_{k=0}^{n-1} aq^k = a(1-q^n)/(1-q)$ ,  $q \neq 1$

无穷等比:  $\sum_{k=0}^{\infty} aq^k = a/(1-q)$ ,  $|q| < 1$

调和级数:  $H_n = \sum_{k=1}^n 1/k = \ln n + \gamma + O(1/n) = \Theta(\log n)$

$\sum_{k=1}^n \log k = \Theta(n \log n)$

### 3.5 放大法估上界

1.  $\sum_{k=1}^n a_k \leq n \cdot \max a_k$ ;
2. 若  $\frac{a_{k+1}}{a_k} \leq r < 1$ , 则  $\sum_{k=0}^n a_k \leq \frac{a_0}{1-r}$  (等比上界).

### 3.6 以积分作为求和近似

若  $f$  单调, 则

$$\int_a^{b+1} f(x) dx \leq \sum_{k=a}^b f(k) \leq \int_{a-1}^b f(x) dx \quad (\text{递增}),$$

反向单调时不等号反过来. 常用例:

$$\sum_{k=1}^n \frac{1}{k} = \int_1^{n+1} \frac{dx}{x} + O(1) = \ln(n+1) + O(1) = \Theta(\log n).$$

[EX] 例 3 (裂项求和)

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)} = \sum_{k=1}^{n-1} \left( \frac{1}{k} - \frac{1}{k+1} \right) = 1 - \frac{1}{n}.$$

[EX] 例 4 (差分求和)

利用差分  $a_k = b_{k+1} - b_k$ :

$$\sum_{k=0}^{n-1} k \cdot 2^k = (n-2) \cdot 2^n + 2.$$

推导提示: 令  $S = \sum k \cdot 2^k$ ,  $2S - S$  差消.

## 4. 多项式 vs 指数时间

| 函数    | $n = 10$    | $n = 30$           | $n = 60$                |
|-------|-------------|--------------------|-------------------------|
| $n$   | $10^{-5}$ s | $3 \times 10^{-5}$ | $6 \times 10^{-5}$      |
| $n^2$ | $10^{-4}$   | $9 \times 10^{-4}$ | $36 \times 10^{-4}$     |
| $n^5$ | 0.1 s       | 24.3 s             | 13 min                  |
| $2^n$ | 0.001 s     | 17.9 min           | 366 世纪                  |
| $3^n$ | 0.059 s     | 6.5 年              | $1.3 \times 10^{13}$ 世纪 |

机器加速 1000 倍后,  $n$  算法解决规模  $1000N$  倍, 但  $2^n$  算法仅增  $\log_2 1000 \approx 9.97$ .

#### [WARN] 启示

改进算法 (从  $2^n$  到  $n^5$ ) 比加速硬件 (从 1 倍到 100 倍) 更有效. 这是算法设计的核心动机.

## 5. 算法分析示例—插入排序

```

INSERTION-SORT(A):
  for j = 2 to n:
    key = A[j]; i = j-1
    while i > 0 and A[i] > key:
      A[i+1] = A[i]; i = i - 1
    A[i+1] = key

```

最坏情况 (输入逆序): 第  $j$  轮内循环执行  $j - 1$  次,

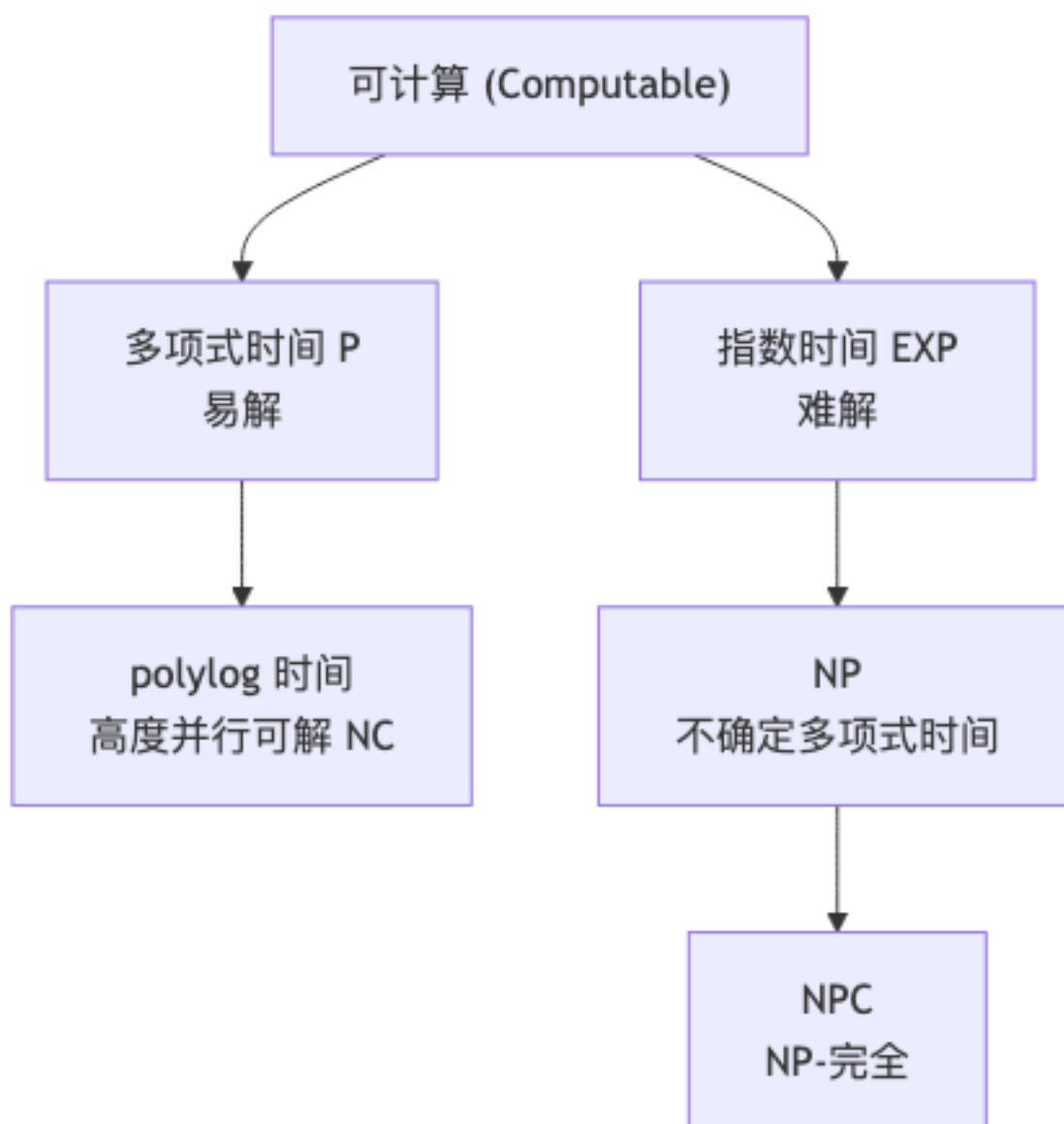
$$W(n) = \sum_{j=2}^n (j-1) = \frac{n(n-1)}{2} = \Theta(n^2). \quad (2)$$

最好情况 (已有序): 内循环 0 次,  $B(n) = n - 1 = \Theta(n)$ .

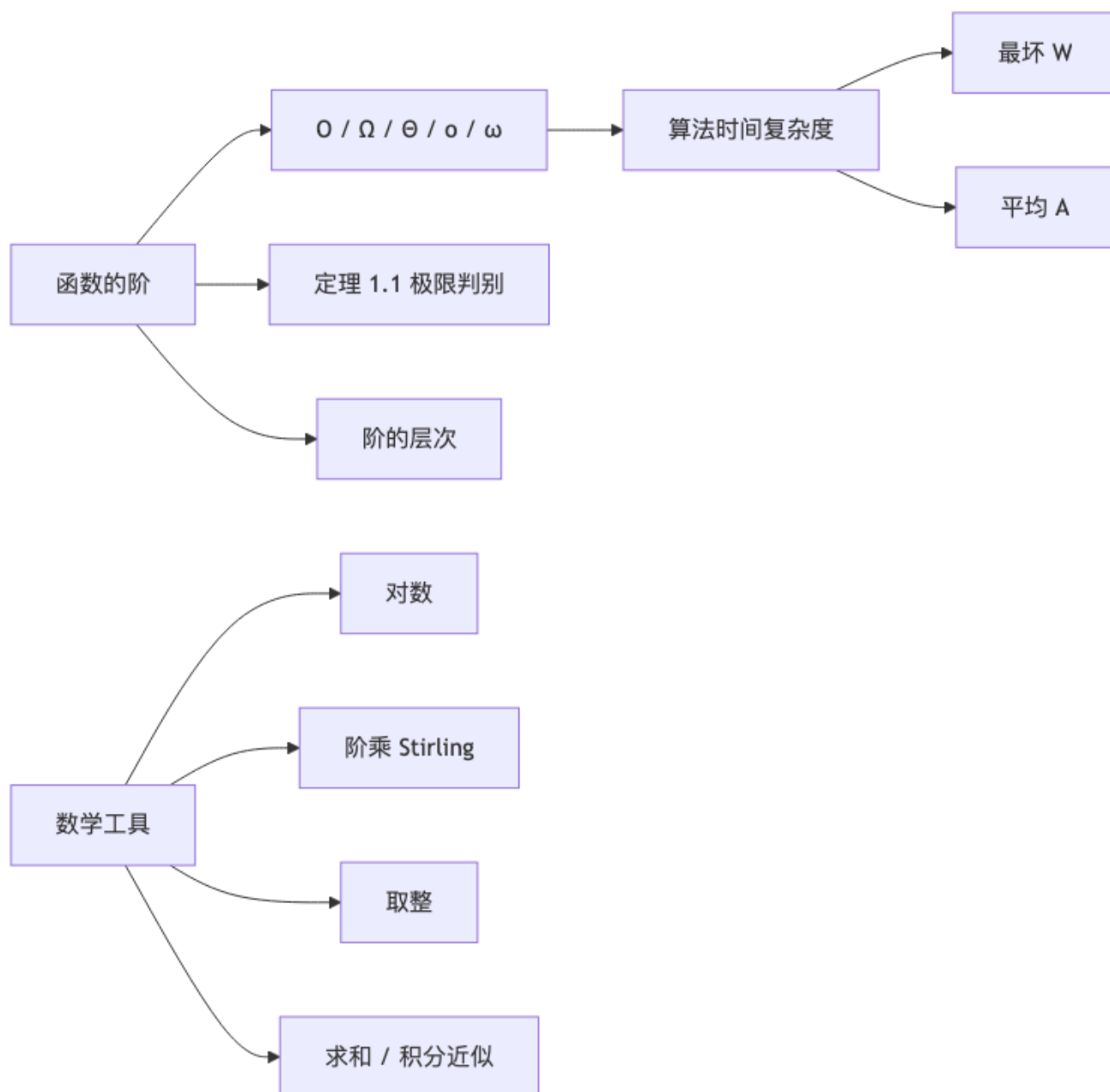
平均情况: 每轮内循环期望  $(j-1)/2$  次,

$$A(n) = \sum_{j=2}^n \frac{j-1}{2} = \frac{n(n-1)}{4} = \Theta(n^2).$$

## 6. 复杂度类层次



## 概念关系



## 记号速查

| 符号                                   | 含义              |
|--------------------------------------|-----------------|
| $O, \Omega, \Theta$                  | 紧上/下/紧界         |
| $o, \omega$                          | 严格上/下界          |
| $\log n$                             | $\log_2 n$ (默认) |
| $\ln n$                              | $\log_e n$      |
| $\lfloor x \rfloor, \lceil x \rceil$ | 下取整 / 上取整       |
| $H_n$                                | 调和数 $\sum 1/k$  |

| 符号     | 含义      |
|--------|---------|
| $W(n)$ | 最坏时间复杂度 |
| $A(n)$ | 平均时间复杂度 |

#### [TIP] 期中复习要点

1. 5 个渐近界符号定义 + 极限判别 (定理 1.1) 必背. 2. 常见阶的排序 (从  $\log \log n$  到  $n^n$ ) 必会比较. 3. Stirling:  $\log n! = \Theta(n \log n)$ , 直接用在排序下界. 4. 调和级数  $H_n = \Theta(\log n)$  — 快速排序平均分析关键. 5. 积分近似求和: 估  $\sum 1/k$ ,  $\sum \log k$ ,  $\sum k \log k$  都会用.

# Lecture 02 — 递推方程的求解

## [TIP] 学习指南

本节核心: 解递推方程  $T(n) = f(T(n-1), T(n/b), \dots) + g(n)$ , 是后续所有分治/DP 复杂度分析的工具. 前置知识: Lec 1 (渐近界, 求和, 积分近似). 重点关注: 三种迭代技巧 (直接、换元、差消)、递归树、主定理三种情形.

## 0. 概念

设序列  $\{a_n\}$ , 若有等式将  $a_n$  与某些  $a_i$  ( $i < n$ ) 联系起来, 称为序列  $\{a_n\}$  的 **递推方程** (recurrence). 算法时间分析的目标: **求闭式 (closed-form) 或渐近阶**.

四类求解方法:

1. 迭代法 (直接 / 换元 / 差消)
2. 递归树
3. 尝试法 (代入法/猜测试验)
4. 主定理

## 1. 迭代法

### 1.1 直接迭代

#### [EX] Hanoi 塔

$$T(n) = 2T(n-1) + 1, T(1) = 1.$$

$$\begin{aligned} T(n) &= 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1 = 2^2T(n-2) + 2 + 1 \\ &= 2^{n-1}T(1) + 2^{n-2} + \dots + 2 + 1 = 2^n - 1. \end{aligned}$$

#### [EX] 插入排序最坏

$$W(n) = W(n-1) + n - 1, W(1) = 0. \text{ 迭代后 } W(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2).$$

## 1.2 换元迭代

[EX] 二分归并排序

$W(n) = 2W(n/2) + n - 1$ ,  $W(1) = 0$ . 设  $n = 2^k$ :

$$\begin{aligned} W(2^k) &= 2W(2^{k-1}) + 2^k - 1 \\ &= 2^2W(2^{k-2}) + 2 \cdot 2^k - 2 - 1 \\ &= \dots = 2^k W(1) + k \cdot 2^k - (2^k - 1) \\ &= n \log n - n + 1 = \Theta(n \log n). \end{aligned}$$

## 1.3 差消化简后迭代

适用于平均情况下的递推, 通常包含求和形式.

[EX] 快速排序平均时间

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(n), \quad T(1) = 0.$$

两边乘  $n$ :

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2. \quad (1)$$

写  $(n-1)T(n-1)$  同样的式, 相减:

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + c(2n-1).$$

整理:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c(2n-1)}{n(n+1)}.$$

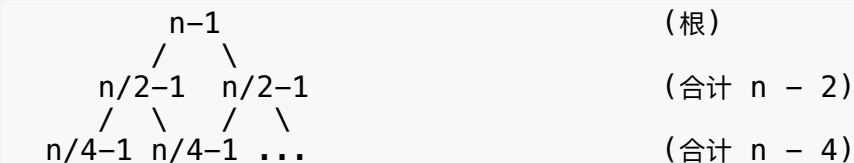
令  $U(n) = T(n)/(n+1)$ , 则  $U(n) = U(n-1) + \Theta(1/n)$ , 故  $U(n) = \Theta(\log n)$ , 即

$$T(n) = \Theta(n \log n).$$

## 2. 递归树法

将递推视作树, 每一层是一次展开的工作量.

[EX] 例:  $T(n) = 2T(n/2) + n - 1$



第  $k$  层 (从 0 计) 总工作量  $= n - 2^k$ , 共  $\log n + 1$  层. 总和

$$\sum_{k=0}^{\log n} (n - 2^k) = n(\log n + 1) - (2^{\log n + 1} - 1) = n \log n - n + 1.$$

[EX] 不平衡分裂:  $T(n) = T(n/3) + T(2n/3) + n$

每层总工作量  $= n$ . 最长路径长  $\log_{3/2} n = O(\log n)$ . 故  $T(n) = O(n \log n)$ . (右子树较深决定最大深度.)

### 3. 尝试法 (代入法)

猜测一个形式, 用数学归纳验证.

[EX] 快速排序  $T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(n)$

• 试  $T(n) = c \cdot n \log n$ :

$$\frac{2}{n} \sum_{i=1}^{n-1} ci \log i + O(n).$$

用  $\sum_{i=1}^{n-1} i \log i \leq \int_1^n x \log x dx = \frac{n^2 \log n}{2} - \frac{n^2}{4 \ln 2} + O(1)$ , 得

$$\leq \frac{2c}{n} \left( \frac{n^2 \log n}{2} \right) + O(n) - \frac{cn}{2 \ln 2} + O(n) = cn \log n - \frac{cn}{2 \ln 2} + O(n).$$

只要  $c$  足够大, 上式  $\leq cn \log n$ , 归纳成立.

[WARN] 尝试法注意

必须 **严格** 证明归纳步, 不能省略  $O(\cdot)$  中的常数. 否则把  $T(k) \leq ck \log k + O(k)$  中  $O$  直接收掉—那是不允许的.

### 4. 主定理 (Master Theorem)

设  $a \geq 1, b > 1$  为常数,  $f(n)$  非负,  $T(n)$  满足

$$T(n) = aT(n/b) + f(n). \quad (2)$$

令  $c^* = \log_b a$ . 则:

1. 情况 1: 若  $f(n) = O(n^{c^* - \varepsilon})$  ( $\varepsilon > 0$ ), 则  $T(n) = \Theta(n^{c^*})$ .

2. 情况 2: 若  $f(n) = \Theta(n^{c^*})$ , 则  $T(n) = \Theta(n^{c^*} \log n)$ .
3. 情况 3: 若  $f(n) = \Omega(n^{c^*+\varepsilon})$  且对某  $c < 1$  与充分大  $n$  有  $a f(n/b) \leq c f(n)$  (正则性), 则  $T(n) = \Theta(f(n))$ .

#### [WARN] 易错点

– 情况 1, 3 是多项式级别差:  $n^\varepsilon$  的 gap;  $f = n^{c^*}/\log n$  不属于情况 1,  $f = n^{c^*} \log n$  不属于情况 3 (gap 仅为  $\log n$ ). – 情况 3 必须验证正则性条件, 否则结论不成立. – 主定理只能处理形如 (2) 的递推, 不适用  $T(n) = T(n-1) + \dots$ .

## 4.1 主定理证明草图

设  $n = b^k$ . 迭代:

$$T(b^k) = a^k T(1) + \sum_{j=0}^{k-1} a^j f(b^{k-j}). \quad (3)$$

第一项  $a^k = a^{\log_b n} = n^{c^*}$ . 第二项视  $f$  与  $n^{c^*}$  的关系而定.

- 若  $f(n) = O(n^{c^*-\varepsilon})$ , 则  $a^j f(b^{k-j}) = O(a^j (b^{k-j})^{c^*-\varepsilon}) = O(a^k b^{-\varepsilon(k-j)})$ , 求和后为等比,  $\Theta(a^k) = \Theta(n^{c^*})$ , 与  $a^k T(1)$  同阶.
- 若  $f(n) = \Theta(n^{c^*})$ , 则每层  $a^j f(b^{k-j}) = \Theta(n^{c^*})$ , 共  $k = \log_b n$  层, 总  $\Theta(n^{c^*} \log n)$ .
- 若  $f$  足够大 + 正则, 几何级数被  $f(n)$  主导, 总  $\Theta(f(n))$ .

## 4.2 典型递推方程

| 递推                                    | $a$ | $b$ | $f$   | 类别 | 结果                                               |
|---------------------------------------|-----|-----|-------|----|--------------------------------------------------|
| 二分检索<br>$T(n) = T(n/2) + 1$           | 1   | 2   | 1     | 2  | $\Theta(\log n)$                                 |
| 归并排序<br>$T(n) = 2T(n/2) + n$          | 2   | 2   | $n$   | 2  | $\Theta(n \log n)$                               |
| Karatsuba<br>$T(n) = 3T(n/2) + n$     | 3   | 2   | $n$   | 1  | $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ |
| Strassen<br>$T(n) = 7T(n/2) + n^2$    | 7   | 2   | $n^2$ | 1  | $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$ |
| Trivial 矩阵乘<br>$T(n) = 8T(n/2) + n^2$ | 8   | 2   | $n^2$ | 1  | $\Theta(n^3)$                                    |
| 最近点对<br>$T(n) = 2T(n/2) + n$          | 2   | 2   | $n$   | 2  | $\Theta(n \log n)$                               |

| 递推                                    | $a$ | $b$ | $f$ | 类别      | 结果          |
|---------------------------------------|-----|-----|-----|---------|-------------|
| Select $T(n) = T(n/5) + T(7n/10) + n$ | —   | —   | —   | (不属主定理) | $\Theta(n)$ |

### 4.3 不属于主定理的反例

- $T(n) = 2T(n/2) + n \log n$ :  $c^* = 1$ ,  $f(n) = n \log n$  既不是  $O(n^{1-\epsilon})$  也不是  $\Omega(n^{1+\epsilon})$ , 也不是  $\Theta(n)$ . 属于扩展情况  $f = \Theta(n^{c^*} \log^k n)$ ,  $k \geq 0$ , 解为  $T(n) = \Theta(n^{c^*} \log^{k+1} n)$ . 此处  $T(n) = \Theta(n \log^2 n)$ .

[EX] 2015 年期中第二题

$T(n) = 4T(n/2) + n^2 \log^k n$ ,  $T(1) = 1$ .  $c^* = \log_2 4 = 2$ ,  $f = n^2 \log^k n = \Theta(n^{c^*} \log^k n)$ . 由扩展主定理:  

$$T(n) = \Theta(n^2 \log^{k+1} n).$$

## 5. 综合实例: 二分归并排序

```

MERGE-SORT(A, p, r):
  if p < r:
    q = (p + r) // 2
    MERGE-SORT(A, p, q)
    MERGE-SORT(A, q+1, r)
    MERGE(A, p, q, r)      # 合并  $O(r-p+1)$ 

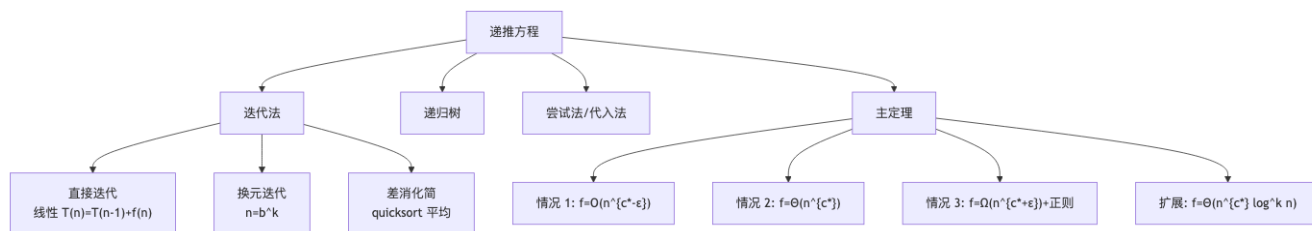
```

递推  $T(n) = 2T(n/2) + n - 1$ , 主定理情形 2,  $T(n) = \Theta(n \log n)$ .

## 6. 综合实例: 快速排序

最坏  $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$ . 最好  $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ . 均衡划分  $T(n) = T(n/10) + T(9n/10) + \Theta(n)$ : 递归树每层和  $\Theta(n)$ , 深度  $\log_{10/9} n = \Theta(\log n)$ ,  $T(n) = \Theta(n \log n)$ . 平均 (随机 pivot): 同 §1.3 推得  $\Theta(n \log n)$ .

## 概念关系



## 记号速查

| 符号                             | 含义                |
|--------------------------------|-------------------|
| $c^* = \log_b a$               | 主定理临界指数           |
| $W(n)$                         | 最坏运行时间            |
| $A(n)$                         | 平均运行时间            |
| $T(b^k)$                       | 主定理中迭代到 $n = b^k$ |
| $\Theta(n^{c^*} \log^{k+1} n)$ | 扩展情形 2 解          |

### [TIP] 期中复习要点

1. 三种迭代各对应典型例: 直接 = 插入排序, 换元 = 归并, 差消 = 快排平均. 2. 递归树画法: 标注每层工作量 + 层数 + 总和; 不平衡分裂用最长路径定深度. 3. 主定理三情形 + 正则性条件 + 不适用的边界 (log 因子) 必须熟练. 4. 常见递推-阶对照表 (4.2 节) 推荐背诵.

# Lecture 03 一分治算法 (Divide and Conquer)

## [TIP] 学习指南

本节核心: 把规模  $n$  的问题拆成  $k$  个规模  $\approx n/b$  的同型子问题, 递归求解后合并. 前置知识: Lec 2 (主定理). 重点关注: Karatsuba 位乘、Strassen 矩阵乘、平面最近点对、Select(中位数) 五大经典例.

## 0. 基本思想与算法框架

```
DIVIDE-AND-CONQUER(P):  
  if |P| <= c:                                # 递归基  
    return solve_directly(P)  
  divide P into P_1, ..., P_k                  # 划分 (Divide)  
  for i = 1 to k:  
    y_i = DIVIDE-AND-CONQUER(P_i)  
  return merge(y_1, ..., y_k)                  # 合并 (Conquer)
```

关键: 连续划分 + 平衡原则. 不平衡划分会导致最坏指数; 子问题应与原问题 同型.

复杂度递推:

$$W(n) = a W(n/b) + f(n),$$

即可直接套主定理.

## 1. 提高效率的两类途径

### 1.1 代数变换—减少子问题个数

把  $a$  减小, 使  $\log_b a$  下降, 改善  $\Theta(n^{\log_b a})$  部分.

### 1.2 预处理—减少递归中的操作

例如最近点对算法事先排序, 避免每次递归再排序.

## 2. 经典实例

### 2.1 芯片测试

问题:  $n$  片芯片, 好芯片至少比坏芯片多 1. 用最少测试次数挑出 1 片好芯片. 两两测试结果: “都好/都坏”或“至少一个坏”.

算法:

```
while k > 3:
    分成 k/2 组, 每组两片测试
    if "都说对方好" : 留 1 片
    else : 都丢
if k = 3: 任取 2 片测试 ...
```

每轮  $n \rightarrow n/2$ , 测试  $n/2$  次. 递推

$$W(n) = W(n/2) + O(n), W(1) = 0, \Rightarrow W(n) = \Theta(n).$$

由主定理情形 3.

## 2.2 求幂 $a^n$

```
POWER(a, n):
    if n == 0: return 1
    t = POWER(a, n // 2)
    if n is even: return t * t
    else : return t * t * a
```

$$T(n) = T(n/2) + \Theta(1) = \Theta(\log n) \text{ (情形 2)}.$$

## 2.3 Fibonacci 数

矩阵形式:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}. \quad (1)$$

用 §2.2 的 fast exponentiation 计算矩阵幂:  $T(n) = \Theta(\log n)$ .

## 2.4 Karatsuba 大整数乘法

问题:  $n$  位整数  $X, Y$  相乘, 设  $n = 2^k$ .

传统:  $\Theta(n^2)$ .

朴素分治: 令  $X = A \cdot 2^{n/2} + B$ ,  $Y = C \cdot 2^{n/2} + D$ . 则

$$XY = AC \cdot 2^n + (AD + BC) \cdot 2^{n/2} + BD.$$

4 次半规模乘法  $W(n) = 4W(n/2) + cn = \Theta(n^2)$ . 没有改进!

Karatsuba 代数变换 (关键):

$$AD + BC = (A - B)(D - C) + AC + BD. \quad (2)$$

只需 3 次半规模乘法 ( $AC, BD, (A - B)(D - C)$ ), 加减线性时间:

$$W(n) = 3W(n/2) + cn \Rightarrow W(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

### [EX] 演算

$1234 \times 5678$ ,  $A = 12, B = 34, C = 56, D = 78$ .  $AC = 672, BD = 2652, (A - B)(D - C) = (-22)(22) = -484$ .  $AD + BC = -484 + 672 + 2652 = 2840$ .  $XY = 672 \cdot 10^4 + 2840 \cdot 10^2 + 2652 = 6720000 + 284000 + 2652 = 7006652$ . (验算:  $1234 \times 5678 = 7006652$ . )

## 2.5 Strassen 矩阵乘法

问题:  $n \times n$  矩阵  $A, B$  相乘. 朴素  $\Theta(n^3)$ .

朴素分治: 每个  $n \times n$  块化为 8 次  $n/2 \times n/2$  乘 + 4 次加  $W = 8W(n/2) + \Theta(n^2) = \Theta(n^3)$ . 仍未改进.

Strassen 公式—7 次乘法即可:

$$\begin{aligned}M_1 &= A_{11}(B_{12} - B_{22}) \\M_2 &= (A_{11} + A_{12})B_{22} \\M_3 &= (A_{21} + A_{22})B_{11} \\M_4 &= A_{22}(B_{21} - B_{11}) \\M_5 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\M_6 &= (A_{12} - A_{22})(B_{21} + B_{22}) \\M_7 &= (A_{11} - A_{21})(B_{11} + B_{12})\end{aligned}$$

合并:

$$\begin{aligned}C_{11} &= M_5 + M_4 - M_2 + M_6 \\C_{12} &= M_1 + M_2 \\C_{21} &= M_3 + M_4 \\C_{22} &= M_5 + M_1 - M_3 - M_7\end{aligned}$$

递推  $W(n) = 7W(n/2) + \Theta(n^2) \Rightarrow W(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$  (情形 1).

### [WARN] 注意

Strassen 实际常数较大且数值稳定性差, 大多在  $n \geq 100$  时才优于朴素.

## 2.6 平面最近点对

问题: 平面上  $n$  个点, 求最小欧氏距离对.

算法 MinDistance(P, X, Y) —  $X, Y$  为按  $x, y$  预排序数组:

1. if  $|P| \leq 3$ : 直接  $O(1)$  算
2. 取垂直线  $\square$ , 把  $P$  分为左  $P_L$  与右  $P_R$
3.  $\delta_L = \text{MinDistance}(P_L, X_L, Y_L)$
4.  $\delta_R = \text{MinDistance}(P_R, X_R, Y_R)$
5.  $\delta = \min(\delta_L, \delta_R)$
6. 取  $\square$  两侧  $\delta$ -范围内的点 (按  $y$  排序), 对每点只比较  $y$  距离  $\leq \delta$  的至多 6 个点
7. 更新  $\delta$

关键观察: 在两侧  $\delta \times 2\delta$  矩形内, 任意两点距离  $\geq \delta$  蕴含至多 6 个候选 (鸽巢:  $\delta/2 \times \delta/2$  方格内至多 1 个点).

复杂度: 预排序  $O(n \log n)$  一次性. 递归  $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$ .

[WARN] 易错点

步 2 的排序若每次递归内都做, 复杂度变为  $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$ . 必须预处理 一次性排序.

## 2.7 快速排序

```
QUICKSORT(A, p, r):
    if p < r:
        q = PARTITION(A, p, r)
        QUICKSORT(A, p, q-1)
        QUICKSORT(A, q+1, r)

PARTITION(A, p, r):
    x = A[p]; i = p; j = r + 1
    loop:
        do j -= 1 while A[j] > x
        do i += 1 while A[i] < x
        if i < j: swap A[i], A[j]
        else: swap A[p], A[j]; return j
```

复杂度:

- 最坏  $W(n) = W(n-1) + O(n) = \Theta(n^2)$ ;
- 最好  $T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$ ;
- 平均 (输入随机)  $T(n) = \Theta(n \log n)$ .

平均推导 (差消法): 见 Lec 2 §1.3.

## 2.8 Select ——一般化选择第 $k$ 小

算法 Select( $S, k$ ) —Blum–Floyd–Pratt–Rivest–Tarjan, “中位数的中位数”:

1. 把  $S$  分成  $\lceil n/5 \rceil$  组, 每组 5 个
2. 每组排序, 取中位数, 共得  $n_M = \lceil n/5 \rceil$  个中位数, 放入  $M$
3.  $m^* = \text{Select}(M, \lceil n_M / 2 \rceil)$  # 中位数的中位数
4. 用  $m^*$  把  $S$  分成  $S_1 (< m^*)$ ,  $\{m^*\}$ ,  $S_2 (> m^*)$
5. if  $k == |S_1| + 1$ : return  $m^*$   
 elif  $k \leq |S_1|$ : return Select( $S_1, k$ )  
 else: return Select( $S_2, k - |S_1| - 1$ )

关键: 选  $m^*$  后, 至少  $\lceil n/10 \rceil$  个数  $< m^*$ , 至少  $\lceil n/10 \rceil$  个  $> m^*$  (因为有一半的组里中位数  $\leq m^*$ , 这些组里至少 3 个数  $\leq m^*$ ). 故  $|S_1|, |S_2| \leq \frac{7n}{10} + O(1)$ .

复杂度:

$$W(n) \leq W(n/5) + W(7n/10) + cn. \quad (3)$$

观察  $1/5 + 7/10 = 9/10 < 1$ : 递归树每层和按比例缩, 几何级数收敛,  $W(n) = \Theta(n)$ .

[EX] 数学归纳证  $W(n) \leq 20cn$

$$W(n) \leq W(n/5) + W(7n/10) + cn \leq 20c(n/5) + 20c(7n/10) + cn = 4cn + 14cn + cn = 19cn \leq 20cn.$$

## 2.9 Find Max / Find MaxMin / Find Second-Max

Find Max:  $W(n) = n - 1$ , 最优.

Find MaxMin (同时求最大最小): 两两分组, 组内比较, 大者进 max-pool 找 max, 小者进 min-pool 找 min.

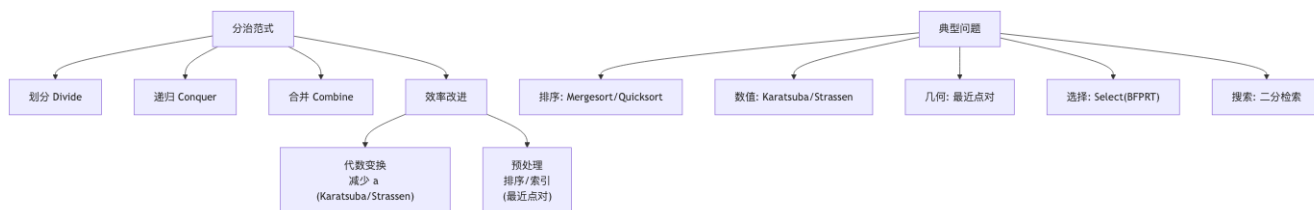
$$W(n) = \lceil n/2 \rceil + 2(\lceil n/2 \rceil - 1) = \lceil 3n/2 \rceil - 2.$$

锦标赛找第二大:

$$W(n) = (n - 1) + (\lceil \log n \rceil - 1) = n + \lceil \log n \rceil - 2.$$

理由:  $n - 1$  次比较得到最大, 然后第二大一定在被 max 直接淘汰过的  $\log n$  个数里, 再比较  $\log n - 1$  次.

## 概念关系



## 记号速查

| 符号       | 含义                  |
|----------|---------------------|
| $a, b$   | 主定理参数 (子问题数 / 缩小因子) |
| $W(n)$   | 最坏比较次数              |
| $T(n)$   | 时间复杂度               |
| $\delta$ | 最近点对当前最小距离          |
| $m^*$    | 中位数的中位数 (Select 主轴) |

## 易错点汇总

### [WARN] 分治算法常见坑

1. 划分必须 **平衡**—否则递归树深度变  $n$  而非  $\log n$ . 2. Karatsuba 中  $A, B, C, D$  是 **半规模整数**, 加减仍是  $O(n)$  不可忽略. 3. Strassen 7 个  $M_i$  必须记准号; 加减  $\Theta(n^2)$ . 4. 最近点对的  $\delta$  带子内必须按  $y$  排序后线性扫描, 不可两两比较. 5. Select 的分组大小 = 5 (不能改 3, 否则  $\frac{1}{3} + \frac{2}{3} = 1$ , 不收敛); 7 也可工作但常数变大. 6. 主定理情形对照表 (Lec 2 §4.2) 必备.

### [TIP] 期中复习要点

1. 五大例 (Karatsuba/Strassen/最近点对/Select/Quicksort) 算法 + 复杂度 + 递推必背. 2. 主定理与递推的对应: 每个算法都要能写出  $T(n) = aT(n/b) + f(n)$ . 3. Select 用“中位数的中位数”—为什么是 5? 因为  $1/5 + 7/10 < 1$  才能保证线性. 4. 锦标赛找第二大: 比较次数  $n + \lceil \log n \rceil - 2$ , 是最优的.

# Lecture 04 —动态规划 (Dynamic Programming)

## [TIP] 学习指南

本节核心: 把优化问题表示为 **多阶段决策**, 利用 **优化原则** 把全局最优拆解为子问题最优, 自底向上以 **备忘录** 避免重复计算. 前置知识: Lec 1–3. 重点关注: 矩阵链乘、LCS、投资问题、0/1 背包、最大子段和、OBST.

## 0. 适用条件

动态规划要求问题满足:

1. **优化原则 (Bellman)**: 全局最优解 任何子序列也是子问题的最优.
2. **子问题重叠**: 递归分治时同一子问题被多次计算 备忘录可节省工作量.

## [WARN] 反例 (优化原则不成立)

“总长度模 10 最小路径”: 从起点经过若干步到终点, 路径总长 mod 10 最小. 子路径的最优 mod 10 与全局最优 mod 10 没必然关联. 故 DP 不可用.

## 1. 设计步骤

1. 多步判断模型: 描述每步决策 + 子问题边界
2. 优化函数:  $f(\text{子问题}) = \text{最优值}$
3. 验证优化原则
4. 列出递推方程 + 边界条件
5. 自底向上填表 (避免重复)
6. 备忘录: 标记函数  $s[\dots]$  记录决策, 用于构造解

## 2. 实例 1: 矩阵链乘法

问题: 矩阵序列  $A_1, A_2, \dots, A_n$ ,  $A_i$  是  $P_{i-1} \times P_i$ . 选加括号方式使乘法总次数最少.

搜索空间: 加括号方案数 = Catalan 数  $\frac{1}{n} \binom{2n-2}{n-1} = \Omega(2^n/n^{3/2})$ , 蛮力指数级.

状态:  $m[i, j]$  = 计算  $A_i \cdots A_j$  的最少乘法次数.

递推方程:

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + P_{i-1}P_kP_j\} & i < j \end{cases} \quad (1)$$

标记函数:  $s[i, j] = k$  记录最优分割点.

伪代码 (自底向上):

```
MATRIX-CHAIN(P, n):
    for i = 1..n: m[i][i] = 0
    for r = 2..n:                                     # r = 链长
        for i = 1..n-r+1:
            j = i + r - 1
            m[i][j] = ∞
            for k = i..j-1:
                q = m[i][k] + m[k+1][j] + P[i-1]*P[k]*P[j]
                if q < m[i][j]:
                    m[i][j] = q
                    s[i][j] = k
    return m, s

PRINT-PARENS(s, i, j):
    if i == j: print "A_" + i
    else:
        print "("
        PRINT-PARENS(s, i, s[i][j])
        PRINT-PARENS(s, s[i][j]+1, j)
        print ")"
```

复杂度: 三重循环, 时间  $\Theta(n^3)$ , 空间  $\Theta(n^2)$ .

递归 (无备忘录) 的复杂度:

$$T(n) \geq \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) = 2 \sum_{k=1}^{n-1} T(k) + n,$$

归纳可证  $T(n) \geq 2^{n-1}$ . 指数!

[EX] 实例

$P = \langle 30, 35, 15, 5, 10, 20 \rangle, n = 5$ .

| $i \backslash j$ | 1 | 2     | 3    | 4    | 5     |
|------------------|---|-------|------|------|-------|
| 1                | 0 | 15750 | 7875 | 9375 | 11875 |
| 2                |   | 0     | 2625 | 4375 | 7125  |
| 3                |   |       | 0    | 750  | 2500  |
| 4                |   |       |      | 0    | 1000  |
| 5                |   |       |      |      | 0     |

最优解:  $(A_1(A_2A_3))(A_4A_5), m[1, 5] = 11875$  次乘法.

### 3. 实例 2: 最长公共子序列 (LCS)

问题:  $X = \langle x_1, \dots, x_m \rangle, Y = \langle y_1, \dots, y_n \rangle$ , 求最长公共子序列.

状态:  $C[i, j] = X_i$  和  $Y_j$  的 LCS 长度.

递推方程:

$$C[i, j] = \begin{cases} 0 & i = 0 \text{ 或 } j = 0 \\ C[i-1, j-1] + 1 & x_i = y_j \\ \max(C[i-1, j], C[i, j-1]) & x_i \neq y_j \end{cases} \quad (2)$$

标记函数  $B[i, j] \in \{\nwarrow, \uparrow, \leftarrow\}$  用于构造解.

伪代码:

```
LCS(X, Y, m, n):
  for i = 0..m: C[i][0] = 0
  for j = 0..n: C[0][j] = 0
  for i = 1..m:
    for j = 1..n:
      if X[i] == Y[j]:
        C[i][j] = C[i-1][j-1] + 1
        B[i][j] = "\nwarrow"
      elif C[i-1][j] >= C[i][j-1]:
        C[i][j] = C[i-1][j]
        B[i][j] = "\uparrow"
      else:
        C[i][j] = C[i][j-1]
        B[i][j] = "\leftarrow"

PRINT-LCS(B, X, i, j):
  if i == 0 or j == 0: return
  if B[i][j] == "\nwarrow": PRINT-LCS(B, X, i-1, j-1); print X[i]
  elif B[i][j] == "\uparrow": PRINT-LCS(B, X, i-1, j)
  else: PRINT-LCS(B, X, i, j-1)
```

复杂度: 时间  $\Theta(mn)$ , 空间  $\Theta(mn)$  (可优化到  $\Theta(\min(m, n))$  只求长度).

[EX] 实例

$X = ABCBDAB, Y = BDCABA$ , LCS = "BCBA"或"BDAB", 长度 4.

### 4. 实例 3: 投资问题

问题:  $m$  元投资  $n$  个项目,  $f_i(x)$  表示第  $i$  项投入  $x$  元的效益. 求最大总效益.

状态:  $F_k(x)$  = 用  $x$  元投到前  $k$  个项目的最大效益.

递推方程:

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\}, \quad F_1(x) = f_1(x). \quad (3)$$

复杂度: 表大小  $m \cdot n$ , 每个  $F_k(x)$  取  $x + 1$  种选择, 总

$$W(n) = \sum_{k=2}^n \sum_{x=0}^m (x+1) = \Theta(nm^2).$$

[EX] 实例:  $m=5, n=4$

题目给出  $f_i(x)$  表, 算法生成  $F_k(x)$ :

| $x$ | $F_1$  | $F_2$  | $F_3$  | $F_4$  |
|-----|--------|--------|--------|--------|
| 1   | 11 (1) | 11 (0) | 11 (0) | 20 (1) |
| 2   | 12 (2) | 12 (0) | 13 (1) | 31 (1) |
| 3   | 13 (3) | 16 (2) | 30 (3) | 33 (1) |
| 4   | 14 (4) | 21 (3) | 41 (3) | 50 (1) |
| 5   | 15 (5) | 26 (4) | 43 (4) | 61 (1) |

解:  $x_1 = 1, x_2 = 0, x_3 = 3, x_4 = 1$ , 总效益 = 61.

## 5. 实例 4: 0/1 背包

问题:  $n$  物品  $(w_i, v_i)$ , 容量  $W$ . 求最大总价值, 每物品取或不取.

状态:  $V[i, w]$  = 前  $i$  件物品、容量  $w$  时的最大价值.

递推:

$$V[i, w] = \begin{cases} 0 & i = 0 \\ V[i-1, w] & w_i > w \\ \max\{V[i-1, w], V[i-1, w-w_i] + v_i\} & w_i \leq w \end{cases} \quad (4)$$

复杂度:  $\Theta(nW)$  时间,  $\Theta(nW)$  空间. 伪多项式— $W$  是输入数值而非比特长度.

## 6. 实例 5: 最大子段和

问题: 数组  $A[1..n]$  (可有负), 求  $\max_{i \leq j} \sum_{k=i}^j A[k]$ .

Kadane DP: 令  $f[i]$  = 以  $A[i]$  结尾的最大子段和.

$$f[i] = \max(A[i], f[i-1] + A[i]), \quad f[0] = -\infty. \quad (5)$$

答案 =  $\max_i f[i]$ . 时间  $\Theta(n)$ , 空间  $\Theta(1)$ .

[EX] example

$A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$ .  $f = [-2, 1, -2, 4, 3, 5, 6, 1, 5]$ . 最大子段和 = 6, 子段  $[4, -1, 2, 1]$ .

## 7. 实例 6: 最优二叉检索树 (OBST)

问题: 关键字  $K_1 < K_2 < \dots < K_n$ , 查询概率  $p_i$  (失败概率  $q_i$  可选). 构造 BST 使加权平均搜索路径长度最小.

状态:  $e[i, j]$  = 关键字  $K_i \dots K_j$  的最优期望代价.

递推:

$$e[i, j] = \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w(i, j)\},$$

其中  $w(i, j) = \sum_{k=i}^j p_k$ . 边界  $e[i, i-1] = 0$ . 时间  $\Theta(n^3)$ .

[WARN] 优化

Knuth 优化:  $\Theta(n^2)$ , 利用 Knuth 的单调性  $r[i, j-1] \leq r[i, j] \leq r[i+1, j]$ .

## 8. 实例 7: 编辑距离 (Levenshtein)

状态  $d[i, j]$  = 字符串  $X_i$  变为  $Y_j$  的最少操作.

递推:

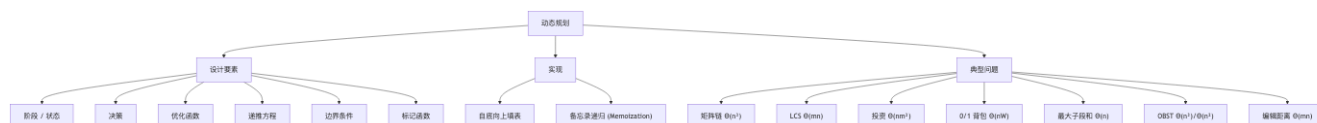
$$d[i, j] = \begin{cases} \max(i, j) & i = 0 \text{ 或 } j = 0 \\ d[i-1, j-1] & x_i = y_j \\ 1 + \min(d[i-1, j], d[i, j-1], d[i-1, j-1]) & x_i \neq y_j \end{cases} \quad (6)$$

时间  $\Theta(mn)$ .

## 9. DP vs 分治

| 维度   | 分治                   | DP                  |
|------|----------------------|---------------------|
| 子问题  | 独立 (无重叠)             | 重叠                  |
| 实现   | 递归                   | 自底向上填表 (或带备忘录递归)    |
| 优化原则 | 不要求                  | 必须                  |
| 工作量  | 递推方程定义               | 子问题数 $\times$ 每个的代价 |
| 典型例  | Mergesort, Karatsuba | 矩阵链, LCS            |

## 概念关系



## 记号速查

| 符号        | 含义                      |
|-----------|-------------------------|
| $m[i, j]$ | 矩阵链最小代价                 |
| $s[i, j]$ | 矩阵链最优分割点                |
| $C[i, j]$ | LCS 长度                  |
| $B[i, j]$ | LCS 方向标记                |
| $F_k(x)$  | 投资问题前 $k$ 项 $x$ 元最大效益   |
| $V[i, w]$ | 0/1 背包前 $i$ 件容量 $w$ 最大值 |
| $f[i]$    | 以 $i$ 结尾的最大子段和          |
| $e[i, j]$ | OBST 期望代价               |
| $d[i, j]$ | 编辑距离                    |

## 易错点汇总

### [WARN] DP 常见坑

1. 优化原则证明 是 DP 适用性的核心, 不能跳过.
2. 矩阵链 DP:  $r$  是 链长 而不是右端点, 必须按  $r$  升序填表.
3. LCS 边界:  $C[0, *] = C[*, 0] = 0$ ; 标记  $B[i, j]$  取最大时若两个相等, 任取其一即可.
4. 0/1 背包是 伪多项式—输入  $W$  的比特长度是  $\log W$ , 所以严格意义不是多项式. NP-Hard 问题.
5. 最大子段和:  $f[i]$  必须包含  $A[i]$ , 不要写成“前  $i$  个中的最大子段和”.
6. 备忘录递归与自底向上等价, 但前者常数更大 (函数调用).

### [TIP] 期中复习要点

1. 矩阵链 + LCS + 0/1 背包 必须能写完整递推 + 伪码 + 复杂度.
2. DP 5 件套: 状态/边界/转移/复杂度/标记函数 (解的构造).
3. 投资问题: 注意  $f_k(x_k) + F_{k-1}(x - x_k)$  而不是  $F_k(x - x_k)$ .
4. 最大子段和 Kadane  $\Theta(n)$ , 用差消把  $\Theta(n^2)$  降为线性.

# Lecture 05 —贪心算法 (Greedy)

## [TIP] 学习指南

本节核心: 每步根据 **局部最优策略** 作不可回退的决策, 期望达到全局最优. 前置知识: Lec 4 (DP) 一贪心是 DP 的特例 (子问题只需 1 个最优解). 重点关注: 活动选择、最小延迟调度、Huffman 编码、Kruskal/Prim MST、Dijkstra 单源最短路径.

## 0. 贪心 vs DP

| 维度   | 贪心                   | DP          |
|------|----------------------|-------------|
| 子问题  | 每步仅 1 个              | 多个          |
| 决策依赖 | 与之前结果无关              | 需子问题全部值     |
| 求解方向 | 自顶向下                 | 自底向上        |
| 正确性  | 必须证明 (归纳/交换)         | 优化原则即可      |
| 复杂度  | 通常 $O(n \log n)$ 或更低 | 多项式 (常高于贪心) |

## [WARN] 贪心不总是最优

0/1 背包按价值密度贪心  $\Rightarrow$  反例; 但分数背包 (分数可选) 按密度贪心 最优.

## 1. 活动选择问题

问题:  $n$  项活动  $S = \{1, \dots, n\}$ , 每项有  $s_i, f_i$  (开始/结束). 求最大两两相容的活动集.

几种排序策略:

1. 按  $s_i$  升序选不冲突—反例 (一个超长活动  $[0, 20]$  阻挡后续两个短活动)
2. 按  $f_i - s_i$  升序—反例 (两个短活动重叠在中间)
3. 按  $f_i$  升序—正确

贪心算法 (按  $f$  升序):

```
GREEDY-SELECT(S):
  sort S by f_i ascending
  A = {1}; j = 1
  for i = 2..n:
    if s_i >= f_j:
      A = A U {i}; j = i
  return A
```

复杂度: 排序  $O(n \log n)$  + 扫描  $O(n)$ .

正确性 (归纳法):

- 归纳基础: 存在最优解包含活动 1 (按  $f$  升序, 1 完成最早).
- 归纳步: 若前  $k$  步选  $i_1, \dots, i_k$  来自某最优解  $A = \{i_1, \dots, i_k\} \cup B$ , 其中  $B \subseteq S' = \{i : s_i \geq f_{i_k}\}$ . 那么  $B$  也是  $S'$  的最优解. 由归纳基础, 存在  $S'$  的最优解  $B'$  含  $i_{k+1}$  (第  $k+1$  个选的活动). 由  $|B'| = |B|$ ,  $\{i_1, \dots, i_k\} \cup B'$  也是  $S$  最优, 包含算法前  $k+1$  步选择.

## 2. 最优装载

问题: 集装箱  $n$  个, 重量  $w_i$ , 船容量  $C$ . 求最多装载件数.

贪心: 按  $w_i$  升序装. 证明: 任意最优解  $S^*$  若不是按升序前  $k$  个, 可交换论证.

复杂度  $O(n \log n)$ .

## 3. 最小延迟调度

问题:  $n$  个任务, 任务  $i$  有处理时间  $t_i$  和截止  $d_i$ . 单机非抢占, 一个任务在  $f_i = \sum_{j \leq i, \sigma(j) \leq \sigma(i)} t_j$  完成. 延迟  $l_i = \max(0, f_i - d_i)$ . 求最大延迟最小 (minmax lateness).

贪心 (Earliest Deadline First, EDF): 按  $d_i$  升序处理.

正确性 (交换论证): 若 EDF 顺序非最优, 存在相邻逆序  $i, j$  ( $d_i < d_j$  但  $i$  在  $j$  后), 交换后最大延迟不减 (经典证明).

复杂度: 排序  $O(n \log n)$ .

## 4. Huffman 编码

问题: 字符集  $C$ , 频率  $f(c)$ . 构造前缀码 (Prefix Code) 使总编码长度  $\sum f(c) \cdot \text{depth}(c)$  最小.

算法:

HUFFMAN(C):

```
Q = min-heap of C by frequency
for i = 1..|C|-1:
    z = new node
    z.left = x = EXTRACT-MIN(Q)
    z.right = y = EXTRACT-MIN(Q)
    z.freq = x.freq + y.freq
    INSERT(Q, z)
return EXTRACT-MIN(Q)          # 树根
```

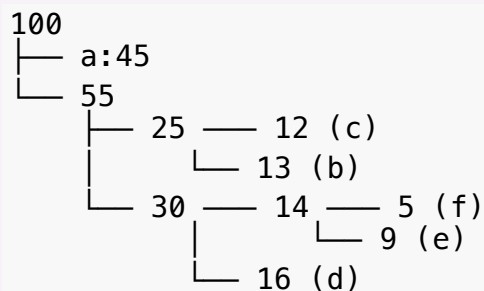
复杂度:  $O(n \log n)$  —  $n-1$  次 extract+insert, 每次  $O(\log n)$ .

正确性 (引理):

- 引理 1 (贪心选择): 设  $x, y$  是最低频两字符, 存在最优前缀码使得  $x, y$  在编码树最深、为兄弟.
- 引理 2 (优化原则): 设  $x, y$  最低频, 令  $z$  代替,  $f(z) = f(x) + f(y)$ , 字典  $C' = C - \{x, y\} + \{z\}$ . 若  $T'$  是  $C'$  的最优树, 则把  $z$  替换为内部节点 (子叶  $x, y$ ) 得到的  $T$  是  $C$  的最优树.

### [EX] 实例

字符  $a : 45, b : 13, c : 12, d : 16, e : 9, f : 5$ .



编码  $a = 0, b = 101, c = 100, d = 111, e = 1101, f = 1100$ . 总比特 =  $45 + 39 + 36 + 48 + 36 + 20 = 224$ .

## 5. 最小生成树

### 5.1 Kruskal 算法

```
KRUSKAL( $G = (V, E, w)$ ):
  sort  $E$  by  $w$  ascending
   $T = \emptyset$ 
  init union-find over  $V$ 
  for each  $(u, v)$  in sorted  $E$ :
    if  $\text{FIND}(u) \neq \text{FIND}(v)$ :
       $T = T \cup \{(u, v)\}$ 
       $\text{UNION}(u, v)$ 
  return  $T$ 
```

复杂度: 排序  $O(m \log m)$  + 并查集  $O(m \alpha(n))$  (路径压缩 + 按秩合并) =  $O(m \log n)$ .

正确性: 数学归纳.

- $n = 2$ : 单边显然最优.
- 设对  $n$  顶点正确. 考虑  $n + 1$  顶点图  $G$ , 最小权边  $e = (i, j)$ . 短接  $i, j$  得  $G'$  ( $n$  顶点), 由归纳假设 Kruskal 在  $G'$  上正确. 令  $T = T' \cup \{e\}$ . 假设存在含  $e$  的最优树  $T^*$  比  $T$  更轻. 在  $T^*$  中短接  $e$  得  $G'$  的生成树  $T^* - \{e\}$ ,  $W(T^* - \{e\}) < W(T')$ , 矛盾.

### 5.2 Prim 算法

```
PRIM( $G, s$ ):
  for  $v \in V$ :  $\text{key}[v] = \infty$ ;  $\pi[v] = \text{nil}$ 
   $\text{key}[s] = 0$ 
   $Q = \text{min-priority-queue of } V \text{ by key}$ 
  while  $Q \neq \emptyset$ :
     $u = \text{EXTRACT-MIN}(Q)$ 
    for each  $(u, v) \in E$ :
      if  $v \in Q$  and  $w(u, v) < \text{key}[v]$ :
         $\pi[v] = u$ 
         $\text{DECREASE-KEY}(Q, v, w(u, v))$ 
```

复杂度: 二叉堆  $O((n + m) \log n) = O(m \log n)$ . Fibonacci 堆  $O(m + n \log n)$ .

## 6. Dijkstra 单源最短路径

问题: 加权有向图  $G = (V, E, w)$ ,  $w \geq 0$ , 源  $s \in V$ . 求  $s$  到所有  $v$  的最短距离.

算法:

```
DIJKSTRA( $G, s$ ):  
   $S = \{s\}; \text{dist}[s] = 0$   
  for  $v \in V - \{s\}$ :  
     $\text{dist}[v] = w(s, v)$       # 若无边则  $+\infty$   
  while  $V - S \neq \emptyset$ :  
     $j = \text{argmin}_{\{v \in V-S\}} \text{dist}[v]$   
     $S = S \cup \{j\}$   
    for  $v \in V - S$ :  
      if  $\text{dist}[j] + w(j, v) < \text{dist}[v]$ :  
         $\text{dist}[v] = \text{dist}[j] + w(j, v)$ 
```

复杂度:

- 朴素  $O(n^2)$ ;
- 二叉堆  $O((n + m) \log n)$ ;
- Fibonacci 堆  $O(m + n \log n)$ .

正确性: 数学归纳, 关键引理: 当顶点  $v$  被加入  $S$  时,  $\text{dist}[v]$  已经是  $s$  到  $v$  的真实最短.

### [WARN] 易错点

Dijkstra 不能处理负权边. 用 Bellman-Ford ( $O(nm)$ ) 或 Johnson 算法.

### [EX] 实例 (6 顶点图)

边:  $(1, 2) = 10, (1, 6) = 3, (2, 4) = 1, (2, 3) = 2, (6, 2) = 2, (6, 5) = 4, (5, 2) = 7, (5, 3) = 8, (4, 3) = 4, (4, 5) = 2$ .

起始:  $S = \{1\}, \text{dist} = [0, 10, \infty, \infty, \infty, 3]$ .

步 1: 选 6, 更新  $\text{dist} = [0, 5, \infty, \infty, 4, 3], S = \{1, 6\}$ . 步 2: 选 5, 更新  $\text{dist} = [0, 5, 12, 6, 4, 3]$  (经  $5 \rightarrow 3$  得 12, 经  $5 \rightarrow 4$  得 6? 注意原题边稍异). ...最终 short =  $[0, 5, 12, 9, 4, 3]$ .

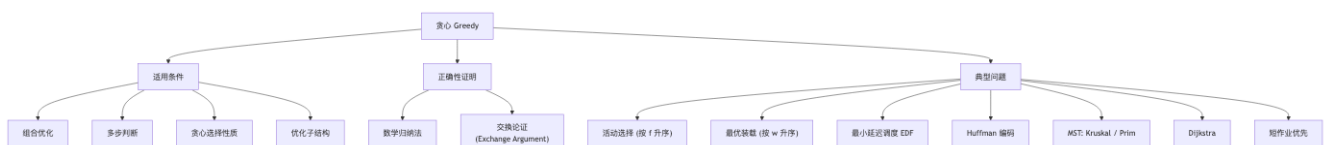
## 7. 单段调度问题 (Single-machine scheduling)

问题:  $n$  任务等长 1, 第  $i$  加工时间  $t_i \in \mathbb{Z}^+$ . 找排列使总等待时间  $\sum_{k=1}^n (n - k + 1) t_{i_k}$  最小.

贪心: 短作业优先 (Shortest Job First). 排序  $O(n \log n)$ .

证明: 交换论证. 若存在逆序  $t_a > t_b$  但  $a$  在  $b$  之前, 交换不增加目标值.

## 概念关系



## 记号速查

| 符号               | 含义                      |
|------------------|-------------------------|
| $S$              | 已确定集合 (Dijkstra/Prim)   |
| $\text{dist}[v]$ | 当前已知 $s$ 到 $v$ 距离       |
| $\text{key}[v]$  | Prim 中 $v$ 到 MST 的最小边权  |
| $\pi[v]$         | 父节点                     |
| $f(c)$           | Huffman 字符频率            |
| EDF              | Earliest Deadline First |

## 易错点汇总

### [WARN] 贪心常见坑

1. 不证明就用贪心—必丢分。一定附正确性证明 (归纳法或交换论证)。2. 反证错例: 0/1 背包不能贪心 (按价值密度反例)。分数背包可以。3. Dijkstra **不支持负权**。出现负权用 Bellman-Ford。4. Huffman 必须用 **最小堆**, 否则每步  $O(n)$ , 总  $O(n^2)$ 。5. Kruskal 必须用 **并查集**, 否则环判定为  $O(n)$ , 总变  $O(mn)$ 。

### [TIP] 期中复习要点

1. 活动选择: 三种排序策略, 只有按  $f$  升序正确。反例必能举出。2. Huffman: 引理 1 (兄弟在最深) + 引理 2 (合并/拆分) 必背。3. Kruskal/Prim 复杂度差异: 稠密图 (Prim 朴素  $O(n^2)$ ) vs 稀疏图 (Kruskal  $O(m \log n)$ )。4. Dijkstra 正确性证明草图 (取  $V - S$  中最近顶点, 反证)。5. 区分贪心 vs DP 一见 §0 表。

# Lecture 06 —回溯 (Backtracking) 与分支限界 (Branch and Bound)

## [TIP] 学习指南

本节核心: 在搜索树上系统地遍历可能的解, 通过约束剪枝避免穷举. 前置知识: 树/图遍历 (DFS/BFS). 重点关注: n 后问题、0/1 背包、TSP、最大团、圆排列; Monte Carlo 估算搜索树规模.

## 0. 基本思想

回溯算法用 **搜索树** 隐式表示解空间. 节点对应部分解向量  $\langle x_1, x_2, \dots, x_k \rangle$ ; 叶子对应可行解.

搜索策略: DFS / BFS / 优先函数 (best-first) / 宽深结合.

节点状态: 白 (未访问) / 灰 (正在访问以它为根的子树) / 黑 (子树遍历完毕).

适用条件 (多米诺性质):

$$P(x_1, \dots, x_k) = 1 \Rightarrow P(x_1, \dots, x_{k-1}) = 1.$$

即部分解可行蕴含其前缀可行 (前缀闭合).

## [WARN] 反例

“ $5x_1 + 4x_2 - x_3 \leq 10, 1 \leq x_i \leq 3$ ”. 由于  $-x_3$  可以减小 LHS, 前缀和大不一定无解. 需要变量替换  $x'_3 = 3 - x_3$  化成  $5x_1 + 4x_2 + x'_3 \leq 13$ , 此时多米诺成立.

## 1. 通用回溯框架

```
RE-BACK(k):
    if k > n: <x_1, ..., x_n> 是解; record/print
    else:
        while S_k ≠ ∅:
            x_k = pick(S_k)
            S_k = S_k - {x_k}
            compute S_{k+1}      # 依赖 x_1..x_k 的可行集
            RE-BACK(k+1)

RE-BACKTRACK(n):
    compute X_1, ..., X_n
    RE-BACK(1)
```

迭代版:

```

BACKTRACK:
  for i = 1..n: compute X_i
  k = 1; compute S_1
  while True:
    while S_k ≠ ∅:
      x_k = min(S_k); S_k -= {x_k}
      if k < n: k += 1; compute S_k
      else: record solution
    if k > 1: k -= 1
    else: break

```

## 2. 经典实例

### 2.1 n 皇后

解向量:  $\langle x_1, \dots, x_n \rangle, x_i \in \{1, \dots, n\}$  表示第  $i$  行皇后的列号.

约束:

1.  $x_i \neq x_j$  (不同列);
2.  $|x_i - x_j| \neq |i - j|$  (不同斜线).

搜索树:  $n$  叉树, 叶子  $n!$  个 (考虑列约束后).

```

N-QUEENS(k):
  if k > n: 输出 <x_1..x_n>
  else:
    for c = 1..n:
      if check(c):
        x_k = c; N-QUEENS(k + 1)

```

# 满足列与斜线约束

复杂度: 最坏  $O(n!)$  一不过实际剪枝后远低. 4 后只展开 26 个节点 (而非  $4^4 = 256$ ).

### 2.2 0/1 背包

解向量:  $\langle x_1, \dots, x_n \rangle, x_i \in \{0, 1\}$ . 搜索树为二叉树 (子集树),  $2^n$  叶子.

约束:  $\sum w_i x_i \leq B$ .

剪枝 (代价函数): 计算当前部分解的最优可达上界:

$$\text{bound}(k) = \sum_{i \leq k} v_i x_i + \sum_{i > k} v_i. \quad (1)$$

若  $\text{bound}(k) \leq \text{best}$ , 剪枝.

#### [EX] 实例

$V = \{12, 11, 9, 8\}, W = \{8, 6, 4, 3\}, B = 13$ . 解  $\langle 0, 1, 1, 1 \rangle$  重量 13, 价值 28 (最优).

### 2.3 货郎问题 (TSP)

解向量: 1 到  $n$  的一个排列  $\langle 1, i_2, \dots, i_n \rangle$  (固定 1 为起点). 搜索树是排列树, 叶子  $(n - 1)!$ .

剪枝: 上界 = 已走路径 + 剩余顶点最小入边和 + 回到 1 的下界.

复杂度: 最坏  $O((n - 1)!)$ , 实际剪枝后大幅减少.

## 2.4 圆排列问题

问题: 给定  $n$  个圆的半径  $r_1, \dots, r_n$ , 用一根直线托起所有圆, 求最短直线段长度.

关键: 相邻两圆  $r_i, r_j$  中心水平距离  $d_{ij} = 2\sqrt{r_i r_j}$  (相切条件).

解向量:  $1..n$  的排列, 表示圆从左到右的顺序.

代价函数 (用于 Branch-and-Bound):

$$l_n = 2(r_1 + r_2 + \dots + r_n) + \dots \quad (\text{基于半径排序最坏近似}).$$

[EX] example

$R = \{1, 1, 2, 2, 3, 5\}$ , 取顺序  $\langle 1, 2, 3, 4, 5, 6 \rangle$  (按半径升序), 算得  $l_6 = 27.4$ .

## 2.5 最大团

问题: 图  $G = (V, E)$ , 求最大完全子图  $C \subseteq V$ .

回溯: 解向量  $\langle x_1, \dots, x_n \rangle$ ,  $x_i \in \{0, 1\}$ . 进入  $x_k = 1$  前检查  $k$  与已选顶点都相邻.

剪枝: 当前团大小 + 剩余顶点数  $\leq$  当前最优, 剪枝.

复杂度: 最坏  $O(n \cdot 2^n)$ .

## 3. 搜索树规模估计—Monte Carlo

MONTE-CARLO:

从根开始随机走一条路径, 直到不可分支

记录每个  $x_i$  的  $|S_i|$  与路径深度

估计总节点数:  $1 + |S_1| + |S_1||S_2| + |S_1||S_2||S_3| + \dots$

重复多次取平均

复杂度: 单次  $O(n)$ . 平均后给出可靠估计.

## 4. 分支限界 (Branch and Bound, BB)

DFS 的变种, 但每个节点配 优先函数 (best-first) 或 代价函数下/上界, 维护:

1. 当前最优解的目标值  $B$ ;
2. 活节点表 (queue / priority queue), 每节点附其代价下界.

B&B:

```
init: root, lowerBound(root)
```

```
active = {root}
```

```
best =  $+\infty$ 
```

```
while active  $\neq \emptyset$ :
```

```
    pick node  $v$  with smallest lowerBound from active    # best-first
```

```
    if  $v$  is leaf:
```

```
        if  $\text{cost}(v) < \text{best}$ :  $\text{best} = \text{cost}(v)$ 
```

```
    else:
```

```
        for each child  $u$ :
```

```
            if  $\text{lowerBound}(u) < \text{best}$ :
```

```
                active = active +  $\{u\}$ 
```

```
# 否则剪枝
return best
```

示例: 0/1 背包用 LP 松弛上界做剪枝.

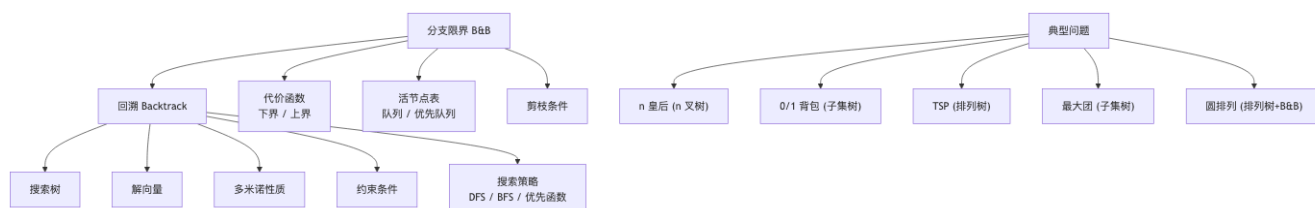
## 5. 代价函数 (圆排列 BB)

对前  $k$  个圆已确定的部分排列, 设当前线段长度  $x_k$ , 剩余圆排成线段下界:

$$L_k = x_k + r_k + r_{i_n} + 2(r_{i_n} + r_{i_{n-1}} + \dots + r_{i_{k+1}}) + r_{i_{k+1}}.$$

若  $L_k \geq B$ , 剪枝.

## 概念关系



## 记号速查

| 符号                           | 含义                                     |
|------------------------------|----------------------------------------|
| $X_i$                        | $x_i$ 可能取值集合                           |
| $S_k$                        | 给定 $x_1, \dots, x_{k-1}$ 时 $x_k$ 的可行集合 |
| $\langle x_1, \dots \rangle$ | 解向量                                    |
| bound                        | 代价函数下/上界                               |
| LB / UB                      | 下界 / 上界                                |

## 易错点汇总

### [WARN] 回溯常见坑

1. 必须验证 **多米诺性质**. 否则前缀可行但延伸全部不可行, 算法可能错过解. 2. 0/1 背包要写解向量取 0/1, 不是物品序号. 3. 复杂度评估: 子集树  $2^n$ , 排列树  $n!$ . 别写反. 4. Monte Carlo 估算: 不是真实复杂度, 仅用于工程估计.

### [TIP] 期中复习要点

1. n 后、子集和、0/1 背包 三种典型搜索空间 (n 叉/子集树/排列树). 2. 代价函数定义清晰: 下界 (min 问题), 上界 (max 问题). 3. Monte Carlo 算法步骤要会写. 4. 分支限界与回溯区别: BB 用 best-first + bound 剪枝, 回溯是 DFS.

# Lecture 07 —线性规划 (Linear Programming)

## [TIP] 学习指南

本节核心: 线性目标函数 + 线性约束的优化问题. 多项式时间可解 (内点法). 前置知识: 线性代数 (秩、矩阵求逆). 重点关注: 标准形、单纯形法、对偶定理、整数线性规划的分支限界.

## 0. 线性规划模型

### 0.1 一般形式

$$\begin{aligned} \min(\max) z &= \sum_{j=1}^n c_j x_j \\ \text{s.t. } \sum_{j=1}^n a_{ij} x_j &\leq (\geq, =) b_i, & i = 1, \dots, m \\ x_j &\geq 0 \quad (j \in J \subseteq \{1, \dots, n\}); \quad x_j \text{ 任意 } (j \notin J) \end{aligned}$$

可行解 / 可行域 / 最优解 / 最优值.

### 0.2 典型模型

#### [EX] 生产计划

3 种原料 (存量 120/150/50), 配 A (单价 12) 和 B (单价 15).

$$\max z = 12x + 15y \text{ s.t. } 0.25x + 0.5y \leq 120, 0.5x + 0.5y \leq 150, 0.25x \leq 50, x, y \geq 0.$$

几何解: 在凸多边形顶点取最优. 解  $(x^*, y^*) = (120, 180)$ ,  $z^* = 4140$ .

#### [EX] 投资组合

5 个项目, 高新技术 (1,2) + 基础工业 (3,4) + 债券 (5), 投资 10 亿.

#### [EX] 运输问题

2 个产地, 3 个销地. min 总运费.

## 0.3 解的几种情况

1. 唯一最优解;
2. 无穷多最优解 (整条边);
3. 有可行解但目标无界 (没有最优解);
4. 无可行解.

几何: 可行域是凸多边形 (可能无界、空集). 若有最优解, 必在顶点处取得.

## 1. 标准形

$$\begin{aligned} \min z &= c^T x \\ \text{s.t. } Ax &= b, \quad b \geq 0 \\ x &\geq 0 \end{aligned}$$

化标准形步骤:

1.  $\max z \Rightarrow \min(-z)$ ;
2.  $b_i < 0$  时, 不等式两边同乘变号;
3.  $\sum a_{ij}x_j \leq b_i \Rightarrow +y_i = b_i$  (松弛变量);
4.  $\sum a_{ij}x_j \geq b_i \Rightarrow -y_i = b_i$  (剩余变量);
5. 自由变量  $x_j$  替换为  $x'_j - x''_j, x'_j, x''_j \geq 0$ .

[EX] 化标准形

$\max z = 3x_1 - 2x_2 + x_3, x_1 + 3x_2 - 3x_3 \leq 10, 4x_1 - x_2 - 5x_3 \geq -30, x_1 \geq 0, x_2 \leq 0, x_3$  任意. 替换  $x_2 = -x'_2, x_3 = x'_3 - x''_3$ . 加松弛  $x_4 \geq 0$  与剩余  $x_5 \geq 0$ :

$$\begin{aligned} \min z &= -3x_1 + 2x'_2 - x'_3 + x''_3 \\ x_1 + 3(-x'_2) - 3(x'_3 - x''_3) + x_4 &= 10 \\ -4x_1 + x'_2 + 5(x'_3 - x''_3) - x_5 &= 30 \\ x_1, x'_2, x'_3, x''_3, x_4, x_5 &\geq 0 \end{aligned}$$

## 2. 基本可行解 (BFS)

设  $A$  秩  $m$  ( $m \leq n$ ). 取  $A$  的  $m$  个线性无关列构成基  $B$ .

- 对应的变量称 **基变量**  $x_B$ , 其余 **非基变量**  $x_N$ .
- 令  $x_N = 0$ , 由  $Bx_B = b$  解得  $x_B = B^{-1}b$ . 称  $x$  是 **基本解**.
- 若  $x_B \geq 0$ , 则  $x$  是 **基本可行解 (BFS)**,  $B$  是 **可行基**.

引理 1:  $Ax = b$  的解  $x$  是基本解  $\Leftrightarrow x$  的非零分量对应的列向量线性无关.

定理 1: 标准形有可行解 有基本可行解. 定理 2: 标准形有最优解 有基本可行解是最优解.

### [EX] example

生产计划标准形:

$$\begin{aligned}\min z &= -12x_1 - 15x_2 \\ 0.25x_1 + 0.5x_2 + x_3 &= 120 \\ 0.5x_1 + 0.5x_2 + x_4 &= 150 \\ 0.25x_1 + x_5 &= 50\end{aligned}$$

基  $B_1 = (P_1, P_2, P_3)$ :  $x_4 = x_5 = 0$ , 解得  $x_1 = 200, x_2 = 100, x_3 = 20$ .  $x^{(1)} = (200, 100, 20, 0, 0)^T$  是 BFS. 基  $B_2 = (P_1, P_2, P_4)$ : 解出  $x_4 = -20 < 0$ , 不是 BFS.

## 3. 单纯形法 (Simplex)

基本步骤:

1. 找到初始 BFS.
2. 最优性检验: 若是最优 (或证明无界) 则结束. 否则进入 4.
3. (再次循环);
4. **基变换**: 用一个非基变量替换一个基变量, 得新可行基, 目标函数 **至少不升** (求 min).

### 3.1 检验数 (reduced cost)

把  $A = [B \mid N]$ ,  $x = [x_B; x_N]$ . 由  $x_B = B^{-1}b - B^{-1}Nx_N$ , 代入目标:

$$z = c_B^T B^{-1}b + (c_N^T - c_B^T B^{-1}N)x_N.$$

令  $\bar{c}_N^T = c_N^T - c_B^T B^{-1}N$  为非基变量 **检验数**. 若  $\bar{c}_N \geq 0$  (对 min), 当前 BFS 最优.

### 3.2 入基变量与出基变量

- **入基**: 选  $\bar{c}_j < 0$  的非基  $j$  (通常取最负).
- **出基** (最小比率检验): 设  $d = B^{-1}A_j$ ,  $x_B = x_B^{(0)} - \theta d$ . 取

$$\theta = \min_{d_i > 0} \frac{x_{B,i}^{(0)}}{d_i}.$$

使  $\theta = x_{B,r}^{(0)}/d_r$  的  $r$  出基.

若所有  $d \leq 0$ , 沿入基方向目标无界.

### 3.3 单纯形表 (Tableau)

|     | $x_1$     | $\dots$ | $x_n$ | $b$    |
|-----|-----------|---------|-------|--------|
| 基行  | $A$       |         |       | $b$    |
| 检验数 | $\bar{c}$ |         |       | $-z_0$ |

每次 pivot: 主元行除主元, 其他行减去倍数. 与 Gauss 消元相同.

### [EX] 一次 pivot

选  $x_2$  入基 (最负  $\bar{c}_2 = -15$ ). 最小比率检验出基  $x_3$ . Pivot 后更新 BFS. 重复直到  $\bar{c} \geq 0$ . 最终  $x^* = (120, 180, 0, 0, 20)^T$ ,  $z^* = -4140$ , 即原 max 的 4140.

## 3.4 初始 BFS —Big-M / 两阶段

若约束没有显式松弛变量给出单位矩阵的初始基, 需引入 人工变量  $y_i \geq 0$  加到约束的 LHS, 改目标:

$$\min z + M \sum_i y_i, \quad M \rightarrow +\infty.$$

若最优时  $y_i > 0$ , 原问题无可行解; 若  $y_i = 0$ , 删除  $y_i$  列得原问题 BFS.

两阶段: 阶段 1 minimize  $\sum y_i$ , 阶段 2 用阶段 1 BFS 起步, minimize 原目标.

## 3.5 复杂度

最坏  $O(2^n)$  (Klee-Minty 立方体). 实际多项式 (Dantzig 经验).

多项式算法: Khachiyan 椭球法, Karmarkar 内点法 ( $O(n^{3.5})$ ).

## 4. 对偶规划

### 4.1 原始 (Primal) 与对偶 (Dual)

| 原始 (P)       | 对偶 (D)         |
|--------------|----------------|
| $\max c^T x$ | $\min b^T y$   |
| $Ax \leq b$  | $A^T y \geq c$ |
| $x \geq 0$   | $y \geq 0$     |

定理 4: 对偶的对偶 = 原始.

### 4.2 弱对偶定理 (定理 5)

若  $x$  是 P 的可行解,  $y$  是 D 的可行解, 则

$$c^T x \leq b^T y. \quad (2)$$

证:  $c^T x \leq (A^T y)^T x = y^T (Ax) \leq y^T b = b^T y$ .

### 4.3 强对偶定理 (定理 7)

若 P 有最优解, 则 D 有最优解, 且  $c^T x^* = b^T y^*$ . 反之亦然.

### 4.4 对偶的几种情形

| P vs D   | 有最优解    | 有可行 + 无界 | 无可行解 |
|----------|---------|----------|------|
| 有最优解     | (1) 同最优 | ×        | ×    |
| 有可行 + 无界 | ×       | ×        | (2)  |
| 无可行解     | ×       | (2)      | (3)  |

## 4.5 一般形式的对偶规则

| Primal max   | Dual min     |
|--------------|--------------|
| $\leq b_i$   | $y_i \geq 0$ |
| $= b_i$      | $y_i$ 任意     |
| $\geq b_i$   | $y_i \leq 0$ |
| $x_j \geq 0$ | $\geq c_j$   |
| $x_j$ 任意     | $= c_j$      |
| $x_j \leq 0$ | $\leq c_j$   |

### [EX] 对偶

$\max 2x_1 - x_2 + 3x_3, x_1 + 3x_2 - 2x_3 \leq 5, -x_1 - 2x_2 + x_3 = 8, x_1, x_2 \geq 0, x_3$  任意.

对偶:

$\min 5y_1 + 8y_2, y_1 - y_2 \geq 2, 3y_1 - 2y_2 \geq -1, -2y_1 + y_2 = 3, y_1 \geq 0, y_2$  任意.

## 4.6 互补松弛 (Complementary Slackness)

若  $x^*, y^*$  分别是 P 与 D 的最优解, 则:

1.  $x_j^* > 0 \Rightarrow$  D 的第  $j$  约束为等式;
2. P 的第  $i$  约束不等式严格成立  $\Rightarrow y_i^* = 0$ .

## 5. 整数线性规划 (ILP) 与分支限界

ILP: 线性规划 + 变量必须取整数. 纯整数 / 混合整数 / 0-1 整数. 松弛 (LP relaxation): 删除整数约束.

性质: LP 松弛最优值是 ILP 最优值的 界 (min 问下界, max 问上界).

### 5.1 分支限界算法

B&B(ILP):

```

solve LP relaxation -> x_LP
if x_LP integer: return x_LP          # 也是 ILP 最优
pick x_i non-integer in x_LP
branch into:
    LP_left: add x_i <= [x_LP_i]
    LP_right: add x_i >= [x_LP_i]
recursively solve, prune if LP_-'s optimum worse than current best

```

## [EX] 实例

$$\min z = -3x - 5y, \quad -x + y \leq \frac{3}{2}, \quad 2x + 3y \leq 11, \quad x, y \geq 0, \text{ 整数.}$$

LP 松弛解  $x = 13/10, y = 14/5, z = -179/10$ . 在  $x$  分支:  $x \leq 1$  LP1 解  $x = 1, y = 5/2, z = -31/2$  (仍非整数), 继续分支... $x \geq 2$  LP2 解  $x = 2, y = 7/3, z = -53/3$ , 继续... 最终得 ILP 最优解  $x^* = 4, y^* = 1, z^* = -17$ .

## 5.2 应用: 最小顶点覆盖

问题: 图  $G = (V, E)$ , 顶点子集  $S \subseteq V$ , 每条边至少一端在  $S$ . 求  $|S|$  最小.

ILP:

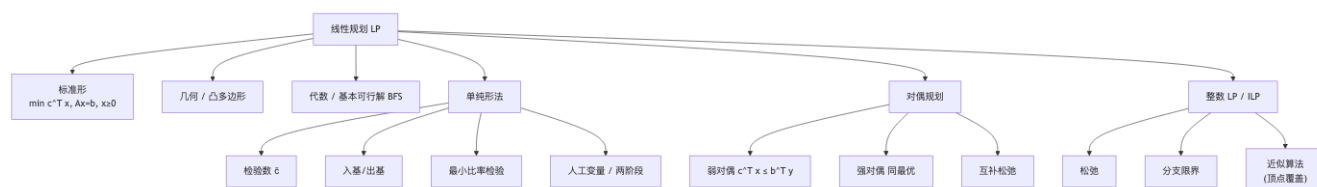
$$\min \sum_{i \in V} x_i, \quad x_i + x_j \geq 1 \quad \forall (i, j) \in E, \quad x_i \in \{0, 1\}.$$

LP 松弛 + 取整 (近似算法):

1. 解 LP 松弛得  $x_i \in [0, 1]$ ;
2. 令  $S = \{i : x_i \geq 1/2\}$ .

可证  $|S| \leq 2|S^*|$  (近似比 2). 顶点覆盖是 NP-Hard 问题.

## 概念关系



## 记号速查

| 符号               | 含义                 |
|------------------|--------------------|
| $x_B, x_N$       | 基变量, 非基变量          |
| $\bar{c}$        | 检验数 (reduced cost) |
| $B^{-1}$         | 基的逆                |
| Big-M            | 大 $M$ 罚法           |
| LP, ILP          | 线性 / 整数线性规划        |
| $S^*$            | 最优解集               |
| $\alpha$ -approx | 近似比 $\alpha$ 的算法   |

## 易错点汇总

### [WARN] LP 常见坑

1. 化标准形时,  $b_i$  必须  $\geq 0$ , 否则两边变号.
2. 自由变量必须 **拆成两个非负变量**  $x = x' - x''$ , 不能直接保留.
3. 单纯形检验数:  $\bar{c}_j = c_j - c_B^T B^{-1} A_j$ , 对 min 问题要求  $\bar{c} \geq 0$  最优.
4. 单纯形可能循环 (Bland 法则: 字典序最小, 防退化循环).
5. 对偶问题方向: max  $\rightarrow$  min 互转, 约束类型与变量符号要按 §4.5 表对应.
6. ILP 分支限界, 不要错过任何子问题; 剪枝时只删 LP 值劣于 best 的子问题.

### [TIP] 期中复习要点

1. 标准形化、BFS 概念、单纯形 pivot 必备.
2. 弱/强对偶定理证明.
3. ILP 分支限界树画图.
4. 顶点覆盖近似—2-approx 是经典.

# Lecture 08 —平摊分析 (Amortized Analysis)

## [TIP] 学习指南

本节核心: 一个数据结构上  $n$  次操作的 **总成本** 上界, 平均到每次操作得到 **平摊代价**, 即便单次操作最坏可能很贵. 前置知识: 栈/队列/二叉树/数组. 重点关注: 三种方法 (聚集 / 记账 / 势能), 经典例 (栈 multipop / 二进制计数器 / 动态表).

## 0. 概念

平摊分析 不同于:

- 平均情况分析 (基于输入概率): 平摊分析 **无随机性**, 是最坏情况  $n$  次操作总和 /  $n$ ;
- 最坏情况分析 (单次最坏): 平摊给出更紧的平均最坏.

三种方法:

1. 聚集分析 (Aggregate):  $n$  次操作总和  $T(n)$ , 每次平摊  $= T(n)/n$ . 所有操作平摊代价相同.
2. 记账法 (Accounting): 给每次操作分配“存款”, 多收的钱用于补贴贵操作.
3. 势能法 (Potential): 定义势函数  $\Phi$ , 平摊代价  $\$ = \$ \text{实际} + \Delta\Phi$ .

## 1. 聚集分析—栈操作

栈操作:  $\text{PUSH}(S, x)$ ,  $\text{POP}(S)$ ,  $\text{MULTIPOP}(S, k)$ .

- 单次 PUSH / POP:  $O(1)$ .
- $\text{MULTIPOP}(S, k)$ : 最坏  $O(\min(k, |S|))$ .

朴素分析:  $n$  次操作每次  $O(n)$ , 总  $O(n^2)$ . 太松.

聚集: 每个元素至多被 PUSH 一次, 因此 POP/MULTIPOP 的总弹出次数  $\leq$  PUSH 总次数  $\leq n$ . 故总成本  $O(n)$ , 平摊  $O(1)$ .

## 2. 聚集分析—二进制计数器

操作: INCREMENT 让  $k$ -bit 二进制计数器  $+1$ .

INCREMENT(A):

```
i = 0
while i < length(A) and A[i] == 1:
    A[i] = 0; i += 1
if i < length(A): A[i] = 1
```

单次最坏  $O(k)$  (全 1 翻成 0).

聚集: 第  $i$  位每  $2^i$  次操作翻转一次.  $n$  次操作中, 位翻转总数:

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \leq n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n. \quad (1)$$

故总  $O(n)$ , 平摊  $O(1)$ .

### 3. 记账法

为每次操作设定 平摊代价  $\hat{c}_i$ , 与实际代价  $c_i$  的差  $\hat{c}_i - c_i$  存入数据结构 (作为存款). 约束: 任何时刻 总存款  $\geq 0$ .

#### 3.1 栈操作记账

| 操作       | 实际代价         | 平摊代价            |
|----------|--------------|-----------------|
| PUSH     | 1            | 2 (多收 1 元给入栈对象) |
| POP      | 1            | 0 (用对象上的存款)     |
| MULTIPOP | $\min(k, s)$ | 0               |

任意时刻栈中对象数  $\geq 0$ , 故存款不为负. 总平摊  $\leq 2n$ .

#### 3.2 二进制计数器记账

每次 INCREMENT 平摊代价 2:

- 1 元用于把某位 0 翻成 1;
- 1 元存在新变成 1 的位上, 备将来翻 0 时使用.

数组中 1 的个数始终  $\geq 0$  (不变量), 总平摊  $\leq 2n$ .

[WARN] 注意

记账的关键是 不变量: 存款总数 = 数据结构中“留存”的某种量  $\geq 0$ .

### 4. 势能法 (Potential Method)

定义势函数  $\Phi: \mathcal{D} \rightarrow \mathbb{R}$ , 平摊代价

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}). \quad (2)$$

求和:

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0). \quad (3)$$

条件: 若  $\Phi(D_n) \geq \Phi(D_0)$  (通常取  $\Phi(D_0) = 0$  且  $\Phi \geq 0$ ), 则  $\sum \hat{c}_i \geq \sum c_i$ , 平摊给出实际的上界.

## 4.1 栈操作势能

势  $\Phi(D_i) = \text{栈中对象数} \geq 0$ .

- PUSH:  $c_i = 1, \Delta\Phi = +1, \hat{c}_i = 2$ .
- POP:  $c_i = 1, \Delta\Phi = -1, \hat{c}_i = 0$ .
- MULTIPOP( $k$ ):  $c_i = \min(k, s), \Delta\Phi = -\min(k, s), \hat{c}_i = 0$ .

总平摊 =  $\sum \hat{c}_i \leq 2n$ .

## 4.2 二进制计数器势能

势  $\Phi(D_i) = b_i = \text{数组中 1 的个数}$ .

第  $i$  次 INCREMENT 翻  $t_i$  个 1 为 0, 再翻 1 个 0 为 1 (或满溢): - 若  $b_i = 0$  (溢出):  $b_{i-1} = t_i = k, \Delta\Phi = -k, c_i = k, \hat{c}_i = 0$ . 但这不该是 0...实际是有限位, 取计数器位数  $k \leq \log_2 n$  时. - 若  $b_i > 0$ :  $b_i = b_{i-1} - t_i + 1, \Delta\Phi = 1 - t_i, c_i = t_i + 1, \hat{c}_i = 2$ .

总平摊  $\leq 2n$ , 总实际  $\leq 2n - b_n + b_0$ , 即便初始非 0 也成立.

[EX] 初值非零

若  $b_0$  与  $b_n$  都在  $[0, k]$  中, 则  $n$  次操作总代价  $\leq 2n - b_n + b_0 = O(n)$  (当  $k = O(n)$ ).

## 5. 动态表 (Dynamic Table)

需要在线扩张/收缩数组, 维持“未使用空间不超过分配空间一定比例”.

### 5.1 仅插入

策略: 满时分配  $2 \times$  新表, 复制旧表.

```

INSERT(T, x):
  if size[T] == 0: alloc 1 slot; size = 1
  if num[T] == size[T]:                # 满
    alloc 2 * size new slots
    copy old to new; free old
    size *= 2
  T[num++] = x

```

聚集: 第  $i$  次插入代价  $c_i = i$  若  $i - 1$  是 2 的幂 (因要复制), 否则  $c_i = 1$ . 总代价:

$$\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \log n \rfloor} 2^j = n + 2n - 1 < 3n.$$

平摊  $\leq 3 = O(1)$ .

记账法: 每次插入收 3 元: 1 元当前操作, 1 元给当前对象 (将来复制时用), 1 元给已存款用尽的旧对象. 扩张时正好用完存款.

势能法:  $\Phi(T) = 2 \cdot \text{num}(T) - \text{size}(T)$ . 满时  $\Phi = \text{num}$ , 半满时  $\Phi = 0$ .

- 不触发扩张:  $\hat{c} = 1 + 2 - 0 = 3$ .
- 触发扩张:  $\hat{c} = \text{num}_i + (2 \text{num}_i - 2(\text{num}_i - 1)) - (2(\text{num}_i - 1) - (\text{num}_i - 1)) = \text{num}_i + 2 - (\text{num}_i - 1) = 3$ .

## 5.2 插入 + 删除

策略: 满 ( $\text{num} = \text{size}$ ) 时双倍扩张,  $\text{num} < \text{size}/4$  时收缩为  $\text{size}/2$ .

势函数:

$$\Phi(T) = \max \left( 2 \text{num}(T) - \text{size}(T), \frac{\text{size}(T)}{2} - \text{num}(T) \right). \quad (4)$$

- 插入平摊  $\leq 3$ .
- 删除平摊  $\leq 2$ .

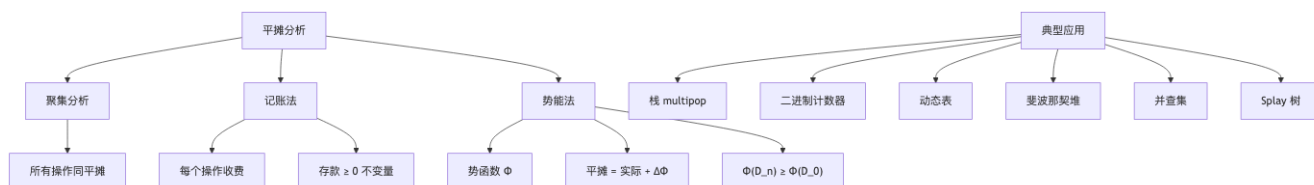
### [WARN] 收缩阈值

若用  $\text{num} < \text{size}/2$  收缩, 容易陷入“插入-触发扩张-删除-触发收缩-插入-...”的反复抖动, 每次  $O(n)$ . 必须用  $1/4$  阈值留出 buffer.

## 6. 其他经典平摊数据结构

| 数据结构           | 操作           | 平摊代价           |
|----------------|--------------|----------------|
| 斐波那契堆          | DECREASE-KEY | $O(1)$         |
| 斐波那契堆          | EXTRACT-MIN  | $O(\log n)$    |
| 并查集 (路径压缩 + 秩) | UNION/FIND   | $O(\alpha(n))$ |
| Splay 树        | 各操作          | $O(\log n)$    |
| Red-Black 树    | 插入/删除颜色翻转    | $O(1)$         |
| 散列表动态扩容        | INSERT       | $O(1)$         |

## 概念关系



## 记号速查

| 符号          | 含义                  |
|-------------|---------------------|
| $c_i$       | 第 $i$ 次操作的实际代价      |
| $\hat{c}_i$ | 第 $i$ 次操作的平摊代价      |
| $\Phi$      | 势函数                 |
| $D_i$       | 第 $i$ 次操作后的数据结构     |
| $\alpha(n)$ | Ackermann 反函数 (并查集) |

## 易错点汇总

### [WARN] 平摊分析常见坑

1. 平摊  $\neq$  平均: 平摊不依赖概率, 是最坏情况.
2. 势能法  $\Phi(D_0)$  不一定为 0, 但需要  $\Phi(D_n) \geq \Phi(D_0)$ .
3. 动态表收缩阈值用  $1/4$ , 不能用  $1/2$ .
4. 记账法存款不变量: 任何时刻总存款  $\geq 0$ .
5. 不同势函数选择会得到不同平摊代价, 选最合适的.

## 练习题

### [EX] 习题

1. 计数器加入 RESET 操作, 使  $n$  次操作 (INCREMENT + RESET 任意混合) 总时间  $O(n)$ .
2. 用两个普通栈实现队列, ENQUEUE/DEQUEUE 平摊  $O(1)$ .
3. 二叉最小堆: INSERT  $O(\log n)$  平摊, EXTRACT-MIN  $O(1)$  平摊, 找势函数.

参考解 (1): 维护指针  $i$  = 最大被设为 1 的位置 + 1. 势  $\Phi = b_i + \text{max bit index}$ . 详见 CLRS Ex 17.

### [TIP] 期中复习要点

1. 三种方法的本质—都给出实际代价的上界, 等价表达.
2. 栈 multipop / 二进制计数器 / 动态表 三个例必背 + 各用三种方法都能做.
3. 势能法选择  $\Phi$  的技巧: 通常选与数据结构“未结清债务”成比例的量.

# Lecture 09 —网络流算法 (1)

## [TIP] 学习指南

本节核心: 容量网络 + 可行流 + 最大流 / 最小割定理 + Ford–Fulkerson 标号法. 前置知识: 图基础 (有向图, BFS/DFS). 重点关注: 最大流–最小割定理证明、F–F 算法的标号过程、辅助网络.

## 0. 容量网络

容量网络  $N = \langle V, E, c, s, t \rangle$ :

- $\langle V, E \rangle$  是有向连通图; 记  $n = |V|, m = |E|$ ;
- $c : E \rightarrow \mathbb{R}^+$  边容量;
- $s$  发点 (source),  $t$  收点 (sink);
- 其余顶点为 中间点.

## 1. 可行流与最大流

设  $f : E \rightarrow \mathbb{R}_0^+$  满足:

1. 容量限制:  $\forall \langle i, j \rangle \in E, 0 \leq f(i, j) \leq c(i, j)$ .
2. 平衡条件:  $\forall i \in V \setminus \{s, t\}, \sum_{j: \langle j, i \rangle \in E} f(j, i) = \sum_{j: \langle i, j \rangle \in E} f(i, j)$  (流入 = 流出).

则称  $f$  是  $N$  的 可行流, 其流量

$$v(f) = \sum_{\langle s, j \rangle \in E} f(s, j) - \sum_{\langle j, s \rangle \in E} f(j, s). \quad (1)$$

流量最大的可行流称作 最大流.

最大流问题 (LP 形式):

$$\begin{aligned} & \max v(f) \\ & \text{s.t. } f(i, j) \leq c(i, j), \forall \langle i, j \rangle \in E \\ & \sum_{j: \langle j, i \rangle \in E} f(j, i) - \sum_{j: \langle i, j \rangle \in E} f(i, j) = 0, \forall i \in V \setminus \{s, t\} \\ & f(i, j) \geq 0, \forall \langle i, j \rangle \in E \end{aligned}$$

## 2. 割 (Cut)

设  $A \subseteq V$  满足  $s \in A, t \in V \setminus A$ . 集合

$$(A, V \setminus A) = \{\langle i, j \rangle \in E : i \in A, j \in V \setminus A\}$$

称为  $N$  的一个 **割集**, 容量

$$c(A, V \setminus A) = \sum_{\langle i, j \rangle \in (A, V \setminus A)} c(i, j). \quad (2)$$

**最小割**: 容量最小的割.

## 3. 三大引理

### 3.1 引理 1 (流量穿过割)

对任意可行流  $f$  与割  $(A, V \setminus A)$ :

$$v(f) = f(A, V \setminus A) - f(V \setminus A, A), \quad (3)$$

其中  $f(X, Y) = \sum_{i \in X, j \in Y, \langle i, j \rangle \in E} f(i, j)$ .

证: 把每个  $i \in A$  的平衡式相加, 内部边正负抵消, 只剩穿过割的边.

### 3.2 引理 2 (弱对偶)

任意可行流  $f$  与割  $(A, V \setminus A)$ :

$$v(f) \leq c(A, V \setminus A). \quad (4)$$

证:  $v(f) = f(A, \bar{A}) - f(\bar{A}, A) \leq f(A, \bar{A}) \leq c(A, \bar{A})$ .

### 3.3 引理 3 (强对偶充分)

若  $v(f) = c(A, V \setminus A)$ , 则  $f$  是最大流,  $(A, V \setminus A)$  是最小割.

## 4. 增广链与最大流性质

### 4.1 链与增广链

设  $f$  是可行流.

- 饱和边:  $f(i, j) = c(i, j)$ ;
- 非饱和边:  $f(i, j) < c(i, j)$ ;
- 零流边:  $f(i, j) = 0$ ;
- 非零流边:  $f(i, j) > 0$ .

链  $P$ : 从  $i$  到  $j$  不考虑方向的边不重复路径. 链中:

- 前向边: 方向与链方向一致;
- 后向边: 方向与链方向相反.

$i$ - $j$  增广链: 链  $P$  上所有前向边 **非饱和**, 所有后向边 **非零流**.

## 4.2 沿增广链修改流量

设  $\delta$  = 增广链上 (前向边的剩余容量) 与 (后向边的流量) 的最小值. 则

$$f'(i, j) = \begin{cases} f(i, j) + \delta & (i, j) \text{ 是 } P \text{ 的前向边} \\ f(i, j) - \delta & (i, j) \text{ 是 } P \text{ 的后向边} \\ f(i, j) & \text{否则} \end{cases}$$

仍是可行流, 且  $v(f') = v(f) + \delta$ .

## 4.3 定理 1 (Ford-Fulkerson 最优性)

可行流  $f$  是最大流  $\Leftrightarrow$  不存在关于  $f$  的  $s$ - $t$  增广链.

证: 必要性显然 (有增广链可继续增). 充分性: 设无  $s$ - $t$  增广链. 令  $A = \{j \in V : \exists s \rightarrow j \text{ 增广链}\}$ .  $s \in A$ ,  $t \notin A$ . 对  $\forall \langle i, j \rangle \in (A, V \setminus A)$ , 必有  $f(i, j) = c(i, j)$  (否则  $s \rightarrow i$  链可延伸); 对  $\forall \langle j, i \rangle \in (V \setminus A, A)$ , 必有  $f(j, i) = 0$ . 由引理 1,  $v(f) = c(A, V \setminus A)$ , 再由引理 3,  $f$  是最大流.

## 4.4 定理 2 (最大流最小割定理, MPMC)

$$\max_{f \text{ 可行}} v(f) = \min_{(A, V \setminus A) \text{ 是割}} c(A, V \setminus A). \quad (5)$$

意义: 最大流的值 = 最小割的容量. 也是 LP 强对偶的特例.

## 5. Ford-Fulkerson 标号法

思想: 从某初始可行流 (通常零流) 开始, 反复找增广链, 直到没有.

标号法: 顶点  $j$  的标号  $(l_j, \delta_j)$ , 表示已找到从  $s$  到  $j$  的增广链:

- $\delta_j = s \rightarrow j$  链上可增加流量的最大值;
- $l_j = +i$  表示链经前向边  $\langle i, j \rangle$  到  $j$ ;
- $l_j = -i$  表示链经后向边  $\langle j, i \rangle$  到  $j$ .

顶点状态: 已标号已检查 / 已标号未检查 / 未标号.

FORD-FULKERSON:

```
f = 0 # 零流
loop: # 一个阶段
  T = {s}; R = V - {s}; l[s] = ε; δ[s] = ∞
  while T ≠ ∅ and t not labeled:
    i = T.pop() # 检查 i
    for j ∈ R with <i, j> ∈ E:
      if f(i, j) < c(i, j):
        l[j] = +i
        δ[j] = min(δ[i], c(i, j) - f(i, j))
        R -= {j}; T += {j}
    for j ∈ R with <j, i> ∈ E:
      if f(j, i) > 0:
        l[j] = -i
        δ[j] = min(δ[i], f(j, i))
        R -= {j}; T += {j}
  if t not labeled: return f # 无 s-t 增广链, 最大流
```

```
# 回溯并修改流量
j = t
while j != s:
    if l[j] == +i: f(i, j) += δ[t]; j = i
    else          : f(|l[j]|, j) -= δ[t]; j = |l[j]|
```

## 5.1 复杂度

设容量为正整数, 最大流值  $v^* \leq C = c$  总和. 每阶段流量至少 +1, 至多  $C$  个阶段. 每阶段  $O(m)$  标号 + 修改. 故  $O(mC)$ .

[WARN] 容量为实数

若容量取实数 (含无理数), F-F 不一定终止! 故必须用 Edmonds-Karp (BFS 最短增广链) 或 Dinic.

## 5.2 实例

[EX] 6 节点网络

边:  $(s, 1) = 10, (s, 2) = 8, (s, 3) = 2, (1, 4) = 12, (2, 1) = 2, (2, 3) = 10, (3, 4) = 6, (3, t) = 2, (4, 3) = 10, (4, t) = 8, (1, 3) = 10$ .

阶段 1: 找增广链  $s \rightarrow 1 \rightarrow 4 \rightarrow t, \delta = \min(10, 12, 8) = 8$ . 阶段 2:  $s \rightarrow 2 \rightarrow 3 \rightarrow t, \delta = 2$ .

阶段 3:  $s \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow t, \dots$  等.

最终  $v(f) = 18$ , 最小割  $(\{s, 2, 3\}, \{1, 4, t\}) = 18$ .

## 6. 提高效率的两条路径

1. 最短增广链 (Edmonds-Karp BFS): 用 BFS 找最短增广链, 复杂度  $O(nm^2)$ .
2. 分层辅助网络 (Dinic): 见 Lec 10.

## 7. 辅助网络 $N(f)$

设可行流  $f$ . 辅助网络 (residual network)  $N(f) = \langle V, E(f), ac, s, t \rangle$ :

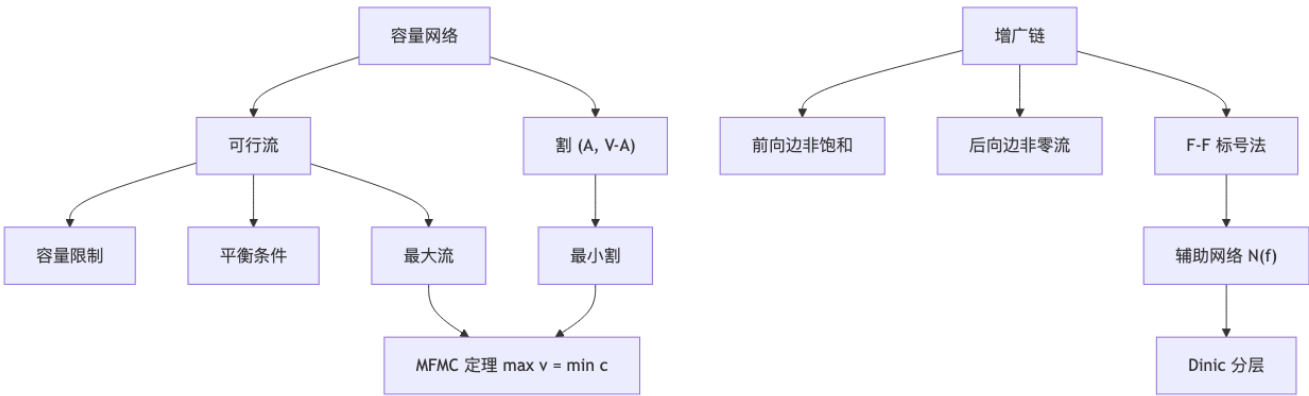
- $E^+(f) = \{\langle i, j \rangle \in E : f(i, j) < c(i, j)\}, ac(i, j) = c(i, j) - f(i, j);$
- $E^-(f) = \{\langle j, i \rangle : \langle i, j \rangle \in E, f(i, j) > 0\}, ac(j, i) = f(i, j);$
- $E(f) = E^+(f) \cup E^-(f).$

在  $N(f)$  上的  $s-t$  路径 原网络  $N$  上关于  $f$  的  $s-t$  增广链.

引理 4 (流量分解): 设  $v^*$  是  $N$  的最大流值,  $f$  是  $N$  的可行流,  $g$  是  $N(f)$  的可行流, 则  $f + g$  是  $N$  的可行流, 且  $v(f + g) = v(f) + v(g)$ . 因此

$$v^* - v(f) = \maxflow(N(f)). \quad (6)$$

概念关系



记号速查

| 符号                                  | 含义         |
|-------------------------------------|------------|
| $N = \langle V, E, c, s, t \rangle$ | 容量网络       |
| $f$                                 | 可行流        |
| $v(f)$                              | 流量         |
| $(A, V \setminus A)$                | 割集         |
| $c(A, V \setminus A)$               | 割容量        |
| $\delta$                            | 增广链修改量     |
| $N(f)$                              | 辅助网络       |
| $E^+(f), E^-(f)$                    | 前向 / 后向辅助边 |

易错点汇总

[WARN] 网络流常见坑

1. 平衡条件只对中间点;  $s, t$  不要求. 2. 增广链中后向边的方向—链方向是  $s \rightarrow t$ , 但后向边的实际有向边是  $t \rightarrow s$  方向的子边. 3. F-F 在实数容量时可能不收敛. 整数容量保证停机. 4. 最小割不唯一, 但最大流值唯一. 5. 复杂度  $O(mC)$  中  $C$  是 **总流量**, 而非边数; 容量大时不实用.

[TIP] 期末复习要点

1. 最大流最小割定理证明 (定理 1 + 定理 2). 2. 三大引理证明 + 流量分解 (引理 4). 3. F-F 算法的标号法伪代码必能写. 4. 辅助网络构造规则 (含后向边).

# Lecture 10 —网络流算法 (2): Dinic、最小费用流、应用

## [TIP] 学习指南

本节核心: Dinic 算法  $O(n^3)$ , 最小费用流 (负回路 / 最短路径), 网络流应用 (运输 / 二部图匹配 / 图像分割).  
前置知识: Lec 9 (F-F, 辅助网络). 重点关注: 分层辅助网络、阻塞流、最小费用流的两种算法.

## 0. 回顾

F-F 算法  $O(mC)$  不实用. Dinic 用 分层辅助网络 与 阻塞流 (blocking flow) 改进到  $O(n^3)$ .

## 1. 分层辅助网络

设  $f$  是  $N$  的可行流,  $N(f)$  是其辅助网络. 在  $N(f)$  上 BFS 从  $s$  开始, 按到  $s$  距离  $d$  把顶点分层:

- $V_k(f) = \{i \in V : d(s, i) = k\}$  for  $0 \leq k \leq d - 1$ ;
- $V_d(f) = \{t\}$  (只考虑到  $t$  的最短路径).

分层辅助网络  $AN(f) = \langle V(f), AE(f), ac, s, t \rangle$ , 其中  $AE(f)$  只保留从  $V_k$  到  $V_{k+1}$  的边 (DAG).

## [WARN] 分层网络性质

$AN(f)$  上  $s$  到  $t$  的任何路径都是  $N(f)$  上的最短增广链. 这正是 Edmonds-Karp 想要的.

## 2. Dinic 算法

思想: 在  $AN(f)$  上找 阻塞流  $g$  (即每条  $s-t$  路径都至少有一条饱和边), 然后  $f \leftarrow f + g$ , 重新构建  $AN(f)$ . 重复直到  $AN(f)$  中无  $s-t$  路径.

DINIC:

```
f = 0
loop:
  build AN(f) by BFS
  if t not reachable in AN(f): return f
  g = 0
  # 在 AN(f) 上找阻塞流 g
  while AN(f) 中存在 s-t 路径:
    pick vertex k with min throughput (流量) th(k)
    from k, push th(k) units forward to t and backward to s, add to g
```

```

delete k and adjacent edges
for <i, j> in AE:
    if g(i,j) == ac(i,j): delete <i,j>
    else: ac(i,j) -= g(i,j)
f += g
重新构建 AN(f)

```

流量 (throughput):  $th(i) = \min(\sum_{<j,i>} ac(j,i), \sum_{<i,j>} ac(i,j))$ , 表示通过  $i$  的最大可能流量.

## 2.1 正确性

引理 6: 每阶段结束时  $g$  是  $AN(f)$  的极大流 (= 阻塞流). 引理 7: 每阶段  $AN(f+g)$  中  $s-t$  距离 严格大于 前一阶段  $AN(f)$  中  $s-t$  距离. 定理 3 (Dinic 复杂度): 算法在  $O(n^3)$  步内终止.

证终止性: 由引理 7,  $s-t$  距离每阶段至少加 1, 距离  $\leq n-1$ , 故至多  $n$  个阶段.

每阶段工作量  $O(n^2)$ :

- 构造  $AN(f) - O(m)$ ;
- 计算流量  $- O(m)$ ;
- 找最小流量顶点  $n$  次, 每次  $O(n) - O(n^2)$ ;
- 边操作: 饱和删边  $O(m)$ , 同一顶点保留 1 条非饱和边  $- O(n^2)$ .

总:  $O(n^3)$ .

## 2.2 简单容量网络 ( $O(n^{1/2}m)$ )

简单容量网络: 所有边容量 = 1, 中间点 in-degree = 1 或 out-degree = 1. 用于二部图匹配等.

阶段数  $O(\sqrt{n})$ , Dinic 总  $O(\sqrt{n} \cdot m)$ .

## 3. 最小费用流

### 3.1 问题

容量-费用网络  $N = \langle V, E, c, w, s, t \rangle$ , 每条边除容量  $c(e)$  还有 单位流量费用  $w(e)$ .

总费用:

$$W(f) = \sum_{e \in E} w(e)f(e). \quad (1)$$

最小费用流 (Min-Cost Max Flow): 在所有最大流中找费用最小者. 或: 给定目标流量  $v_0$ , 找费用最小的可行流.

### 3.2 负回路算法 (Klein)

思想: 给定最大流  $f$ . 在辅助网络  $N(f)$  上, 边  $\langle i, j \rangle \in E^+(f)$  费用 =  $w(i, j)$ , 边  $\langle j, i \rangle \in E^-(f)$  费用 =  $-w(i, j)$  (反向退流, 费用退还). 若存在 负费用回路, 沿回路推流, 总费用下降.

KLEIN-MIN-COST:

```

f = max-flow (Dinic)
loop:
    build N(f) with edge costs
    find negative cost cycle (Bellman-Ford)

```

```

    if none: return f
    push flow  $\delta$  along cycle ( $\delta = \min$  residual capacity)

```

Floyd-Warshall 也可用来找负回路 (检测  $d[i, i] < 0$ ).

### 3.3 最短路径算法 (Successive Shortest Path)

思想: 从零流开始, 反复增 1 个流量, 但走 **费用最短** 增广链.

SSP:

```

    f = 0
    while v(f) < v_0:
        find shortest (by cost) s-t augmenting path in N(f)
        push  $\delta = \min\{\text{remaining capacity}, v_0 - v(f)\}$  along it

```

**最短路径:** 由于辅助网络含负边 (后向边), 必须用 Bellman-Ford  $O(nm)$  或 Johnson's reweighting + Dijkstra  $O(m + n \log n)$ .

总复杂度:  $O(v_0 \cdot \text{SP})$ .

### 3.4 Floyd-Warshall 全源最短路径

求所有顶点对最短路径:

```

FLOYD(W):
    D^(0) = W
    for k = 1..n:
        for i = 1..n:
            for j = 1..n:
                D^(k)[i][j] = min(D^(k-1)[i][j], D^(k-1)[i][k] + D^(k-1)[k][j])
    return D^(n)

```

复杂度  $O(n^3)$ . 可处理 **负权边** (无负回路).

## 4. 网络流应用

### 4.1 带需求的流通

**问题:** 流通图  $N = \langle V, E, c, S, T \rangle$ , 每个  $v$  有需求  $d_v$  (正 = 收, 负 = 供, 0 = 中间). 找可行函数  $f: E \rightarrow \mathbb{R}_0^+$ :

- 容量  $0 \leq f(e) \leq c_e$ ;
- 守恒  $f_{\text{in}}(v) - f_{\text{out}}(v) = d_v$ .

**必要条件:**  $\sum d_v = 0$ , 设  $D = \sum_{d_v > 0} d_v = -\sum_{d_v < 0} d_v$ .

**转化为最大流:**

1. 加超源  $s^*$  与超汇  $t^*$ .
2. 对  $v \in S$  (供给), 加  $\langle s^*, v \rangle$  容量  $-d_v$ .
3. 对  $v \in T$  (需求), 加  $\langle v, t^* \rangle$  容量  $d_v$ .
4. 求  $s^*-t^*$  最大流  $f^*$ . 若  $v(f^*) = D$ , 有可行流通; 否则无.

**拓展:** 边下界  $l_e \leq f(e) \leq c_e$ , 可通过“强制把  $l_e$  推流”转化.

## 4.2 运输问题 (Hitchcock)

$m$  产地  $A_i$  产  $a_i$ ,  $n$  销地  $B_j$  需  $b_j$ ,  $\sum a_i = \sum b_j$ . 单位运费  $w_{ij}$ .

建模为最小费用流:

- $V = \{s, t, A_1 \dots A_m, B_1 \dots B_n\}$ ;
- $\langle s, A_i \rangle$ :  $c = a_i, w = 0$ ;
- $\langle B_j, t \rangle$ :  $c = b_j, w = 0$ ;
- $\langle A_i, B_j \rangle$ :  $c = +\infty, w = w_{ij}$ .

求流量  $v_0 = \sum a_i$  的最小费用流.

## 4.3 二部图最大匹配

二部图  $G = (X \cup Y, E)$ . 构造容量网络:  $s \rightarrow x$  容量 1,  $x \rightarrow y$  容量 1,  $y \rightarrow t$  容量 1. 求  $s-t$  最大流 = 最大匹配数. Dinic 在简单容量网络上  $O(\sqrt{n} \cdot m)$  —著名的 Hopcroft-Karp 复杂度.

## 4.4 图像分割

问题: 像素  $V$  + 邻居对  $E$ . 每像素  $i$  有“前景似然” $a_i$  与“背景似然” $b_i$ ; 每邻居对  $(i, j)$  有分离罚分  $p_{ij} > 0$ .

目标: 分  $V = A \cup B$  ( $A$  前景,  $B$  背景), 最大化

$$q(A, B) = \sum_{i \in A} a_i + \sum_{j \in B} b_j - \sum_{(i,j) \in E, |A \cap \{i,j\}|=1} p_{ij}. \quad (2)$$

化为最小割:

$$q'(A, B) = \sum_{i \in A} b_i + \sum_{j \in B} a_j + \sum_{(i,j) \in E, |A \cap \{i,j\}|=1} p_{ij}, \quad (3)$$

则  $q(A, B) = Q - q'(A, B)$  ( $Q = \sum_i (a_i + b_i)$ ).  $\max q \quad \min q'$ .

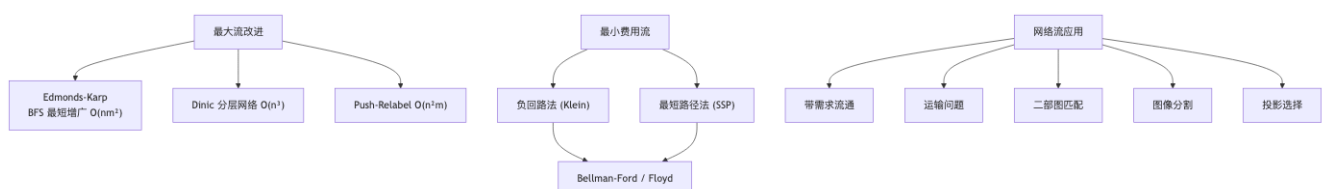
容量网络构造:

- 加超源  $s$  (前景) 与超汇  $t$  (背景);
- $\langle s, i \rangle$ :  $c = a_i$ ;
- $\langle i, t \rangle$ :  $c = b_i$ ;
- 邻居  $(i, j)$ : 加双向边  $\langle i, j \rangle, \langle j, i \rangle$ , 容量都 =  $p_{ij}$ .

最小  $s-t$  割 =  $\min q'(A, B)$ . 求最大流 = 最小割.

复杂度  $O(n^3)$  (Dinic).

## 概念关系



## 记号速查

| 符号       | 含义                         |
|----------|----------------------------|
| $AN(f)$  | 分层辅助网络                     |
| $V_k(f)$ | 距离 $s$ 为 $k$ 的顶点层          |
| $th(i)$  | 流通量                        |
| $g$      | 阻塞流                        |
| $w(e)$   | 边费用                        |
| $W(f)$   | 流总费用                       |
| $D$      | 总需求 $= \sum_{d_v > 0} d_v$ |

## 易错点汇总

### [WARN] 易错点

1. Dinic 阶段计数  $\leq n$  (距离单调上升), 不是  $m$ . 2. 分层网络只保留  $V_k \rightarrow V_{k+1}$  的边, 跨层或回退的边都丢弃. 3. 阻塞流  $\neq$  最大流; 但每阶段做完阻塞流后, 增广链的最短长度严格上升. 4. 最小费用流: 后向边的费用是负的 (退流退费). 5. 图像分割中, 邻居边是 **无向** 的, 构造时要两个方向都加.

### [TIP] 期末复习要点

1. Dinic 三步骤: BFS 分层  $\rightarrow$  找阻塞流  $\rightarrow$  重建. 复杂度  $O(n^3)$ . 2. 简单容量网络 (二部图匹配) Dinic 是  $O(\sqrt{nm})$ . 3. 最小费用流: 负回路法 vs 最短路径法. 4. 带需求流通的超源超汇构造. 5. 图像分割  $\rightarrow$  最小割转化 (前景 = 与  $s$  同侧, 背景 = 与  $t$  同侧).

# Lecture 11 一问题复杂度 (1): 下界与决策树

## [TIP] 学习指南

本节核心: 不只问“我的算法多快”一而是“任何算法最快能多快”. 这是问题复杂度 (problem complexity) 的研究对象. 前置知识: Lec 3 (Select), Lec 4 (DP). 重点关注: 平凡下界、决策树证排序下界、构造最坏输入、检索问题.

## 0. 算法评价

### 0.1 算法正确性

- 方法正确性: 算法思路对一证明相关引理与定理.
- 程序正确性: 实现忠实于算法.

### 0.2 工作量 (时间)

基本运算选择:

| 问题    | 基本运算 |
|-------|------|
| 检索    | 比较   |
| 矩阵乘法  | 实数乘法 |
| 排序    | 比较   |
| 遍历二叉树 | 指针赋值 |

最坏  $W(n)$  vs 平均  $A(n)$ .

### 0.3 空间复杂度

- 存储程序 + 输入:  $O(n)$ .
- 额外空间: 中间结果 + 操作单元.
- 原地算法: 额外空间  $O(1)$ .

### 0.4 简单性

简单算法易验证、易调试, 但不一定快. 应在效率合理前提下追求简单.

## 1. 算法最优性

设  $A$  解问题  $P$ . 若不存在算法在最坏情况下比  $A$  更快, 则  $A$  是 **最坏情况最优** (worst-case optimal).

## 1.1 寻找最优算法

1. 设计算法  $A$ , 算  $W(n) \rightarrow$  类的上界
2. 找函数  $F(n)$ , 证: 任何算法都有规模  $n$  输入需  $\geq F(n)$  次运算  $\rightarrow$  类的下界
3. 若  $W(n) = \Theta(F(n))$ :  $A$  是最优.
4. 若  $W(n) > F(n)$ : 要么改进  $A$  让  $W(n)$  降到  $F(n)$ , 要么提升下界  $F'(n) > F(n)$ .

## 2. 平凡下界

### 2.1 输入/输出规模下界

任何算法都必须读完输入 / 写完输出. 故下界  $\geq \max(\text{input size}, \text{output size})$ .

[EX] example

– 矩阵乘法: 输入  $2n^2$ , 输出  $n^2$   $\Omega(n^2)$ . – 排序:  $\Omega(n)$  (输出排序).

### 2.2 最少运算次数

直接论证必须做多少次基本运算.

[EX] 找最大元素

输入  $n$  个数, 必须淘汰  $n - 1$  个 (每个被淘汰必经过至少一次比较), 故  $\geq n - 1$  次比较. 顺序比较算法  $W(n) = n - 1$  是最优.

### 2.3 找最大和最小

任何算法需要  $\geq 3n/2 - 2$  次比较. (证明见 Lec 13.) 算法 FindMaxMin (Lec 3)  $W(n) = 3n/2 - 2$  最优.

## 3. 决策树 (Decision Tree)

适用于以 比较 为基本运算的问题. 把算法表示为二叉树:

- 内节点 = 一次比较  $\langle i, j \rangle$ ;
- 左/右子树 = 不同比较结果下的后续;
- 叶子 = 输出.

关键事实:

- 任意输入对应根到叶的一条路径;
- 最坏情况比较次数 = 决策树 高度;
- 比较有  $\{<, =, >\}$  三种结果, 但通常合并“=”入某一分支, 视作 二叉决策树.

### 3.1 二叉树高度下界

$h$ -高二叉树至多  $2^h$  个叶子. 若必须区分  $N$  种输出, 则

$$h \geq \lceil \log_2 N \rceil. \quad (1)$$

## 4. 检索问题复杂度

问题: 有序数组  $L[1..n]$ , 给定  $x$ , 找  $x$  在  $L$  中的位置或返回 0.

输出:  $n + 1$  种可能 (位置  $1, \dots, n$ , 或不在).

决策树: 二叉;  $h \geq \lceil \log(n + 1) \rceil$ .

二分检索  $W(n) = \lceil \log(n + 1) \rceil$  —最优.

### 4.1 二分检索算法

```
BINSEARCH(L, x, p, r):
    if p > r: return 0
    q = (p + r) // 2
    if L[q] == x: return q
    elif L[q] < x: return BINSEARCH(L, x, q+1, r)
    else: return BINSEARCH(L, x, p, q-1)
```

复杂度:  $T(n) = T(n/2) + O(1) = \Theta(\log n)$ .

### 4.2 平均情况

假设每位置等概率,  $A(n) \approx \log n - 1$ . 也是  $\Theta(\log n)$ .

## 5. 排序问题复杂度下界

将在 Lec 12 详细处理. 关键结论:

定理: 任何基于比较的排序算法最坏需  $\Omega(n \log n)$  次比较.

证 (决策树): 排序输出是  $n!$  个置换中的一个. 决策树高度

$$h \geq \lceil \log_2 n! \rceil = \Omega(n \log n).$$

(由 Stirling,  $\log n! = \Theta(n \log n)$ .)

## 6. 构造最坏输入 (Adversary Argument)

适用于 下界证明: 任给一个算法  $A$ , 构造一个对手 (adversary) 实时回答  $A$  的每次比较, 让  $A$  至少做  $F(n)$  步.

例: 选第二大.

- 算法必须确定有  $n - 1$  个数比 max 小 至少  $n - 1$  次比较.
- 第二大必在被 max 直接淘汰过的  $\leq \log n$  个数中, 再比较  $\log n - 1$  次.

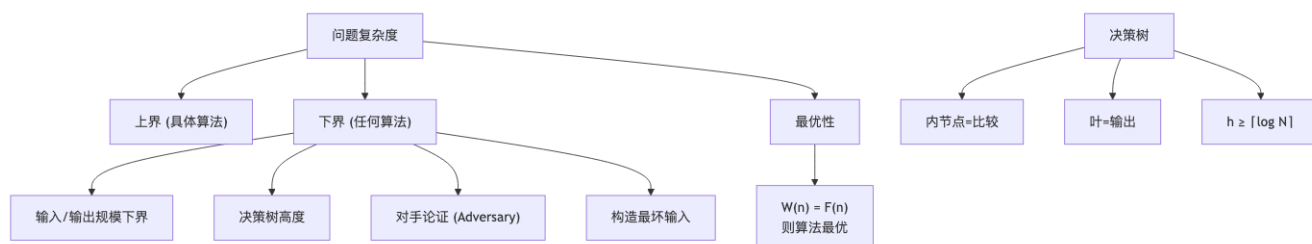
故任意算法  $W(n) \geq n + \log n - 2$ . 锦标赛 (Lec 3) 达到此下界, 最优.

## 7. 时间复杂度分析—排序与选择算法的优化

下表 (Lec 13 §3 的预览):

| 问题          | 算法            | $W(n)$                   | 空间          |
|-------------|---------------|--------------------------|-------------|
| 选最大         | 顺序            | $n - 1$                  | $O(1)$      |
| 选 max 与 min | 顺序            | $2n - 3$                 | $O(1)$      |
| 选 max 与 min | 算法 FindMaxMin | $3n/2 - 2$               | $O(1)$      |
| 选第二大        | 顺序            | $2n - 3$                 | $O(1)$      |
| 选第二大        | 锦标赛           | $n + \log n - 2$         | $O(n)$      |
| 选中位数        | 排序选           | $O(n \log n)$            | $O(1)$      |
| 选中位数        | Select        | $O(n)$ (常数 $\leq 2.95$ ) | $O(\log n)$ |

## 概念关系



## 记号速查

| 符号        | 含义        |
|-----------|-----------|
| $W(n)$    | 算法最坏时间    |
| $F(n)$    | 类下界       |
| 决策树       | 比较算法的执行模型 |
| Adversary | 构造最坏输入的对手 |

## 易错点汇总

### [WARN] 下界证明常见坑

1. 决策树下界 仅适用于比较模型. 计数排序、桶排序不属此类, 可低于  $\Omega(n \log n)$ .
2. 构造最坏输入: 必须对 **每个算法** 都能构造, 不能假定算法形式.
3. 平均下界 vs 最坏下界: 不要混淆.
4.  $\log n!$  不要算成  $n \log n$  的精确值—用 Stirling 给出  $\Theta$ .

### [TIP] 期末复习要点

1. 算法最优性的 3 步证明流程.
2. 平凡下界 (输入输出规模, 必要比较数) 的两种使用.
3. 决策树论证—二分检索  $\Theta(\log n)$ , 排序  $\Omega(n \log n)$ .
4. 对手论证—选第二大下界推导.

# Lecture 12 一问题复杂度 (2): 排序问题

## [TIP] 学习指南

本节核心: 冒泡排序、堆排序、排序问题的  $\Omega(n \log n)$  比较下界 (决策树). 前置知识: Lec 1 (Stirling), Lec 11 (决策树). 重点关注: 逆序数与冒泡复杂度、堆的构造与 Heapify、B-树高度下界.

## 1. 冒泡排序

```
BUBBLE-SORT(L, n):  
    FLAG = n                # 最后一次交换的位置  
    while FLAG > 1:  
        k = FLAG - 1  
        FLAG = 1  
        for j = 1..k:  
            if L[j] > L[j+1]:  
                swap L[j], L[j+1]  
            FLAG = j
```

特点: 交换仅发生在 相邻 元素之间.

### 1.1 逆序与逆序数

逆序: 置换  $a_1 \dots a_n$  中, 若  $i < j$  但  $a_i > a_j$ , 称  $(a_i, a_j)$  为一个逆序.

逆序数  $\text{inv}(\pi) = \#$  置换  $\pi$  中逆序总数.

逆序序列  $(b_1, b_2, \dots, b_n)$ :  $b_i$  = 在  $i$  右边且小于  $i$  的元素个数. 性质  $0 \leq b_i \leq i - 1$ .  $n!$  个置换  $\leftrightarrow$   $n!$  个逆序序列, 一一对应.

## [EX] example

置换 3 1 6 5 8 7 2 4. 逆序序列  $(0, 0, 2, 0, 2, 3, 2, 3)$ . 逆序数 = 12.

### 1.2 冒泡复杂度

最坏情况 (输入完全逆序, 逆序数 =  $n(n-1)/2$ ):

冒泡每次相邻交换 消除恰好 1 个逆序. 故比较次数  $\geq$  逆序数. 最大逆序数  $n(n-1)/2$ .

$$W(n) = \Theta(n^2). \quad (1)$$

平均情况: 置换  $\pi$  与逆向  $\pi'$  的逆序数和  $= n(n-1)/2$ .  $n!$  个置换分成  $n!/2$  对, 每对逆序之和  $= n(n-1)/2$ .

平均逆序数  $= n(n-1)/4$ , 平均比较次数  $\Theta(n^2)$ .

[WARN] 注意

冒泡排序的 最坏与平均 都是  $\Theta(n^2)$ , 比插入排序的最坏  $\Theta(n^2)$  略差 (常数).

## 2. 堆 (Heap)

### 2.1 定义

设  $T$  是深度为  $d$  的二叉树, 节点存  $L$  的元素.  $T$  是 堆 当:

1. 所有内节点 (除最多一个) 度为 2;
2. 所有树叶在相邻两层;
3. 第  $d-1$  层的树叶在内节点的右边;
4. 第  $d-1$  层最右内节点可能度 1;
5. 每个节点的元素  $\geq$  其儿子的元素 (大顶堆).

只满足 (1)-(4) 但不满足 (5) 称作 堆结构.

### 2.2 数组存储

数组  $A[1..n]$ , 节点  $i$ :

- 左儿子  $\text{left}(i) = 2i$ ;
- 右儿子  $\text{right}(i) = 2i + 1$ ;
- 父亲  $\text{parent}(i) = \lfloor i/2 \rfloor$ .

### 2.3 Heapify (向下调整)

```
HEAPIFY(A, i):
    l = 2i; r = 2i + 1
    largest = i
    if l <= heap-size and A[l] > A[largest]: largest = l
    if r <= heap-size and A[r] > A[largest]: largest = r
    if largest != i:
        swap A[i], A[largest]
        HEAPIFY(A, largest)
```

复杂度:  $T(n) = T(2n/3) + \Theta(1) = O(\log n)$ .

### 2.4 Build-Heap

```
BUILD-HEAP(A):
    heap-size = length(A)
    for i =  $\lfloor n/2 \rfloor$  downto 1:
        HEAPIFY(A, i)
```

复杂度分析:

引理 (高  $h$  节点数):  $n$  节点完全二叉树中, 高  $h$  节点至多  $\lceil n/2^{h+1} \rceil$  个.

证 (归纳  $h$ ):

- $h = 0$  (叶): 叶数  $= \lceil n/2 \rceil$ .
- 假设对  $h - 1$  成立. 从  $T$  移走所有叶得  $T'$ , 节点数  $n' = n - n_0$ ,  $T'$  中高  $h - 1$  节点数  $= T$  中高  $h$  节点数  $n_h$ . 由归纳  $n_h \leq \lceil n'/2^h \rceil = \lceil (n - \lceil n/2 \rceil)/2^h \rceil = \lceil n/2^{h+1} \rceil$ .

Build-Heap 时间:

$$T(n) = \sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^{h+1}}\right) = O(n). \quad (2)$$

利用  $\sum h/2^{h+1} = 1$ . 构建堆是  $O(n)$ , 而不是  $O(n \log n)$ .

## 2.5 Heap-Sort

```
HEAP-SORT(A):
  BUILD-HEAP(A)                                # O(n)
  for i = n downto 2:
    swap A[1], A[i]
    heap-size -= 1
    HEAPIFY(A, 1)                              # O(log i)
```

复杂度:  $O(n) + (n - 1) \cdot O(\log n) = O(n \log n)$ .

空间:  $O(1)$  额外 (原地).

## 3. 排序问题复杂度下界

### 3.1 决策树

任取算法  $A$ , 输入  $L = \{x_1, \dots, x_n\}$ . 构造决策树:

- 根 =  $A$  的第一次比较  $\langle i, j \rangle$ ;
- 假设节点  $k$  已标记  $\langle i, j \rangle$ :
  - 若  $x_i < x_j$  算法结束, 左儿子标“输入对应的输出”;
  - 若  $x_i < x_j$  继续比较  $\langle p, q \rangle$ , 左儿子标  $\langle p, q \rangle$ .
  - 右儿子 ( $x_i > x_j$  分支) 类似.

算法最坏比较次数 = 决策树的深度.

### 3.2 B-树

删去非二叉的内节点 (即不分支的内节点), 得 **B-树**. 性质:

- B-树深度  $\leq$  决策树深度;
- B-树有  $\geq n!$  个叶子 (每个置换对应一个).

由 B-树满二叉的高度下界:

$$h \geq \lceil \log_2 n! \rceil. \quad (3)$$

### 3.3 排序下界

**定理 4:** 任何基于比较的排序算法最坏需  $\geq \lceil \log_2 n! \rceil = \Theta(n \log n)$  次比较.

证:

$$\log_2 n! = \sum_{k=1}^n \log_2 k \geq \int_1^n \log_2 x \, dx = n \log_2 n - n/\ln 2 = \Theta(n \log n).$$

精确: Stirling,  $\log_2 n! = n \log_2 n - n \log_2 e + O(\log n)$ .

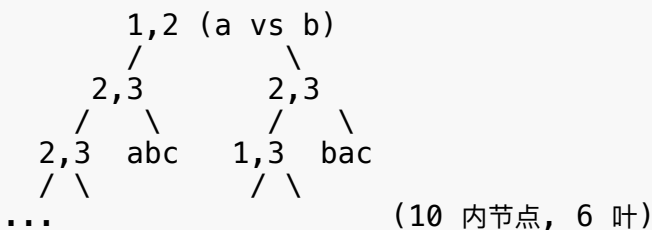
## [WARN] 适用范围

仅适用于 **基于比较** 的排序. 计数排序 ( $O(n + k)$ ), 基数排序 ( $O(d(n + b))$ ), 桶排序 ( $O(n)$  期望) 可以低于此下界, 但它们不是比较排序.

### 3.4 推论 (排序算法最优性)

- 归并排序 / 堆排序:  $W(n) = \Theta(n \log n)$ , 最坏意义最优.
- 快速排序: 平均  $\Theta(n \log n)$ , 平均意义最优; 最坏  $\Theta(n^2)$  不最优.
- 冒泡 / 插入 / 选择:  $\Theta(n^2)$ , 非最优.

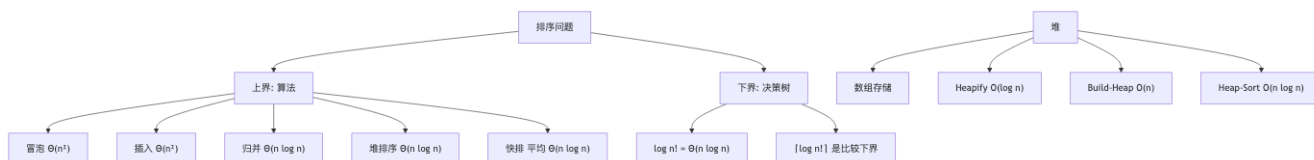
#### 4. 决策树示例一冒泡排序 $n = 3$



## [EX] 实例

输入  $\langle a, b, c \rangle$ , 冒泡需在  $3! = 6$  种排列中找到正确顺序. 最坏  $h = 3$  次比较.

## 概念关系



## 记号速查

| 符号         | 含义                  |
|------------|---------------------|
| 逆序数        | 置换中逆序总数             |
| $b_i$      | 在 $i$ 右且小于 $i$ 的元素数 |
| Heapify    | 向下调整                |
| Build-Heap | 自底向上建堆 $O(n)$       |
| B-树        | 决策树去掉非二叉内节点         |

## 易错点汇总

### [WARN] 易错点

1. Build-Heap 是  $O(n)$ , 不是  $O(n \log n)$  — 注意求和  $\sum h/2^h = O(1)$ .
2. 冒泡 vs 插入: 都是  $\Theta(n^2)$ , 但冒泡平均不优于最坏 (因为每次只能消除 1 个逆序).
3. 排序下界  $\log n!$  仅适用 **比较模型**.
4. 决策树是 **二叉** 的, 因为每次比较只有两种结果.
5. 堆的数组下标 1-based, 父子关系:  $i \rightarrow 2i, 2i + 1$ .

### [TIP] 期末复习要点

1. 逆序数概念 + 冒泡平均  $n(n-1)/4$  的推导.
2. Build-Heap  $O(n)$  的严格证明 (高  $h$  节点数引理).
3. 排序下界  $\lceil \log n! \rceil$  的决策树证明.
4. 堆排序 = Build-Heap +  $n-1$  次 Heapify.

# Lecture 13 一问题复杂度 (3): 选择问题 + NP 完全理论

## [TIP] 学习指南

本节核心: 用 **对手论证** 给选择问题构造最坏输入; 介绍 P, NP, NPC 和典型 NP-完全问题. 前置知识: Lec 11-12. 重点关注: 找最大最小下界  $3n/2 - 2$ , 找第二大下界  $n + \log n - 2$ , NP-完全证明.

## 1. 选择问题下界—找最大与最小

**定理 6:** 任何通过比较找最大和最小的算法至少需要  $3n/2 - 2$  次比较.

### 1.1 构造最坏输入 (信息单位)

每个数有四种状态:

- N: 未参加比较
- W: 赢 (至少 1 次)
- L: 输 (至少 1 次)
- WL: 赢且输

**输出条件:** 算法终止时, 必须确定 max (只带 W) 与 min (只带 L), 中间  $n - 2$  个数都带 WL, 总共  $2n - 2$  个“信息单位”(W 或 L 标记).

一次比较增加的信息:

| 比较类型                                                  | 增量 |
|-------------------------------------------------------|----|
| (N, N) $\rightarrow$ (W, L)                           | 2  |
| (W, N) $\rightarrow$ (W, L) 或 (W, WL) $\rightarrow$ ? | 1  |
| (W, L) $\rightarrow$ (W, L)                           | 0  |

**对手策略:** 给参与比较的两个变量 **赋值** 使每次比较至多产生 1 个信息单位 (而不是 2).

- 若两个都是 N, 对手必须给一个 W 一个 L (强制 2 个信息单位). 这种情况无法避免, 但 N 个数 (即起初的 N 个) 至多被“成对”用于 N-N 比较  $\lfloor n/2 \rfloor$  次.
- 其余比较 (至少含一个 W 或 L) 对手可让信息增量  $\leq 1$ .

总比较次数  $T(n)$  满足:

$$2 \cdot \lfloor n/2 \rfloor + 1 \cdot (T(n) - \lfloor n/2 \rfloor) \geq 2n - 2,$$

即  $T(n) \geq 2n - 2 - \lfloor n/2 \rfloor = \lceil 3n/2 \rceil - 2$ .

□

## 1.2 上界—FindMaxMin 算法

FIND-MAX-MIN(L):

|                                 |                               |
|---------------------------------|-------------------------------|
| 两两分组, 共 $\lfloor n/2 \rfloor$ 组 |                               |
| 组内比较得 $\max_i, \min_i$          | # $\lfloor n/2 \rfloor$ 次     |
| 在 $\max_i$ 中找最大                 | # $\lfloor n/2 \rfloor - 1$ 次 |
| 在 $\min_i$ 中找最小                 | # $\lfloor n/2 \rfloor - 1$ 次 |

总比较  $\lfloor n/2 \rfloor + 2(\lfloor n/2 \rfloor - 1) = \lceil 3n/2 \rceil - 2$ . 与下界匹配, 最优.

## 2. 选择问题下界—找第二大

定理 7: 任何找第二大的算法至少需  $n + \lceil \log n \rceil - 2$  次比较.

### 2.1 关键观察

第二大 = 输给最大的数中的最大. 设最大 = max.

- 直接淘汰 max 的次数 = max 的“out-degree”—比 max 大的数为 0, 故 max 至少与  $\log n$  个数比较过 (才能“认证”自己最大).
- 第二大必在被 max 淘汰过的  $\geq \log n$  个数中.

下界证明 (信息单位法):

- 找最大需  $n - 1$  次比较 (淘汰  $n - 1$  个数).
- 在  $\geq \log n$  个候选中找第二大, 至少  $\log n - 1$  次比较.
- 合计  $n + \log n - 2$ .

但“认证最大需直接淘汰  $\log n$  个”还需对手论证: 任意算法对所有数等价处理, 决策树深度  $\geq \log n$ .

### 2.2 上界—锦标赛算法

TOURNAMENT(L):

$n$  个数两两分组比赛, 大者晋级  
每个被淘汰的数记入淘汰它的数的链表  
共  $\lceil \log n \rceil$  轮,  $n-1$  次比较得 max  
在 max 的链表 ( $\lceil \log n \rceil$  个数) 中线性找最大:  $\lceil \log n \rceil - 1$  次  
总:  $n + \lceil \log n \rceil - 2$

与下界匹配, 最优.

## 3. 选择算法复杂度表

| 算法             | 最坏                      | 空间          |
|----------------|-------------------------|-------------|
| 选最大 (顺序)       | $n - 1$                 | $O(1)$      |
| 选 max+min (顺序) | $2n - 3$                | $O(1)$      |
| FindMaxMin     | $3n/2 - 2$              | $O(1)$      |
| 选第二大 (顺序)      | $2n - 3$                | $O(1)$      |
| 锦标赛            | $n + \log n - 2$        | $O(n)$      |
| 选中位数 (排序)      | $O(n \log n)$           | $O(1)$      |
| Select (BFPRT) | $O(n)$ , 常数 $\leq 2.95$ | $O(\log n)$ |

## 4. 计算复杂性理论—P, NP, NPC

### 4.1 判定问题 (Decision Problem)

仅输出“yes/no”的问题. 优化问题可转判定问题, 如“存在  $k$ -顶点覆盖?”.

### 4.2 P 类

P = 存在多项式时间算法的判定问题. 包括: 最短路径、最大流、LP、字符串匹配等.

### 4.3 NP 类

NP = 存在多项式时间 验证器 的判定问题. 即给定一个“证据”串, 能在多项式时间验证其正确性.

等价定义: 存在 非确定性图灵机 (NTM) 多项式时间接受.

显然  $P \subseteq NP$  (确定 非确定).

著名问题:  $P = NP$ ?

### 4.4 多项式时间归约 ( $\leq_p$ )

定义: 问题  $A \leq_p B$  当存在多项式时间可计算的函数  $f$  使得  $\forall x: x \in A \Leftrightarrow f(x) \in B$ .

性质:

- 传递:  $A \leq_p B, B \leq_p C \Rightarrow A \leq_p C$ ;
- $A \leq_p B$  且  $B \in P \Rightarrow A \in P$ ;
- 若  $A \leq_p B$  且  $A$  是 NPC, 则  $B$  也是 NP-Hard.

### 4.5 NP-完全 (NPC) 与 NP-Hard

$B$  是 NP-完全 当:

1.  $B \in NP$ ;
2.  $\forall A \in NP, A \leq_p B$ .

NP-Hard = 满足 (2) 但不必 (1).

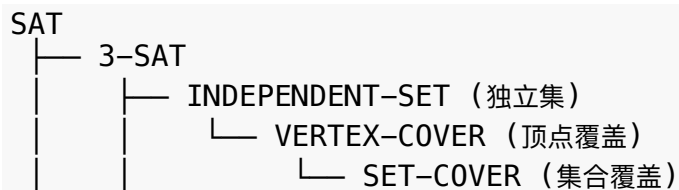
### 4.6 Cook-Levin 定理

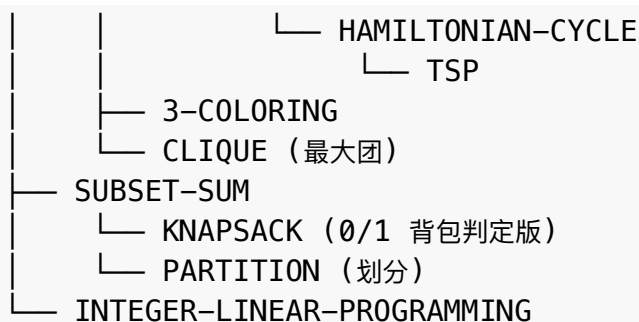
定理 (Cook-Levin 1971): SAT (布尔可满足性问题) 是 NP-完全.

证明思路: 任意 NTM 在多项式时间运行 可编码为 CNF 布尔公式, 公式可满足 NTM 接受.

## 5. 经典 NP-完全问题

下面是从 SAT 归约出发的归约链:





## 5.1 3-SAT

问题: 给定 CNF 公式  $\varphi$ , 每子句恰 3 个文字.  $\varphi$  是否可满足?

3-SAT  $\in$  NPC: SAT  $\leq_p$  3-SAT (引入新变量把  $k$ -子句拆成 3-子句).

## 5.2 INDEPENDENT-SET (独立集)

问题:  $G = (V, E)$ ,  $k$ . 是否存在  $|S| \geq k$  的独立集 (顶点互不相邻)?

3-SAT  $\leq_p$  IS: 对  $\varphi$  的每个子句  $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$ , 建 3 顶点三角形; 不同子句中互补的文字间加边.

$\varphi$  可满足  $\iff$  图存在  $k$ -独立集 (每子句选一文字).

## 5.3 VERTEX-COVER (顶点覆盖)

问题:  $G = (V, E)$ ,  $k$ . 是否存在  $|S| \leq k$  使每条边至少一端在  $S$ ?

IS  $\leq_p$  VC:  $S$  是独立集  $\iff V \setminus S$  是顶点覆盖. 故  $|IS_{max}| + |VC_{min}| = |V|$ .

## 5.4 HAMILTONIAN-CYCLE & TSP

HAM-CYCLE: 图中是否存在哈密顿回路 (访问每点恰一次).

VC  $\leq_p$  HAM-CYCLE: 复杂的 gadget 构造.

TSP: 完全图加权, 是否存在长度  $\leq B$  的哈密顿回路? HAM-CYCLE  $\leq_p$  TSP (边权 0/1).

## 5.5 SUBSET-SUM & KNAPSACK

SUBSET-SUM: 整数集  $S$  与目标  $T$ , 是否有子集和  $= T$ ?

3-SAT  $\leq_p$  SUBSET-SUM (经典构造).

KNAPSACK 决策版: 与 SUBSET-SUM 等价.

[WARN] 伪多项式

0/1 背包 DP  $O(nW)$  是 伪多项式—输入  $W$  的比特数  $\log W$ , 所以 DP 时间在比特长度意义下是指数.

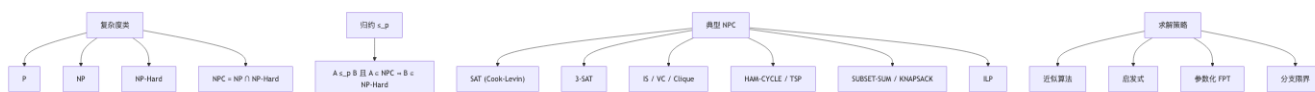
## 6. NP-完全问题求解策略

| 策略   | 描述                                        | 例             |
|------|-------------------------------------------|---------------|
| 近似算法 | 多项式时间, 解 $\leq \epsilon \cdot \text{OPT}$ | 顶点覆盖 2-approx |
| 启发式  | 无理论保证但实践有效                                | 模拟退火、遗传算法     |
| 参数化  | 固定某参数 $k$ 后多项式                            | $k$ -顶点覆盖 FPT |
| 限制特例 | 输入限制后多项式                                  | 平面图最大独立集      |
| 分支限界 | 智能剪枝                                      | ILP           |
| 随机算法 | 概率正确/优化                                   | Monte Carlo   |

## 7. 当前已知

- $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ .
- 严格  $P \subsetneq EXPTIME$  (时间层次).
- $P \stackrel{?}{=} NP$ : 千禧七大数学难题之一, 悬赏 100 万美元.
- 普遍相信  $P \neq NP$ . 若成立则 NPC 都不是  $P$ .

## 概念关系



## 记号速查

| 符号                   | 含义                    |
|----------------------|-----------------------|
| $P$                  | 多项式时间可解               |
| $NP$                 | 多项式时间可验证              |
| $NPC$                | NP-完全                 |
| $NP\text{-Hard}$     | NP 中所有问题都 $\leq_p$ 到它 |
| $\leq_p$             | 多项式时间归约               |
| $SAT / 3\text{-}SAT$ | 布尔可满足性                |
| $IS / VC$            | 独立集 / 顶点覆盖            |
| $TSP$                | 旅行商问题                 |

## 易错点汇总

### [WARN] NP 理论易错点

1.  $NP \neq \text{"non-polynomial"}$  — 是 “nondeterministic polynomial”. 2.  $NP\text{-Hard}$  不一定在  $NP$  中 (如 halting problem). 3.  $NPC$  证明两步: 是  $NP$  + 归约自某  $NPC$ . 4. 归约方向要对: 若证  $B$  是  $NP\text{-Hard}$ , 是从 已知  $NPC$  归约 到  $B$ , 不是反过来. 5. 多项式归约必须 多项式时间 可计算. 6. 伪多项式  $\neq$  多项式. 0/1 背包  $NPC$ .

### [TIP] 期末复习要点

1. FindMaxMin 下界  $3n/2 - 2$  与上界匹配—信息单位证明. 2. 锦标赛找第二大  $n + \log n - 2$  最优. 3.  $P, NP, NPC$  定义. 4. Cook-Levin 定理叙述; 经典  $NPC$  归约链记几个 ( $3\text{-}SAT \rightarrow IS \rightarrow VC$ ). 5.  $NPC$  问题处理策略.

# Lecture 14 —NP 完全 (NP-Completeness)

## [TIP] 学习指南

本节核心: 多项式时间归约 ( $\leq_p$ ), NP-完全 (NPC) 定义, Cook-Levin 定理, 经典 NPC 归约链. 前置知识: Lec 11 (问题复杂度 1), Lec 12 (排序下界), Lec 13 ( $P, NP, \leq_p$ ). 重点关注:  $\leq_p$  的传递性, NPC 证明两步法, SAT  $\leq_p$  3-SAT 局部替换法, 3-SAT  $\rightarrow$  IS/VC/CLIQUE 的 gadget 构造, Cook-Levin 编码思路. 课程意义: NP 完全理论给出了“难解”的等价类划分, 是判定一个问题在多项式时间内 **能否被攻克** 的工具.

## 1. 核心问题与动机 (Motivation)

工程实践中很多优化问题至今未找到多项式时间算法, 例如哈密顿回路 (HC)、货郎问题 (TSP)、0-1 背包. 它们的共同结构:

- **解的验证容易**: 给定一条回路或一个装包方案, 多项式时间检查可行性与目标值;
- **解的搜索困难**: 解空间组合爆炸, 至今没找到多项式算法.

**NP 完全理论** 不直接回答“这些问题难解”, 而是建立一个 **难度等价类**: 在 NPC 中任意一个问题若有多项式算法, 则 NP 内所有问题都有, 即  $P = NP$ . 这把“证明  $P \neq NP$ ”与“找一个 NPC 的多项式算法”等价起来, 给出了实践中的判定标尺: 遇到一个未知问题, 若能证明它是 NPC, 则停止寻找精确多项式算法, 改走近似/启发式路线.

## 2. 复习: 判定问题、P、NP

### 2.1 判定问题 (Decision Problem)

形式化: 判定问题  $\Pi = \langle D, Y \rangle$ ,  $D$  是实例集合,  $Y \subseteq D$  是“Yes”实例集合.

例:

- HC:  $D =$  所有无向图;  $Y =$  含哈密顿回路的图.
- TSP:  $D = (G, w, B)$  三元组;  $Y =$  存在长度  $\leq B$  的哈密顿回路.

**搜索/优化问题  $\rightarrow$  判定问题**: 对优化问题  $\Pi^*$ , 增加一个阈值  $K$  得到判定版本“ $\text{OPT} \geq K$  ?”或“ $\text{OPT} \leq K$  ?”. 若  $\Pi^*$  多项式时间可解, 则判定版本也多项式时间可解; 通常反之亦真 (二分搜索阈值).

### 2.2 P 类

$$P = \{ \Pi \mid \exists \text{多项式时间确定型算法判定} \Pi \}.$$

包括: 排序、最小生成树、最短路径、最大流、线性规划、2-SAT 等.

## 2.3 NP 类 (多项式时间可验证)

定义:  $\Pi = \langle D, Y \rangle \in \text{NP}$ , 当存在多项式时间算法  $A(I, t)$  和多项式  $p$ , 使得

$$I \in Y \iff \exists t, |t| \leq p(|I|), A(I, t) = \text{Yes}.$$

$t$  称为 **证据 (certificate)**,  $A$  称为 **验证算法**.

等价定义: 存在非确定型图灵机 (NTM) 在多项式时间接受  $\Pi$ . NTM 的执行分两步:

1. **猜想 (guess)**: 在多项式步内非确定地猜出长度  $\leq p(|I|)$  的字符串  $t$ ;
2. **检查 (verify)**: 用确定型多项式时间算法验证  $t$ .

定义  $I \in Y \iff$  存在某条计算路径接受.

[NOTE] NP 名称来源

NP = Nondeterministic Polynomial, 不是 “Non-Polynomial”. 这是初学者最常见的错误.

定理:  $P \subseteq \text{NP}$ .

证:  $\Pi \in P$  时取验证算法  $A(I, t) =$  “忽略  $t$ , 直接对  $I$  跑多项式算法”即可.  $\square$

著名公开问题:  $P \stackrel{?}{=} \text{NP}$ . 普遍相信  $P \neq \text{NP}$ .

## 3. 多项式时间变换 ( $\leq_p$ )

### 3.1 定义

判定问题  $\Pi_1 = \langle D_1, Y_1 \rangle, \Pi_2 = \langle D_2, Y_2 \rangle$ . 函数  $f: D_1 \rightarrow D_2$  是  $\Pi_1$  到  $\Pi_2$  的 **多项式时间变换**, 当:

1.  $f$  多项式时间可计算;
2.  $\forall I \in D_1: I \in Y_1 \iff f(I) \in Y_2$ .

记  $\Pi_1 \leq_p \Pi_2$ . 直觉:  $\Pi_2$  至少和  $\Pi_1$  一样难, 因为可以用  $\Pi_2$  的算法解  $\Pi_1$ .

[WARN] 归约方向最易错

$\Pi_1 \leq_p \Pi_2$  表示“ $\Pi_2$  比  $\Pi_1$  难”(或等难). 证某问题  $\Pi$  是 NP-Hard, 要 从已知 NPC 归约到  $\Pi$ , 写作  $\text{NPC} \leq_p \Pi$ , 不是反过来.

### 3.2 例: $\text{HC} \leq_p \text{TSP}$

给定 HC 实例  $G = \langle V, E \rangle$ , 构造 TSP 实例  $f(G) = (V, d, D)$ :

$$d(u, v) = \begin{cases} 1, & (u, v) \in E \\ 2, & (u, v) \notin E \end{cases}, \quad D = |V|.$$

正确性:  $G$  含哈密顿回路  $\iff$  TSP 实例存在长度  $\leq |V|$  的回路 (即所有边权全为 1).

构造时间  $O(n^2)$ . 故  $\text{HC} \leq_p \text{TSP}$ .

### 3.3 例: 最大生成树 $\leq_p$ 最小生成树

给定连通赋权图  $G = \langle V, E, W \rangle$  和阈值  $D$ , 构造:

$$n = |V|, M = \max\{W(e) \mid e \in E\} + 1, W'(e) = M - W(e), B = (n - 1)M - D.$$

若  $T$  是  $G$  的生成树,  $W'(T) = (n - 1)M - W(T)$ . 故

$$W(T) \geq D \iff W'(T) \leq (n - 1)M - D = B.$$

故最大生成树  $\leq_p$  最小生成树. 反推 (因为最小生成树  $\in P$ ): 最大生成树  $\in P$ .

### 3.4 性质

**定理 1 (传递性):**  $\Pi_1 \leq_p \Pi_2, \Pi_2 \leq_p \Pi_3 \Rightarrow \Pi_1 \leq_p \Pi_3$ .

证: 设  $f, g$  分别为  $\Pi_1 \rightarrow \Pi_2, \Pi_2 \rightarrow \Pi_3$  的多项式变换, 计算时间上界分别为  $p, q$  (单调递增多项式). 令  $h(I) = g(f(I))$ .

- 时间: 计算  $f(I)$  用  $p(|I|)$  步;  $f(I)$  是合理指令的输出, 单步输出长度  $\leq k$  (常数), 故  $|f(I)| \leq k \cdot p(|I|)$ . 计算  $g(f(I))$  用  $q(|f(I)|) \leq q(kp(|I|))$  步. 合计  $p(|I|) + q(kp(|I|))$ , 多项式.
- 正确性:  $I \in Y_1 \iff f(I) \in Y_2 \iff g(f(I)) \in Y_3$ .

故  $h$  是  $\Pi_1 \rightarrow \Pi_3$  的多项式变换.  $\square$

**定理 2 (向下封闭):**  $\Pi_1 \leq_p \Pi_2$  且  $\Pi_2 \in P \Rightarrow \Pi_1 \in P$ .

证: 设  $f$  是变换 ( $A$  计算  $f$ ),  $B$  是  $\Pi_2$  的多项式算法. 构造  $\Pi_1$  算法  $C$ : 输入  $I \in D_1$ , 跑  $A$  得  $f(I)$ , 再跑  $B$  输出.  $C$  多项式时间, 且  $C$  输出 Yes  $\iff B$  输出 Yes  $\iff f(I) \in Y_2 \iff I \in Y_1$ .  $\square$

**推论:**  $\Pi_1 \leq_p \Pi_2$  且  $\Pi_1$  难解  $\Rightarrow \Pi_2$  难解 (逆否).

应用: 由  $HC \leq_p TSP$ , 若  $HC$  难解则  $TSP$  难解.

## 4. NP-完全 (NPC) 与 NP-难 (NP-Hard)

### 4.1 定义

$\Pi$  是 **NP-难 (NP-Hard)**: 对所有  $\Pi' \in NP$ ,  $\Pi' \leq_p \Pi$ .

$\Pi$  是 **NP-完全 (NPC)**:  $\Pi$  是 NP-难 且  $\Pi \in NP$ .

记号:  $NPC \subseteq NP\text{-Hard}$ .  $NP\text{-Hard}$  类问题不一定可验证, 可能严格难于  $NP$  (如停机问题  $NP\text{-Hard}$  但不可计算, 故不在  $NP$ ).

### 4.2 关键定理

**定理 3:** 若存在 NP-难问题  $\Pi \in P$ , 则  $P = NP$ .

证: 任取  $\Pi' \in NP$ ,  $\Pi' \leq_p \Pi$ . 由定理 2 (向下封闭),  $\Pi' \in P$ . 故  $NP \subseteq P$ , 又  $P \subseteq NP$ , 得  $P = NP$ .  $\square$

**推论:** 假设  $P \neq NP$ ,  $NP\text{-难}$  问题都不在  $P$ .

**定理 4:** 若  $\Pi$  NP-难,  $\Pi \leq_p \Pi'$ , 则  $\Pi'$  NP-难.

证: 任取  $\Pi'' \in \text{NP}$ , 由  $\Pi$  NP-难,  $\Pi'' \leq_p \Pi$ ; 又  $\Pi \leq_p \Pi'$ , 由传递性  $\Pi'' \leq_p \Pi'$ .  $\square$

推论 (“NPC 证明捷径”): 证  $\Pi'$  是 NPC, 只需两步:

1.  $\Pi' \in \text{NP}$  (给验证算法/NTM);
2. 找一个已知 NPC  $\Pi$ , 证  $\Pi \leq_p \Pi'$ .

这是几乎所有 NPC 证明的模板.

## 5. Cook-Levin 定理 (SAT 是 NPC)

### 5.1 SAT 问题

合取范式 (CNF): 形如  $C_1 \wedge C_2 \wedge \cdots \wedge C_m$ , 每个  $C_j$  是若干文字 (变元或其否定) 的析取.

SAT (Boolean Satisfiability): 给定 CNF 公式  $F$ , 是否存在赋值使  $F$  为真?

例:  $F = (\bar{x}_1 \vee x_2) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge x_2$ . 取  $t(x_1) = 1, t(x_2) = 1, t(x_3) = 0$  使  $F = 1$ , 故 SAT.

### 5.2 定理 (Cook 1971, Levin 1973)

Cook-Levin 定理: SAT 是 NP-完全的.

SAT  $\in \text{NP}$ : 验证算法接收赋值  $t$ , 在  $O(|F|)$  时间计算  $F$  在  $t$  下的真值, 输出 Yes 当且仅当  $F$  真.

NP-难性证明大纲:

任取  $\Pi \in \text{NP}$ , 由定义存在 NTM  $M$  在多项式  $p(n)$  时间接受  $\Pi$ . 目标: 对任意输入  $x$  (规模  $n$ ), 多项式时间构造 CNF 公式  $\varphi_x$  使得

$$x \in Y_\Pi \iff \varphi_x \text{ 可满足.}$$

编码思路: 计算 tableau

$M$  接受  $x$  当且仅当存在一条接受计算路径, 长度  $\leq p(n)$ , 工作带也只用前  $p(n)$  格. 把这条路径表示成 tableau (二维表):

$$\text{tableau}[i][j] = \text{第 } i \text{ 步时第 } j \text{ 格的内容, } 0 \leq i, j \leq p(n).$$

每格内容: 工作带符号  $\cup$  {状态  $\times$  符号} (状态附在读写头位置的格上). 设符号集为  $\Gamma$ , 状态集为  $Q$ , “元符号”集合  $C = \Gamma \cup (Q \times \Gamma)$ .

布尔变元

对每个  $(i, j, s) \in [0, p(n)]^2 \times C$ , 引入变元

$$y_{i,j,s} = 1 \iff \text{tableau}[i][j] = s.$$

变元总数  $O(p(n)^2 \cdot |C|) = O(p(n)^2)$ , 多项式.

## 约束 (CNF 子句)

公式  $\varphi_x$  是以下子句的合取:

1. **格状态唯一:** 对每个  $(i, j)$ , 恰有一个  $s$  使  $y_{i,j,s} = 1$ . 写成  $(\bigvee_{s \in C} y_{i,j,s}) \wedge \bigwedge_{s \neq s'} (\neg y_{i,j,s} \vee \neg y_{i,j,s'})$ .
2. **初始条件:** 第 0 行对应初始格局.  $y_{0,0,(q_0,x_1)} = 1, y_{0,j,x_{j+1}} = 1$  对  $1 \leq j \leq n-1, y_{0,j,\square} = 1$  对  $n \leq j \leq p(n)$  (空白符).
3. **接受条件:** 存在  $i$  使第  $i$  行某格状态为接受状态.  $\bigvee_{i,j,q_{acc},s} y_{i,j,(q_{acc},s)}$ .
4. **转移合法:** 这是最关键的约束. NTM 的下一格内容只取决于当前格及左右两个邻格. 故第  $i+1$  行第  $j$  格的内容由第  $i$  行的  $(j-1, j, j+1)$  决定. 对每个  $2 \times 3$  “窗口”  $W$ , 若  $W$  与 NTM 的某条 (非确定) 转移规则一致, 标记为“合法窗口”. 对每个  $(i, j)$ , 加约束: 以  $(i, j)$  为中心的窗口必须合法. 子句形如  $\bigvee_{(s_1,s_2,s_3,s'_1,s'_2,s'_3) \text{ 合法}} (y_{i,j-1,s_1} \wedge \cdots \wedge y_{i+1,j+1,s'_3})$ .

合法窗口集合大小由  $M$  的转移表决定 (常数), 故每个  $2 \times 3$  位置的约束子句长度多项式. NTM 的非确定性体现在: 同一前置内容可对应多个合法的后继内容, 这给了赋值选择空间.

## 正确性与多项式性

- **正确性:**  $\varphi_x$  可满足  $\iff$  存在一组  $y$  赋值描述一个合法的、接受的计算路径  $\iff M$  接受  $x \iff x \in Y_\Pi$ .
- **构造时间:** 变元数  $O(p(n)^2)$ , 子句数  $O(p(n)^2)$  (常数子句长度), 故公式长度  $O(p(n)^2)$ . CNF 化后还需多项式因子. 总体多项式时间.

故  $\Pi \leq_p \text{SAT}$ ,  $\Pi$  任取  $\Rightarrow \text{SAT}$  是 NP-难.  $\square$

### [NOTE] Cook-Levin 的意义

此定理是第一个“天然”NPC 问题. 之后所有 NPC 证明都可从 SAT 出发归约, 无须再回到 NTM 编码. 这是 1971 年 Cook 工作的里程碑意义.

## 6. 3-SAT 是 NPC

### 6.1 3-SAT

**3 元合取范式:** 每个简单析取式恰有 3 个文字的 CNF.

**3-SAT:** 给定 3-CNF 公式  $F$ , 是否可满足?

### 6.2 定理: 3-SAT NPC

**3-SAT  $\in$  NP:** 同 SAT.

**SAT  $\leq_p$  3-SAT (局部替换法):** 给定 CNF  $F = C_1 \wedge \cdots \wedge C_m$ , 对每个子句  $C_j$  单独构造 3-CNF  $F_j$  使得  $C_j$  可满足  $\iff F_j$  可满足.  $f(F) = F_1 \wedge \cdots \wedge F_m$ .

设  $C_j$  有  $k$  个文字  $z_1, z_2, \dots, z_k$ :

- (1)  $k = 1$ :  $C_j = z_1$ . 引入新变元  $y_{j1}, y_{j2}$ , 令

$$F_j = (z_1 \vee y_{j1} \vee y_{j2}) \wedge (z_1 \vee y_{j1} \vee \bar{y}_{j2}) \wedge (z_1 \vee \bar{y}_{j1} \vee y_{j2}) \wedge (z_1 \vee \bar{y}_{j1} \vee \bar{y}_{j2}).$$

四个子句覆盖  $(y_{j1}, y_{j2})$  所有取值组合, 故  $F_j$  可满足  $\iff z_1 = 1$ .

(2)  $k = 2$ :  $C_j = z_1 \vee z_2$ . 引入新变元  $y_j$ , 令

$$F_j = (z_1 \vee z_2 \vee y_j) \wedge (z_1 \vee z_2 \vee \bar{y}_j).$$

(3)  $k = 3$ :  $F_j = C_j$ .

(4)  $k \geq 4$ :  $C_j = z_1 \vee z_2 \vee \dots \vee z_k$ . 引入  $k - 3$  个新变元  $y_{j1}, \dots, y_{j,k-3}$ , 令

$$F_j = (z_1 \vee z_2 \vee y_{j1}) \wedge (\bar{y}_{j1} \vee z_3 \vee y_{j2}) \wedge (\bar{y}_{j2} \vee z_4 \vee y_{j3}) \wedge \dots \wedge (\bar{y}_{j,k-3} \vee z_{k-1} \vee z_k).$$

正向: 若  $t$  满足  $C_j$ , 设  $t(z_i) = 1$ . 令

- $i \in \{1, 2\}$ : 所有  $y_{js} = 0$ ;
- $i \in \{k-1, k\}$ : 所有  $y_{js} = 1$ ;
- $3 \leq i \leq k-2$ :  $y_{js} = 1$  ( $1 \leq s \leq i-2$ ),  $y_{js} = 0$  ( $i-1 \leq s \leq k-3$ ).

可验证每个子句被满足.

反向: 若  $t(F_j) = 1$ , 反证  $C_j$  不满足即所有  $z_i = 0$ . 此时  $F_j$  变为只含  $y$  变元的链:

$$y_{j1} \wedge (\bar{y}_{j1} \vee y_{j2}) \wedge \dots \wedge (\bar{y}_{j,k-4} \vee y_{j,k-3}) \wedge \bar{y}_{j,k-3}.$$

由  $y_{j1} = 1$  链式推  $y_{j2} = 1, \dots, y_{j,k-3} = 1$ , 但末尾  $\bar{y}_{j,k-3} = 0$ , 矛盾.  $\square$

例:  $F = (x_1 \vee \bar{x}_2) \wedge \bar{x}_3 \wedge (x_1 \vee x_2 \vee \bar{x}_3 \vee x_4 \vee x_5)$ . 构造

$$\begin{aligned} F_1 &= (x_1 \vee \bar{x}_2 \vee y_{11}) \wedge (x_1 \vee \bar{x}_2 \vee \bar{y}_{11}), \\ F_2 &= (\bar{x}_3 \vee y_{21} \vee y_{22}) \wedge (\bar{x}_3 \vee y_{21} \vee \bar{y}_{22}) \wedge (\bar{x}_3 \vee \bar{y}_{21} \vee y_{22}) \wedge (\bar{x}_3 \vee \bar{y}_{21} \vee \bar{y}_{22}), \\ F_3 &= (x_1 \vee x_2 \vee y_{31}) \wedge (\bar{y}_{31} \vee \bar{x}_3 \vee y_{32}) \wedge (\bar{y}_{32} \vee x_4 \vee x_5). \end{aligned}$$

构造时间多项式, 故  $\text{SAT} \leq_p 3\text{-SAT}$ ,  $3\text{-SAT NPC}$ .  $\square$

## 7. MAX-SAT 是 NPC (限制法)

MAX-SAT: 给定子句  $C_1, \dots, C_m$  和正整数  $K$ , 是否存在赋值使至少  $K$  个子句为真?

子问题:  $\Pi' = \langle D', Y' \rangle$  是  $\Pi = \langle D, Y \rangle$  的子问题, 当  $D' \subseteq D$  且  $Y' = D' \cap Y$ .

限制法:  $\Pi'$  是  $\Pi$  子问题, 若  $\Pi'$  NP-难  $\Rightarrow \Pi$  NP-难. 直观: 一般情况不比特殊情况容易.

定理: MAX-SAT 是 NPC.

证: SAT 是 MAX-SAT 的子问题 (取  $K = m$  即要求全部子句成立). 故  $\text{SAT} \leq_p \text{MAX-SAT}$  (恒等映射加  $K = m$ ). 已知 SAT NPC, 故 MAX-SAT NP-难. MAX-SAT  $\in$  NP (验证算法接收赋值, 数满足子句).  $\square$

## 8. 图论 NPC 三件套: CLIQUE / IS / VC

### 8.1 问题定义

- CLIQUE (团):  $G = (V, E), k$ . 是否存在  $|S| \geq k$  使  $S$  内任两点相邻?
- INDEPENDENT-SET (IS, 独立集):  $G = (V, E), k$ . 是否存在  $|S| \geq k$  使  $S$  内任两点不相邻?
- VERTEX-COVER (VC, 顶点覆盖):  $G = (V, E), k$ . 是否存在  $|S| \leq k$  使每条边至少一端在  $S$ ?

### 8.2 三者的两两等价归约

引理: 设  $\bar{G}$  是  $G$  的补图. 则  $S$  是  $G$  的团  $\iff S$  是  $\bar{G}$  的独立集.

证:  $S$  是  $G$  的团  $\iff S$  内任两点在  $G$  中相邻  $\iff S$  内任两点在  $\bar{G}$  中不相邻  $\iff S$  是  $\bar{G}$  的独立集.  $\square$

引理:  $S$  是  $G$  的独立集  $\iff V \setminus S$  是  $G$  的顶点覆盖.

证:  $S$  独立集  $\iff G$  内每条边至少有一端不在  $S \iff$  每条边至少一端在  $V \setminus S \iff V \setminus S$  是顶点覆盖.  $\square$

故归约链 (恒等映射加补图/补集):

$$\text{CLIQUE} \leq_p \text{IS} \leq_p \text{VC}, \quad \text{VC} \leq_p \text{IS} \leq_p \text{CLIQUE}.$$

三者两两  $\leq_p$ -等价: 一个 NPC 则三个全 NPC.

### 8.3 3-SAT $\leq_p$ IS (gadget 构造)

给定 3-CNF  $F = C_1 \wedge \dots \wedge C_m$ , 构造图  $G$  和  $k = m$ :

- 子句三角形 (clause gadget): 对每个子句  $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$ , 建一个 3 顶点三角形 (三条内边), 顶点分别标记三个文字  $l_{j1}, l_{j2}, l_{j3}$ .
- 冲突边 (consistency edge): 对不同子句的两个顶点  $u$  (标记  $x$ ) 和  $v$  (标记  $\bar{x}$ ), 加一条边  $uv$ .

正确性:  $F$  可满足  $\iff G$  有大小  $\geq m$  的独立集.

- $\Rightarrow$ : 设  $t$  满足  $F$ . 每子句中至少一个文字为真, 选其顶点  $v_j \in S$ .  $|S| = m$ . (i)  $v_j$  与  $v_{j'}$  不同三角形, 无三角内边; (ii) 若有冲突边  $v_j v_{j'}$ , 则  $v_j$  与  $v_{j'}$  互补, 但  $t$  同时使两者真, 矛盾. 故  $S$  独立.
- $\Leftarrow$ : 设  $|S| = m$  独立. 每三角内顶点两两相邻,  $S$  在每三角至多选 1 个; 总共  $m$  三角, 每三角恰好选 1 个. 设  $S$  中所选文字赋值为 1. 由冲突边的存在,  $S$  中无  $x$  与  $\bar{x}$  同选, 故赋值一致. 每子句至少一个文字真,  $F$  满足.

构造时间  $O(m^2)$ . 故 3-SAT  $\leq_p$  IS, IS NPC.  $\square$

由 8.2 等价, CLIQUE NPC, VC NPC.

#### [EX] 实例

$F = (x_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$ . 建 6 顶点 2 三角形, 冲突边:  $x_1 \leftrightarrow \bar{x}_1, x_2 \leftrightarrow \bar{x}_2, x_3 \leftrightarrow \bar{x}_3$  (各 1 条). 求 2-独立集  $\iff F$  可满足.

## 8.4 SET-COVER

SET-COVER: 全集  $U$ , 子集族  $\mathcal{F} = \{S_1, \dots, S_m\}$ , 正整数  $k$ . 是否存在  $\leq k$  个子集覆盖  $U$ ?

$VC \leq_p$  SET-COVER:  $U = E$  (边集),  $S_v = \{e \in E : v \in e\}$  (顶点关联边). 顶点覆盖  $\iff$  集合覆盖. 故 SET-COVER NPC.

## 9. 哈密顿回路 (HC) 与货郎问题 (TSP)

### 9.1 HC

HC:  $G$  是否存在哈密顿回路 (访问每点恰一次)?

$VC \leq_p$  HC (gadget 构造大纲): 给定  $(G = (V, E), k)$  的 VC 实例, 构造图  $H$  含哈密顿回路  $\iff G$  有  $\leq k$  的顶点覆盖.

构造思路 (Karp 1972):

- **边 gadget**: 每条边  $e = (u, v)$  用 12 个内部节点的子图替代, 该子图只能由两种方式被哈密顿路径“穿过”:  
(a) 仅从  $u$  侧穿一次; (b) 仅从  $v$  侧穿一次; (c) 两侧各穿一次. 三选一.
- **选择 gadget**: 加  $k$  个“选择顶点” $s_1, \dots, s_k$ , 与每个顶点  $v \in V$  的 gadget 入口/出口相连. 哈密顿回路通过  $s_i$  进入某条“顶点链”, 走完该顶点所属边 gadget, 回到选择顶点.

**正确性**: 顶点覆盖  $C$  ( $|C| = k$ )  $\iff$  选择  $k$  个顶点对应的链, 每条边 gadget 被至少一端覆盖, 哈密顿回路存在.

具体构造细节繁琐 (Karp 原始论文), 此处略.

HC  $\in$  NP: 验证算法接收回路, 检查每点恰访问一次. 故 HC NPC.

### 9.2 TSP

TSP: 完全图  $(V, d)$ , 阈值  $B$ . 是否存在长度  $\leq B$  的哈密顿回路?

HC  $\leq_p$  TSP: 已在 3.2 给出. TSP  $\in$  NP. 故 TSP NPC.

### 9.3 有向 HC

有向 HC: 有向图是否有哈密顿回路?

HC  $\leq_p$  有向 HC: 每条无向边换两条有向边. 或反过来 有向 HC  $\leq_p$  HC (用 3 节点代替每个有向边).

## 10. 数论类 NPC: SUBSET-SUM / PARTITION / KNAPSACK

### 10.1 SUBSET-SUM

SUBSET-SUM: 正整数集  $S = \{a_1, \dots, a_n\}$  和目标  $T$ . 是否存在  $S' \subseteq S$  使  $\sum_{a \in S'} a = T$ ?

3-SAT  $\leq_p$  SUBSET-SUM (gadget): 给定 3-CNF  $F$  含  $n$  个变元  $x_1, \dots, x_n$  和  $m$  个子句  $C_1, \dots, C_m$ .

构造  $2n + 2m$  个  $(n + m)$  位十进制数:

- 对每个  $x_i$ , 两个数  $t_i, f_i$  (代表  $x_i = 1$  或 0):
  - 第  $i$  位 (变元位) 都为 1, 其余变元位 0;
  - 第  $n + j$  位 (子句位) 为 1 当且仅当对应文字 ( $x_i$  或  $\bar{x}_i$ ) 出现在  $C_j$ .
- 对每个  $C_j$ , 两个“填充数” $g_j, h_j$ :

- 仅在第  $n + j$  位有值, 分别为 1 和 2.

目标  $T$ : 每个变元位 = 1 (恰选一个  $t_i$  或  $f_i$ ), 每个子句位 = 4.

**正确性:** 变元位约束保证赋值一致; 子句位 = 4 要求该子句对应的真文字数 + 选用的填充值之和 = 4, 即至少一个真文字 (1, 2, 或 3 个真) 配合填充数 (0, 1, 2, 或 3 之和) 达到 4. 由填充数最多贡献 3, 至少需要 1 个真文字.

构造为十进制设计 (无进位), 多项式时间. 故 SUBSET-SUM NPC.

## 10.2 PARTITION

**PARTITION:** 正整数集  $S$ , 是否可划分为两个子集  $S_1, S_2$  使  $\sum S_1 = \sum S_2$ ?

**SUBSET-SUM  $\leq_p$  PARTITION:** 设 SUBSET-SUM 实例  $(S = \{a_1, \dots, a_n\}, T)$ ,  $A = \sum a_i$ . 构造 PARTITION 实例  $S' = S \cup \{A + 2T, 3A - 2T\}$  略繁琐, 常用变体:  $S' = S \cup \{2T - A, 2A - 2T\}$  当  $T < A$  时 (跳过细节). 故 PARTITION NPC.

## 10.3 0-1 KNAPSACK 判定版

**KNAPSACK:** 物品  $(w_i, v_i)$ , 重量上限  $B$ , 价值下限  $K$ . 是否有子集  $T$  使  $\sum_T w_i \leq B$  且  $\sum_T v_i \geq K$ ?

**SUBSET-SUM  $\leq_p$  KNAPSACK:** 取  $w_i = v_i = a_i$ ,  $B = K = T$ . 故 KNAPSACK NPC.

[WARN] 伪多项式与强 NPC

0-1 背包有 DP  $O(nB)$ . 这是 **伪多项式**:  $B$  的输入规模是  $\log B$  比特,  $O(nB) = O(n \cdot 2^{\log B})$  在比特长度意义下指数. 故 KNAPSACK 仍 NPC.

一个 NPC 问题若不存在伪多项式算法 (除非  $P=NP$ ), 称作 **强 NPC**. 如 TSP, 3-SAT. KNAPSACK 是 **弱 NPC** (有伪多项式 DP).

## 11. 其他 NPC 问题

### 11.1 3-COLORING

**3-COLORING:** 图  $G$  是否能用 3 种颜色染色, 相邻不同色?

**3-SAT  $\leq_p$  3-COLORING (gadget):**

- **基础三角形**  $T_F, T_T, T_B$  (False, True, Base) 三顶点两两相邻, 强制三色.
- **变元 gadget:** 每变元  $x_i$  配两顶点  $v_i, \bar{v}_i$ , 加边  $v_i \bar{v}_i$  和  $v_i T_B, \bar{v}_i T_B$ . 强制  $v_i, \bar{v}_i$  同时为  $T/F$  中一个 (互补).
- **子句 gadget:** 对子句  $(l_1 \vee l_2 \vee l_3)$ , 用 6 节点 OR-gate 子图, 使该 gadget 可 3 染色  $\iff$  至少一文字为  $T$ .

故 3-COLORING NPC.

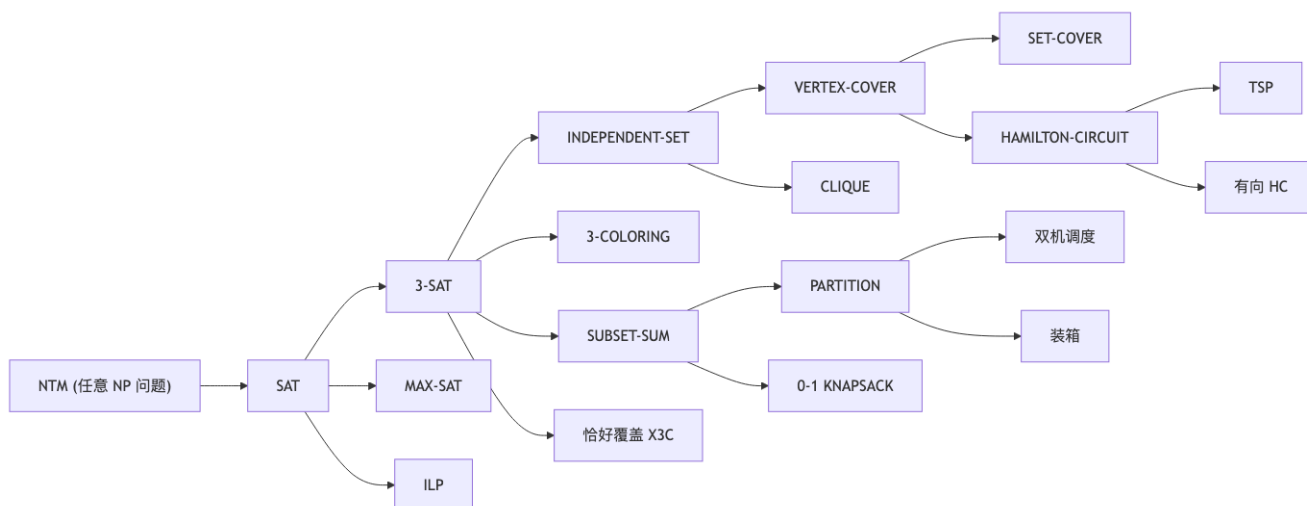
### 11.2 双机调度 / 装箱 / ILP

- **双机调度 (P2||C\_max):** 两台机器,  $n$  个任务时长  $p_i$ , 阈值  $K$ . 能否使最大完成时间  $\leq K$ ? PARTITION  $\leq_p$  双机调度 (取  $K = \sum p_i / 2$ ).
- **装箱 (BIN-PACKING):**  $n$  个物品  $s_i \in (0, 1]$ ,  $k$  个容量为 1 的桶, 能否装入? PARTITION  $\leq_p$  装箱.
- **ILP (整数线性规划):** 给定  $A \in \mathbb{Z}^{m \times n}, b, c, k$ . 是否存在  $x \in \mathbb{Z}^n$  使  $Ax \leq b, c^T x \geq k$ ? SAT 可直接归约为 ILP (布尔变元  $x_i \in \{0, 1\}$ , 子句改成线性不等式). 故 ILP NPC. 注 LP  $\in P$ , **整数** 约束使 ILP 变 NPC.

## 11.3 恰好覆盖 (EXACT-COVER / X3C)

**X3C:** 全集  $U$  ( $|U| = 3k$ ), 子集族  $\mathcal{F}$  每  $S \in \mathcal{F}$  含恰 3 个元素. 是否存在  $\mathcal{F}' \subseteq \mathcal{F}$  划分  $U$  (恰好覆盖每元素一次)?  $3\text{-SAT} \leq_p \text{X3C}$ .

## 12. NP-完全归约图



记忆要点: SAT 是“根”, 3-SAT 是“主桥”. 大多 NPC 证明从 3-SAT 出发.

## 13. NP-Hard vs NPC 区分

| 概念                      | $\in \text{NP?}$ | 所有 NP 问题 $\leq_p$ 它? |
|-------------------------|------------------|----------------------|
| NPC                     | 是                | 是                    |
| NP-Hard                 | 不一定              | 是                    |
| NP                      | 是                | 不一定                  |
| NP-Hard $\setminus$ NPC | 否                | 是                    |

典型 NP-Hard 但非 NPC:

- 停机问题: NP-Hard ( $\text{SAT} \leq_p \text{停机}$ , 因为停机不可计算可“判定”一切), 但不可计算故不在 NP.
- TSP 优化版 (求最短回路长度): NP-Hard, 但优化版不是判定问题, 严格不属 NP. 判定版 TSP  $\in \text{NPC}$ .

## 14. coNP、PSPACE 与复杂度阶梯

### 14.1 coNP

coNP:  $\Pi \in \text{coNP} \iff \bar{\Pi} \in \text{NP}$  (其中  $\bar{\Pi}$  把 Yes/No 翻转).

例: TAUTOLOGY (所有赋值使  $F$  真)  $\in \text{coNP}$ , 因  $\overline{\text{TAUT}} = \text{“存在使 } F \text{ 为假的赋值”} = \overline{\text{SAT}}$  的某种形式.

已知:  $P \subseteq \text{NP} \cap \text{coNP}$ . 公开问题:  $\text{NP} \stackrel{?}{=} \text{coNP}$  (普遍相信不等).

## 14.2 PSPACE

PSPACE: 多项式空间内可判定的问题. 包括 QBF (量词布尔公式), 一些两人博弈 (围棋广义版).

关系:

$$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME.$$

由时间层次定理,  $P \subsetneq EXPTIME$ , 故上链中至少一处严格.

## 14.3 复杂度阶梯总览

| 类       | 资源          |
|---------|-------------|
| L       | 对数空间        |
| P       | 多项式时间       |
| NP      | 多项式时间可验证    |
| coNP    | 否定问题在 NP    |
| PSPACE  | 多项式空间       |
| EXPTIME | 指数时间        |
| EXSPACE | 指数空间        |
| R       | 可计算 (无资源限制) |

## 15. 处理 NPC 的实践策略

| 策略         | 思路                                            | 例                                                                |
|------------|-----------------------------------------------|------------------------------------------------------------------|
| 近似算法       | 多项式时间保证 $\leq \alpha \cdot OPT$               | VC 的 2-近似 (任取一条边的两端都入解), TSP (Euclidean) 的 1.5-近似 (Christofides) |
| 参数化算法      | 固定某参数 $k$ , 算法 $O(f(k) \cdot \text{poly}(n))$ | $k$ -VC 的 $O(2^k \cdot n)$ , FPT 类                               |
| 启发式 / 元启发式 | 无理论保证, 实践有效                                   | 模拟退火、遗传算法、局部搜索                                                   |
| 限制特例       | 输入限制后多项式可解                                    | 平面图 IS, 树上 VC, 区间图 CLIQUE                                        |
| 分支限界       | 智能剪枝精确求解                                      | ILP 求解器 (CPLEX, Gurobi), SAT 求解器 (CDCL)                          |
| 伪多项式 DP    | $O(\text{poly}(n) \cdot V)$ , $V$ 是数值         | 0-1 背包 DP $O(nB)$                                                |
| 随机算法       | 概率正确/优化                                       | Karger 最小割 (随机收缩)                                                |
| 平均情况分析     | 最坏 NPC, 平均易解                                  | 随机 3-SAT 在子句/变元比远离阈值时                                            |

例: VC 的 2-近似:

APPROX-VC(G):

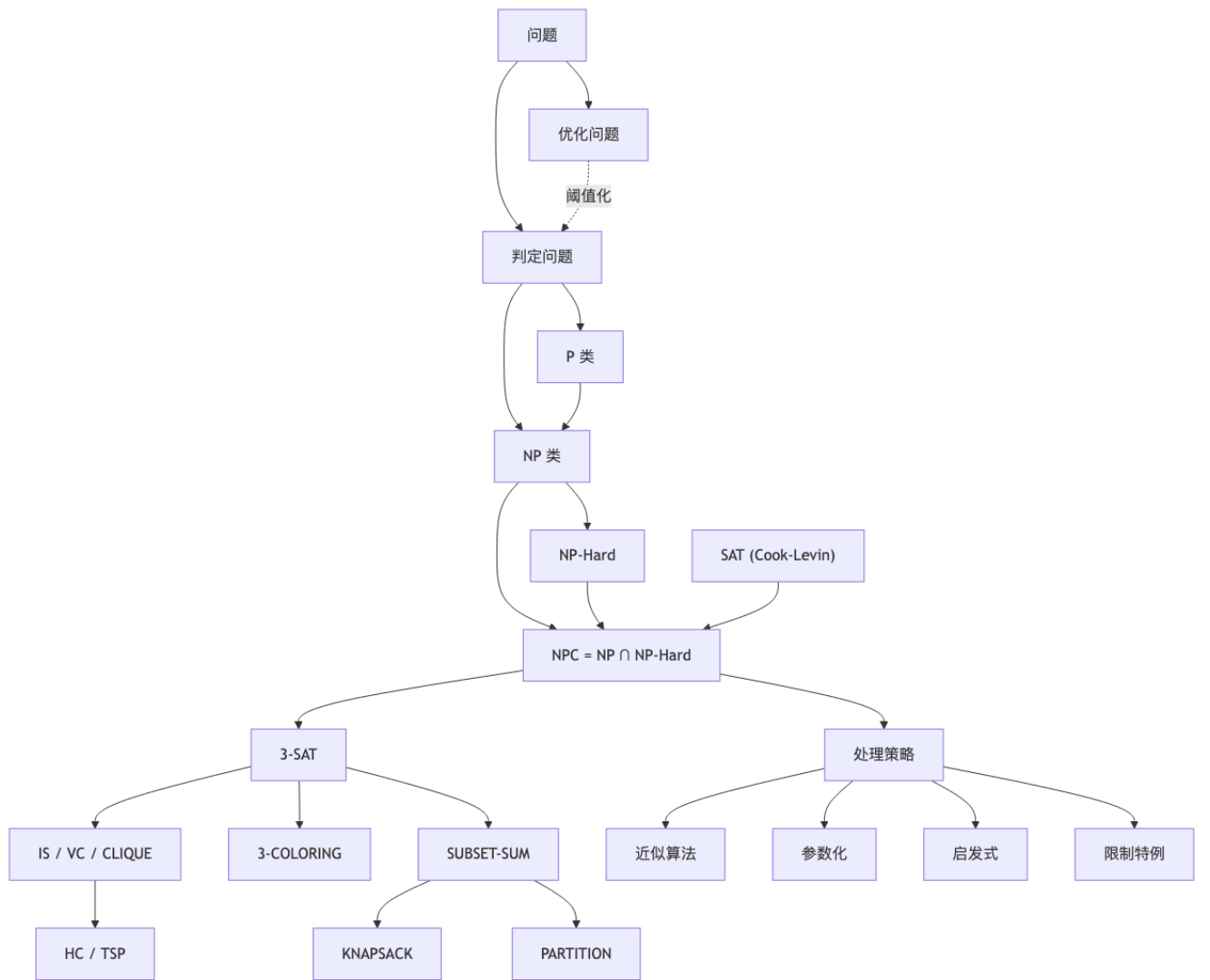
```

C = ∅
while ∃ edge (u, v) in E:
    add u, v to C
    remove all edges incident to u or v
return C

```

保证:  $|C| \leq 2 \cdot |VC_{\min}|$ . 证: 设  $C$  取了  $k$  条边的端点, 共  $2k$  个; 这  $k$  条边互不共端 (匹配), 任意 VC 至少含每条边的一端, 故  $|VC_{\min}| \geq k$ .

## 16. 概念关系



## 17. 关键定理一览

| 定理               | 内容                                                                      |
|------------------|-------------------------------------------------------------------------|
| $\leq_p$ 传递性     | $\Pi_1 \leq_p \Pi_2, \Pi_2 \leq_p \Pi_3 \Rightarrow \Pi_1 \leq_p \Pi_3$ |
| $\leq_p$ 向下封闭    | $\Pi_1 \leq_p \Pi_2, \Pi_2 \in P \Rightarrow \Pi_1 \in P$               |
| NP-难 $\cap$ P 非空 | $\Rightarrow P = NP$                                                    |
| NP-难传递           | $\Pi$ NP-难, $\Pi \leq_p \Pi' \Rightarrow \Pi'$ NP-难                     |
| Cook-Levin       | SAT NPC                                                                 |
| 3-SAT NPC        | $SAT \leq_p 3-SAT$ (局部替换)                                               |
| MAX-SAT NPC      | SAT 子问题 (限制法)                                                           |
| IS NPC           | $3-SAT \leq_p IS$ (三角形 gadget + 冲突边)                                    |

| 定理             | 内容                                      |
|----------------|-----------------------------------------|
| VC/CLIQUE NPC  | 与 IS 等价                                 |
| HC NPC         | $VC \leq_p HC$ (边/选择 gadget)            |
| TSP NPC        | $HC \leq_p TSP$ (边权 1/2)                |
| SUBSET-SUM NPC | $3-SAT \leq_p SUBSET-SUM$ (十进制数构造)      |
| KNAPSACK NPC   | $SUBSET-SUM \leq_p KNAPSACK$            |
| 3-COLOR NPC    | $3-SAT \leq_p 3-COLOR$ (OR-gate gadget) |
| 时间层次           | $P \subsetneq EXPTIME$                  |

## 18. 记号速查

| 符号                                | 含义                                     |
|-----------------------------------|----------------------------------------|
| P                                 | 多项式时间可解                                |
| NP                                | 多项式时间可验证 (nondeterministic polynomial) |
| coNP                              | $\Pi \in coNP \iff \bar{\Pi} \in NP$   |
| PSPACE                            | 多项式空间可解                                |
| EXPTIME                           | 指数时间可解                                 |
| NPC                               | NP-完全 = $NP \cap NP\text{-Hard}$       |
| NP-Hard                           | 所有 NP 问题 $\leq_p$ 它                    |
| $\leq_p$                          | 多项式时间 (Karp) 归约                        |
| SAT                               | 布尔可满足性                                 |
| 3-SAT                             | 3-CNF 可满足性                             |
| IS / VC / CLIQUE                  | 独立集 / 顶点覆盖 / 团                         |
| HC / TSP                          | 哈密顿回路 / 货郎问题                           |
| SUBSET-SUM / KNAPSACK / PARTITION | 数论 NPC                                 |
| MAX-SAT                           | 最大可满足 (至少 K 个子句)                       |
| 3-COLOR / X3C                     | 三染色 / 恰好覆盖                             |
| OPT                               | 优化问题的最优值                               |
| 伪多项式                              | 运行时间是数值的多项式, 不是比特长度的                   |

## 19. 易错点汇总

### [WARN] NP 完全理论易错点

1. NP 是“nondeterministic polynomial”, 与“non-polynomial”无关. 2. NP-Hard 不一定在 NP (如停机问题). 3. 证 NPC 两步: (i)  $\Pi \in NP$ ; (ii) 已知  $NPC \leq_p \Pi$ . 不可漏 (i). 4. 归约方向: 证  $\Pi$  NP-难, 从已知 NPC 归约到  $\Pi$ , 不是反过来. 顺序:  $NPC \rightarrow \Pi$ . 5. 多项式归约必须 **多项式时间** 可计算, 且  $|f(I)|$  必须多项式有界. 6. 伪多项式  $\neq$  多项式. 0-1 KNAPSACK NPC, 但有  $O(nB)$  DP. 7. 优化版与判定版: 复杂度类 (P, NP, NPC) 严格只对判定问题定义. 8. Cook-Levin 把 NP 性 (NTM 接受路径) 编码成布尔公式; tableau 大小  $O(p(n)^2)$ , 子句长度常数. 9. 三色与二色:  $2-COLORING \in P$  (二分图检测),  $3-COLORING$  NPC. 临界跳变. 10.  $LP \in P$ , ILP NPC. 整数约束使复杂度跳变. 11.  $S$  独立集  $\iff V \setminus S$  顶点覆盖, 但  $|IS_{\max}| + |VC_{\min}| = |V|$ , 注意不是补集大小相加随便. 12.  $\leq_p$  (Karp) 与  $\leq_T$  (Cook/Turing) 不同: Karp 是单调用  $f$  计算一次; Turing 可多次调用. NPC 默认 Karp.

[TIP] 期末 / 期中复习要点

1. **必背定义**:  $P$ ,  $NP$  (验证算法 /  $NTM$ ),  $NPC$ ,  $NP\text{-}Hard$ ,  $\leq_p$ . 2. **必证定理**:  $\leq_p$  传递性,  $NPC$  证明捷径 (两步法). 3. **必懂证明大纲**: Cook-Levin ( $NTM$  tableau  $\rightarrow$   $CNF$ ),  $SAT \leq_p 3\text{-}SAT$  (局部替换),  $3\text{-}SAT \leq_p IS$  (三角形 + 冲突边),  $VC \leq_p HC$  (gadget 思路即可),  $HC \leq_p TSP$  (边权  $1/2$ ). 4. **必记 NPC 经典清单**:  $SAT$ ,  $3\text{-}SAT$ ,  $MAX\text{-}SAT$ ,  $IS$ ,  $VC$ ,  $CLIQUE$ ,  $SET\text{-}COVER$ ,  $HC$ ,  $TSP$ ,  $SUBSET\text{-}SUM$ ,  $PARTITION$ ,  $KNAPSACK$ ,  $3\text{-}COLORING$ ,  $ILP$ ,  $BIN\text{-}PACKING$ . 5. **必识别**: 给一个新问题, 能找到合适的  $NPC$  已知问题做归约 (常用桥  $3\text{-}SAT$ ,  $VC$ ,  $SUBSET\text{-}SUM$ ). 6. **处理策略**: 近似/参数化/启发式/限制特例/伪多项式  $DP$ , 各举一例. 7. **复杂度阶梯**:  $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ , 严格关系仅  $P \subsetneq EXPTIME$ .

# Lecture 15 —NP 完全 (2): 归约链、子问题分析与 Turing 归约

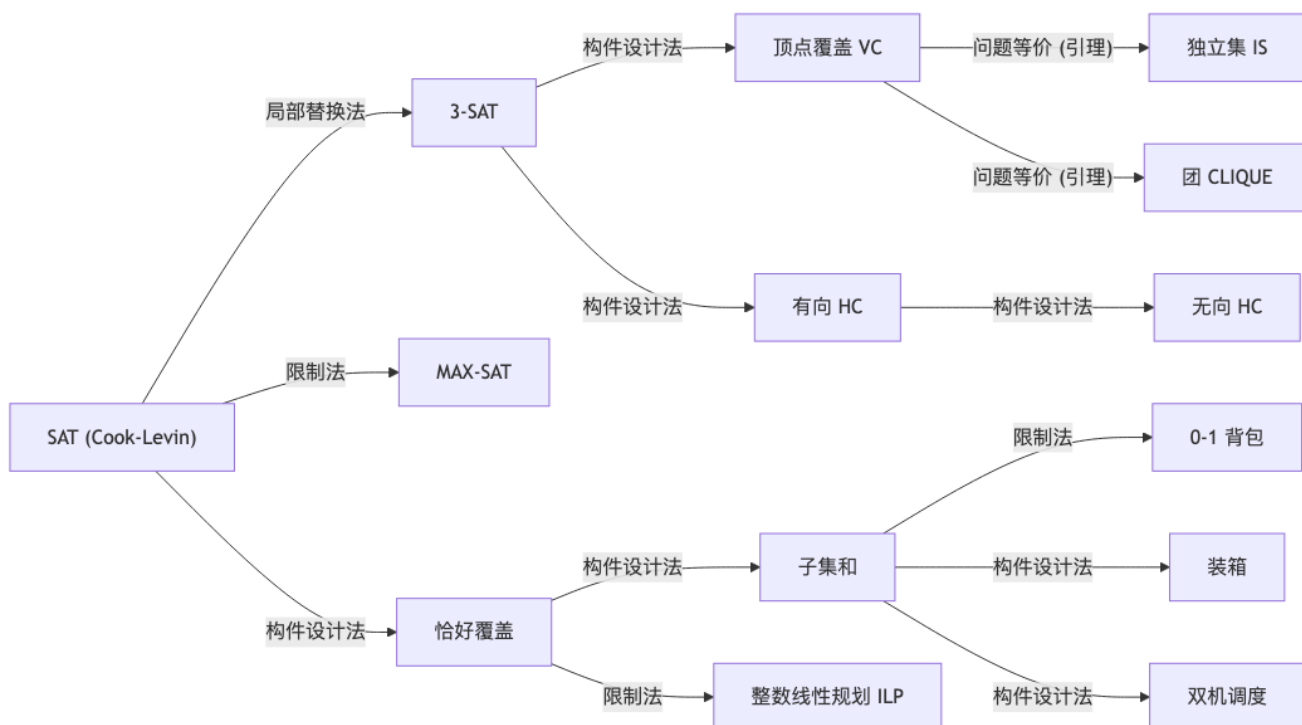
## [TIP] 学习指南

本节核心：用 三种方法 (局部替换法 / 构件设计法 / 限制法) 把 3-SAT 归约到一组核心 NP-完全问题；把 NP-完全理论 应用到优化/搜索问题——引入 Turing 归约、NP-hard、NP-easy、NP-等价的 概念。前置知识：Lec 13 ( $P, NP, \leq_p$ )、Lec 14 (Cook-Levin、 $\leq_p$  传递性、 $SAT \rightarrow 3-SAT$ )。重点关注：(i)  $3-SAT \rightarrow VC$  的构件设计法 (变元构件 + 析取式构件 + 联络边)；(ii)  $VC / 团 / 独立集$  三者两两等价；(iii) 调度问题的子问题谱系图；(iv) TSO 是 NP-等价的证明 (Turing 归约 + 二分法求最优值)。课程意义：本讲把 Lec 14 “NPC 是难度等价类”的抽象论断落到 可操作的归约手段 和 如何对自己的研究问题进行 NPC 判定 的研究方法论。

## 1. 全图：NPC 归约谱系

### [NOTE] 本讲核心地图

围绕 SAT / 3-SAT 这两个“母 NPC”，三种归约手段把难度类向外辐射到下面所有问题。读笔记时随时回看本图，明确每一步证明使用了哪种手段。



### [WARN] 三种归约手段速记

– **限制法**：把目标问题加约束 / 减实例 / 固定参数后恰好是已知 NPC 问题 (例 3-SAT 是 SAT 的子集；0-1 背包是子集和的退化)。– **局部替换法**：用 **机械规则** 把已知问题的实例逐项改写 (例 SAT 的长子句  $C = (\ell_1 \vee \dots \vee \ell_k)$  在引入  $k - 3$  个新变元后变成 3-SAT 的  $k - 2$  条子句)。– **构件设计法**：给目标问题专门设计 **变元构件 / 子句构件 / 联络边** 等 gadget，使其 Yes/No 行为与原问题对应。是最常用也最有“创造性”的手段。

## 2. $3\text{-SAT} \leq_p \text{顶点覆盖 (VC)}$ ：构件设计法范例

### 2.1 三个互相等价的问题

设  $G = \langle V, E \rangle$  是无向图， $V' \subseteq V$ 。

- 定义 15.1 (顶点覆盖)  $V'$  是  $G$  的 **顶点覆盖**  $\iff G$  的每条边至少有一个端点在  $V'$  中。
- 定义 15.2 (团)  $V'$  是  $G$  的 **团**  $\iff \forall u, v \in V', u \neq v, (u, v) \in E$ 。
- 定义 15.3 (独立集)  $V'$  是  $G$  的 **独立集**  $\iff \forall u, v \in V', (u, v) \notin E$ 。

### [NOTE] 三者的对偶关系 (核心引理)

对任意  $G = \langle V, E \rangle$  和  $V' \subseteq V$ ，下面三个命题等价：

1.  $V'$  是  $G$  的 **顶点覆盖**；
2.  $V - V'$  是  $G$  的 **独立集**；
3.  $V - V'$  是补图  $G^c = \langle V, E^c \rangle$  的 **团**。

**证明** (1) $\iff$ (2):  $V'$  覆盖每条边  $\iff$  每条边至少有一个端点  $\in V'$   $\iff$  两端不同时在  $V - V'$   $\iff$   $V - V'$  中任意两点之间无边  $\iff V - V'$  是独立集。(2) $\iff$ (3):  $(u, v) \notin E \iff (u, v) \in E^c$ 。  
□

判定问题：

| 问题     | 实例              | 询问                        |
|--------|-----------------|---------------------------|
| VC     | $G, K \leq  V $ | 是否存在 $ V'  \leq K$ 的顶点覆盖? |
| CLIQUE | $G, J \leq  V $ | 是否存在 $ V'  \geq J$ 的团?    |
| IS     | $G, J \leq  V $ | 是否存在 $ V'  \geq J$ 的独立集?  |

由引理立刻得到 (转换均在线性时间):

$$\text{VC}(G, K) = \text{IS}(G, |V| - K) = \text{CLIQUE}(G^c, |V| - K). \quad (15.1)$$

因此 三者两两多项式可归约 (问题等价), 只需证明其中一个是 NPC, 其余两个自动 NPC。

## 2.2 定理 (VC $\in$ NPC)

[WARN] 重要定理

定理 15.1 顶点覆盖 (VC) 是 NP-完全的。

证 (两步法):

(a) VC  $\in$  NP. 非确定性算法: 猜  $V' \subseteq V$  且  $|V'| \leq K$ , 多项式时间内逐条检查每条边是否被覆盖。

(b) 3-SAT  $\leq_p$  VC. 给定 3-CNF

$$F = C_1 \wedge C_2 \wedge \dots \wedge C_m, \quad C_j = z_{j1} \vee z_{j2} \vee z_{j3}, \quad z_{jk} \in \{x_i, \neg x_i\}_{i=1}^n,$$

构造 VC 实例  $f(F) = (G, K)$ , 其中  $K = n + 2m$ 。

(b.1) 变元构件 (variable gadget): 对每个变元  $x_i$  引入两个顶点  $x_i, \neg x_i$  并加边

$$E_1 = \{(x_i, \neg x_i) \mid 1 \leq i \leq n\}. \quad (15.2)$$

即  $n$  条互不相邻的 二元边。

(b.2) 析取式构件 (clause gadget): 对每个子句  $C_j$  引入 3 个顶点  $[z_{jk}, j]$  ( $k = 1, 2, 3$ ) 和一个三角形

$$E_2 = \{([z_{j1}, j], [z_{j2}, j]), ([z_{j2}, j], [z_{j3}, j]), ([z_{j3}, j], [z_{j1}, j]) \mid 1 \leq j \leq m\}. \quad (15.3)$$

即  $m$  个三角形  $K_3$ 。

(b.3) 联络边 (literal-to-clause edge): 把每个三角形顶点连到它表示的文字

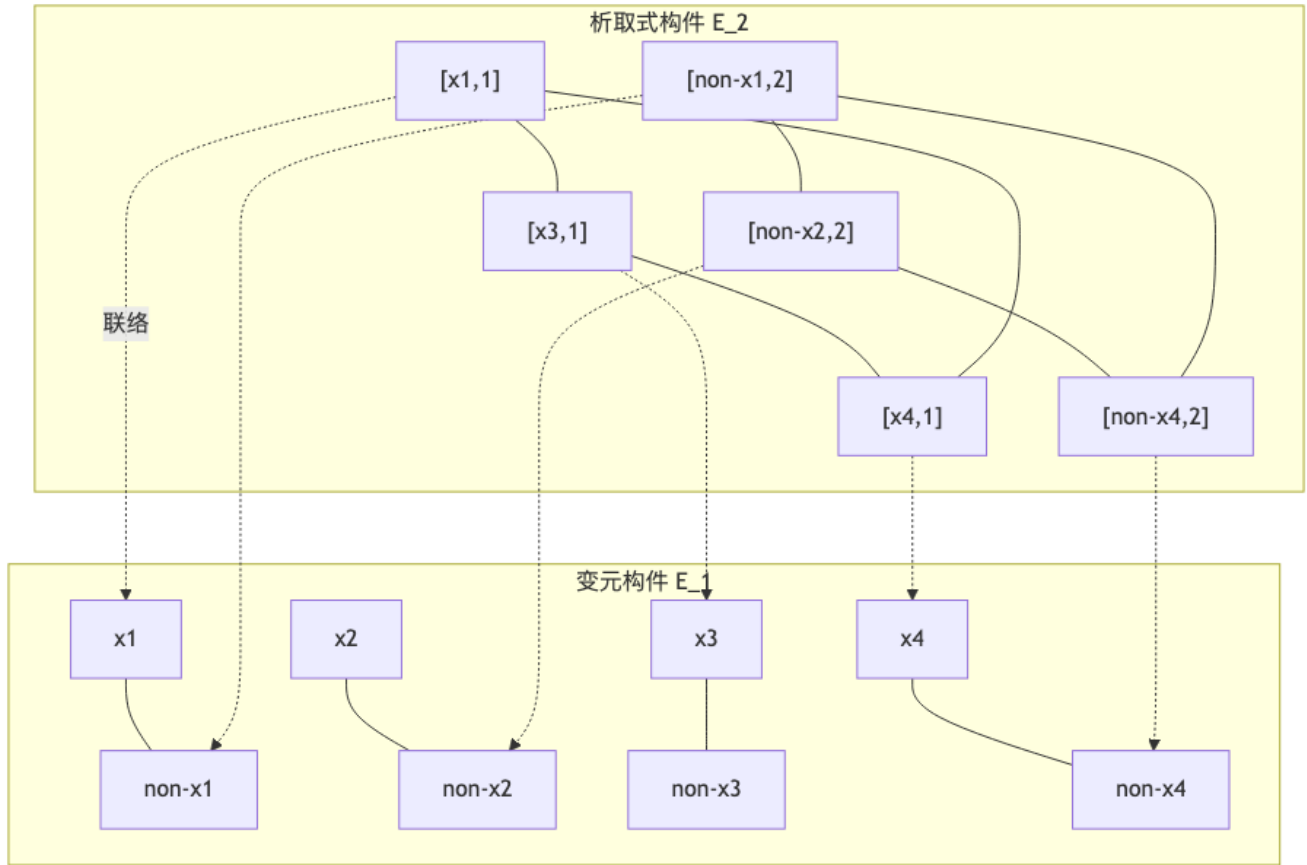
$$E_3 = \{([z_{jk}, j], z_{jk}) \mid k = 1, 2, 3, 1 \leq j \leq m\}. \quad (15.4)$$

总规模:  $|V| = 2n + 3m$ ,  $|E| = n + 6m$ , 可在  $O(n + m)$  时间构造。

[EX] 实例:  $U = \{x_1, x_2, x_3, x_4\}$ ,  $C = (x_1 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_4)$ ,  $K = 4 + 2 \cdot 2 = 8$

变元构件: 4 条互不相邻的边  $(x_i, \neg x_i)$ 。析取式构件: 2 个三角形  $\{[x_1, 1], [x_3, 1], [x_4, 1]\}$  和  $\{[\neg x_1, 2], [\neg x_2, 2], [\neg x_4, 2]\}$ 。联络边: 每个三角形顶点  $\rightarrow$  它表示的文字 (共 6 条虚边)。

图示 (变元构件 / 析取式构件 / 联络边):



## 2.3 等价性证明 (核心步骤)

目标:  $F$  可满足  $\iff G$  含  $\leq K$  顶点的 VC, 且  $K = n + 2m$  恰好等于顶点数下界。

下界: 任何 VC 至少要覆盖  $n$  条边  $E_1$  (每条边需取  $\geq 1$  端点) 和  $m$  个三角形  $E_2$  (每个三角形需取  $\geq 2$  顶点)。

$$|V'| \geq n + 2m = K. \quad (15.5)$$

因此  $|V'| \leq K$  当且仅当 恰好等于  $K$ , 且  $V'$  必须在每条变元边上取 恰好 1 个端点、在每个三角形上取 恰好 2 个顶点。

( $\Rightarrow$ )  $F$  可满足  $\Rightarrow G$  有  $K$  顶点 VC:

设  $t$  为  $F$  的成真赋值。对每个  $i$ , 选  $x_i$  (若  $t(x_i) = 1$ ) 或  $\neg x_i$  (若  $t(x_i) = 0$ ), 共  $n$  个顶点——覆盖  $E_1$ 。

对每个  $C_j$ , 至少存在一个文字  $z_{jk}^*$  使  $t(z_{jk}^*) = 1$ , 相应的联络边  $([z_{jk}^*, j], z_{jk}^*)$  已由  $E_1$  选中的端点覆盖; 从三角形剩下两个顶点  $[z_{jk}, j]$  ( $k \neq k^*$ ) 入选——共  $2m$  个顶点。这  $2m$  个顶点同时覆盖了  $E_2$  的所有三角形边及其余两条联络边。

总计  $n + 2m = K$  顶点。✓

( $\Leftarrow$ )  $G$  有  $K$  顶点 VC  $\Rightarrow F$  可满足:

由下界,  $V'$  在每条变元边上取 1 个端点——这定义了  $t(x_i) = 1$  (若  $x_i \in V'$ ) 或 0 (若  $\neg x_i \in V'$ )。

对每个三角形,  $V'$  取了 2 个顶点, 剩下 1 个顶点  $[z_{jk^*}, j]$  没有被取; 它的联络边  $([z_{jk^*}, j], z_{jk^*})$  必须由  $z_{jk^*} \in V'$  覆盖——即  $t(z_{jk^*}) = 1$ , 所以  $C_j$  真。✓

□

### [WARN] 易错点

1.  $K$  不能小于  $n + 2m$ , 否则下界论证失效; 不能大于  $n + 2m$ , 否则不能强制“每条边/每个三角形恰好取最少”。2. 联络边  $E_3$  不能漏——没有它就无法把变元的赋值与子句的真值挂钩。3. 三角形而非“一条三边线”: 必须  $K_3$ , 因为要保证“取 2 顶点”才能覆盖全部 3 条边。

### [TIP] 推论: IS、CLIQUE 也是 NPC

由 (15.1) 和  $VC \in NPC$  立得 IS、CLIQUE 都是 NPC。这是“问题等价”加速 NPC 蔓延的典型例子——证一个, 赠两个。

## 3. 其他基本 NPC 问题

### [NOTE] 速记表

下面 7 个问题构成 Garey & Johnson 的“基本六大 NPC + ILP”列表, 是后续证明新问题 NPC 的常用归约源。

### 3.1 七大基本 NPC 问题

| 问题                 | 实例                                                                | 询问                                                         | 常用作...的归约源         |
|--------------------|-------------------------------------------------------------------|------------------------------------------------------------|--------------------|
| 有向哈密顿回路 (DHC)      | 有向图 $D$                                                           | $D$ 有哈密顿回路吗?                                               | → HC, TSP          |
| 恰好覆盖 (Exact Cover) | $A = \{a_i\}_{i=1}^n$ ,<br>$W = \{S_j\}_{j=1}^m \subseteq 2^A$    | 存在 $U \subseteq W$ 使 $\bigcup_{S \in U} S = A$ ?           | → 子集和, ILP         |
| 子集和 (Subset Sum)   | $X = \{x_i\}_{i=1}^n \subset \mathbb{Z}^+$ , $N \in \mathbb{Z}^+$ | 存在 $T \subseteq X$ 使 $\sum_{x \in T} x = N$ ?              | → 0-1 背包, 装箱, 双机调度 |
| 装箱 (Bin Packing)   | $n$ 物品权重 $w_j \in \mathbb{Z}^+$ , 箱数 $K$ , 箱容 $B$                 | 能用 $K$ 个箱子装下所有物品 ( $\sum_{j \in \text{箱}_k} w_j \leq B$ )? | —                  |
| 双机调度               | 2 台同构机, $n$ 作业 $J_i$ 时长 $t_i$ , 截止 $D$                            | 能在 $D$ 内完成?                                                | —                  |
| 0-1 背包             | $n$ 物品 $(v_i, w_i)$ , 容量 $W$ , 价值阈值 $V$                           | 选物品使 $\sum w_i \leq W$ 且 $\sum v_i \geq V$ ?               | —                  |
| 整数线性规划 (ILP)       | $A \in \mathbb{Z}^{m \times n}$ , $b \in \mathbb{Z}^m$            | $Ax \leq b, x \geq 0, x \in \mathbb{Z}^n$ 有解?              | —                  |

### [EX] 恰好覆盖小实例

$A = \{1, 2, 3, 4, 5\}$ ,  $S_1 = \{1, 2\}$ ,  $S_2 = \{1, 3, 4\}$ ,  $S_3 = \{2, 4\}$ ,  $S_4 = \{2, 5\}$ .  $\{S_2, S_4\}$  是一个恰好覆盖:  $\{1, 3, 4\} \sqcup \{2, 5\} = A$ , 交为空。若把  $S_4$  改为  $\{3, 5\}$ , 则  $\{S_1 \cup S_4\}$  含 3 两次, 且没有任何  $\{S\}$ -子族能恰好划分  $A$  ——不存在恰好覆盖。

## 3.2 归约方法总结

| 目标           | 归约源   | 手段           |
|--------------|-------|--------------|
| 3-SAT        | SAT   | 局部替换法        |
| 恰好覆盖         | SAT   | 构件设计法        |
| MAX-SAT      | SAT   | 限制法          |
| VC / 团 / 独立集 | 3-SAT | 构件设计法 + 问题等价 |
| 有向 HC        | 3-SAT | 构件设计法        |
| 无向 HC        | 有向 HC | 构件设计法        |
| 子集和          | 恰好覆盖  | 构件设计法        |
| 0-1 背包       | 子集和   | 限制法          |
| 装箱           | 子集和   | 构件设计法        |
| 双机调度         | 子集和   | 构件设计法        |
| ILP          | 恰好覆盖  | 限制法          |

### [TIP] NPC 证明套路

1. 挑一个已知 NPC 的“邻居”——结构上和目标问题相像的源; 2. 三选一: 限制法 (源是目标的特例)、局部替换法 (机械改写)、构件设计法 (设计 gadget); 3. 写两步: (i)  $\Pi \in \text{NP}$ ; (ii) 给出归约  $f$ , 证明  $I \in Y_{\Pi_0} \iff f(I) \in Y_{\Pi}$ , 且  $f$  多项式时间可计算。

## 4. 应用 1: 用 NPC 理论分析子问题谱

### [NOTE] 研究方法论

给一个抽象问题, 先把参数轴画清楚, 再在每个参数格点上判定 P / NPC / 未知——这把“我这个 case 难不难”转化为“在哪条线上分界”。

### 4.1 多处理机调度问题 (有偏序约束)

优化版: 给定任务集  $T$ ,  $m$  台机器,  $\forall t \in T, l(t) \in \mathbb{Z}^+$ ,  $T$  上偏序  $\prec$ 。调度  $\sigma: T \rightarrow \{0, 1, \dots, D\}$  称为可行 当且仅当

- (C1)  $\forall t \in T, \sigma(t) + l(t) \leq D$ ;
- (C2)  $\forall i \in [0, D], |\{t: \sigma(t) \leq i < \sigma(t) + l(t)\}| \leq m$ ; (15.6–15.8)
- (C3)  $\forall t \prec t', \sigma(t) + l(t) \leq \sigma(t')$ .

求最小  $D$  的  $\sigma$ 。

判定版: 给定  $D \in \mathbb{Z}^+$ , 问是否存在  $\sigma$  满足 (C1)–(C3)?

[EX] 6 个任务,  $m=2$

任务长度  $(l_1, \dots, l_6) = (2, 2, 1, 1, 1, 1)$ , 偏序  $\{t_1 \prec t_3, t_2 \prec t_4, t_3 \prec t_5, t_4 \prec t_6\}$ 。调度 1:  $D = 6$  (机器 1:  $t_1, t_3, t_5$ ; 机器 2:  $t_2, t_4, t_6$ )。调度 2 (更优):  $D = 5$ , 通过更紧密的时间槽实现。

## 4.2 三轴参数空间

[NOTE] 子问题谱系图

把参数沿三个轴细分, 得到一张子问题”地图”。各格点的复杂度差别很大——越往右上越难。

$m=1, l$  任意,  $\prec$  任意  $\rightarrow P$

$m$  任意,  $l=1, \prec=\emptyset \rightarrow P$

$m$  任意,  $l=1, \prec$  为树  $\rightarrow P$  (关键路径)

$m=2, l$  为常数,  $\prec$  任意  $\rightarrow P$  (Gabow 1982)

$m \geq 3$  或  $l$  任意 或  $\prec$  任意  $\rightarrow NPC$

## 4.3 几个属于 P 的子问题

[TIP] 算法核心 (列示, 不展开正确性证明)

(a)  $m = 1, l$  任意,  $\prec$  任意: 累加  $\sum l(t)$ , 若  $\leq D$  则按拓扑序顺序排即可; 否则 No。

(b)  $m$  任意,  $l \equiv 1, \prec = \emptyset$ : Time =  $\lceil |T|/m \rceil$ , 直接均分。

(c)  $m$  任意,  $l \equiv 1, \prec$  为树: 关键路径算法: 从叶子开始, 每轮至多取  $m$  个节点向下移; 总轮数 = 树高, 多项式时间。

(d)  $m = 2, l$  为常数,  $\prec$  任意: Gabow (1982) 近似线性时间。

## 4.4 PNP 假设下的复杂度图

[WARN] 重要定理 (Ladner)

当  $P \neq NP$  时, 存在  $NP \setminus (P \cup NPC)$  的问题。换言之,  $NP$  内不是  $NPC$  也不是  $P$  的“中间地带”非空。

研究方法论: 扩大已知区域, 缩小未知区域——一个问题分析完毕, 要么进  $P$  要么进  $NPC$ , 剩下的少量未知问题成为研究焦点 (如 GI 图同构、整数因子分解)。

## 5. 应用 2: 把 NPC 理论推广到搜索 / 优化问题

### 5.1 Turing 归约

[WARN] 定义 15.4 (Turing 归约)

设  $\Pi_1, \Pi_2$  是搜索或优化问题,  $A$  是以解  $\Pi_2$  的假想子程序  $s$  为预言机解  $\Pi_1$  的算法。若: - 只要  $s$  是多项式时间的,  $A$  也是多项式时间的, 则称  $A$  是从  $\Pi_1$  到  $\Pi_2$  的多项式时间 Turing 归约, 记

$$\Pi_1 \propto_T \Pi_2. \quad (15.9)$$

与多项式变换  $\leq_p$  的关系:

- $\leq_p$  是一次性把实例从  $\Pi_1$  翻译到  $\Pi_2$ , 只调用一次  $\Pi_2$  的判定器;
- $\propto_T$  允许多次调用  $\Pi_2$  的预言机, 且预言机可用于非判定问题;

- 多项式变换  $\Rightarrow$  Turing 归约 (反之未必)。

## 5.2 NP-hard / NP-easy / NP-equivalent

### [NOTE] 定义 15.5

设  $\Pi$  是搜索 / 优化问题:

- **NP-hard**: 存在 NPC 问题  $\Pi'$  使  $\Pi' \propto_T \Pi$  ——“至少和 NPC 一样难”。
- **NP-easy**:  $\Pi \propto_T \Pi'$  对某个 NPC 问题  $\Pi'$  ——“在 NPC 预言机帮助下多项式可解”。
- **NP-equivalent (NP-等价)**: 同时 NP-hard 且 NP-easy。

### [TIP] 对应关系

许多优化问题对应的判定问题  $\Pi^{\text{dec}}$  是 NPC, 则其优化版  $\Pi^{\text{opt}}$  通常: - NP-hard: 因为多项式时间解  $\Pi^{\text{opt}}$  自然给出  $\Pi^{\text{dec}}$  的答案; - NP-easy: 用  $\Pi^{\text{dec}}$  的预言机加 **二分搜索 + 重构** 可解  $\Pi^{\text{opt}}$ ; - 故 NP-等价。

## 5.3 例: TSO (货郎优化问题) 是 NP-等价

### [WARN] 定理 15.2

货郎优化问题 (TSO) 是 NP-等价的。

**证明:** TSO 是 NP-hard 显然 (TSP 判定  $\leq_p$  TSO)。下证 TSO 是 NP-easy ——通过 TSE (TSP Extension) 这个中间问题搭桥。

**TSE 实例:** 城市集  $C = \{c_1, \dots, c_m\}$ , 距离  $d : C \times C \rightarrow \mathbb{Z}^+$ , 长度限制  $B \in \mathbb{Z}^+$ , 部分旅行  $\pi = \langle c_{\sigma(1)}, \dots, c_{\sigma(k)} \rangle$  ( $\sigma$  是  $k$ -排列)。

**TSE 询问:** 能否把  $\pi$  延伸为

$$\langle c_{\sigma(1)}, \dots, c_{\sigma(k)}, c_{\sigma(k+1)}, \dots, c_{\sigma(m)} \rangle$$

使全长  $\leq B$ ?

### [NOTE] 为什么用 TSE 不直接用 TSP?

TSE 不仅判定“是否存在”, 还允许我们**指定起点和部分前缀**, 便于在二分搜索后**重构**出一条具体的最优路径。直接用 TSP 只能告诉 Yes/No, 重构需要额外的 self-reduction。

**TSE  $\in$  NP:** 猜剩余排列, 多项式时间累加长度检验。

**TSE 是 NPC:** 当  $\pi = \langle c_1 \rangle$  时退化为 TSP; TSP 是 NPC。

### 5.3.1 Turing 归约 $\text{TSO} \propto_T \text{TSE}$

**Step 1: MinLength** ——二分法求最优长度  $B^*$

设  $d_{\max} = \max\{d(c_i, c_j)\}$ , 则任何巡回长  $\in [m, md_{\max}]$ 。用解 TSE 的预言机  $s$ :

**Algorithm MinLength**

```

B_min  $\leftarrow$  m; B_max  $\leftarrow$  m  $\cdot$  d_max
while B_max - B_min > 1:
    B  $\leftarrow$   $\lfloor (B_{\min} + B_{\max})/2 \rfloor$ 
    if s(C, d,  $\langle c_1 \rangle$ , B) = "Yes":
        B_max  $\leftarrow$  B
    else:
        B_min  $\leftarrow$  B
return B*  $\leftarrow$  B_max

```

调用  $s$  次数  $= O(\log(m d_{\max})) = O(\log m + \log d_{\max})$ 。

### Step 2: FindSolution —— 重构最优路径

#### Algorithm FindSolution

```

 $\pi \leftarrow \langle c_1 \rangle$ ;  $M \leftarrow \{2, 3, \dots, m\}$ ;  $i \leftarrow 2$ 
while  $i \leq m$ :
     $j \leftarrow \min M$ 
     $\pi' \leftarrow \pi ++ \langle c_j \rangle$ 
    if  $s(C, d, \pi', B^*) = \text{"Yes"}$ :
         $\pi \leftarrow \pi'$ ;  $M \leftarrow M \setminus \{j\}$ ;  $i \leftarrow i + 1$ 
    else:
         $j \leftarrow$  next greater element in  $M$  (after the failed one)
         $\pi \leftarrow (\text{previous } \pi) ++ \langle c_j \rangle$     # 回退并尝试更大下标
return  $\pi$ 

```

#### [NOTE] 重构正确性

在已找到最优长度  $B^*$  后, 从前缀  $\langle c_1 \rangle$  出发, 贪心选最小下标  $j$  加入; 若  $s$  回答 No, 说明这个  $j$  不能延伸至  $B^*$ , 换下一个下标。每一步必有一个可行下标 (否则  $B^*$  不可达, 矛盾)。

### 5.3.2 调用次数与多项式性

#### [WARN] 复杂度证明 (核心式子)

第  $i$  步至多  $m - i$  次失败尝试 + 1 次成功, 调用  $s$  共

$$\sum_{i=2}^m (m - i + 1) = (m - 1) + (m - 2) + \dots + 1 = \frac{(m - 1)m}{2} = O(m^2). \quad (15.10)$$

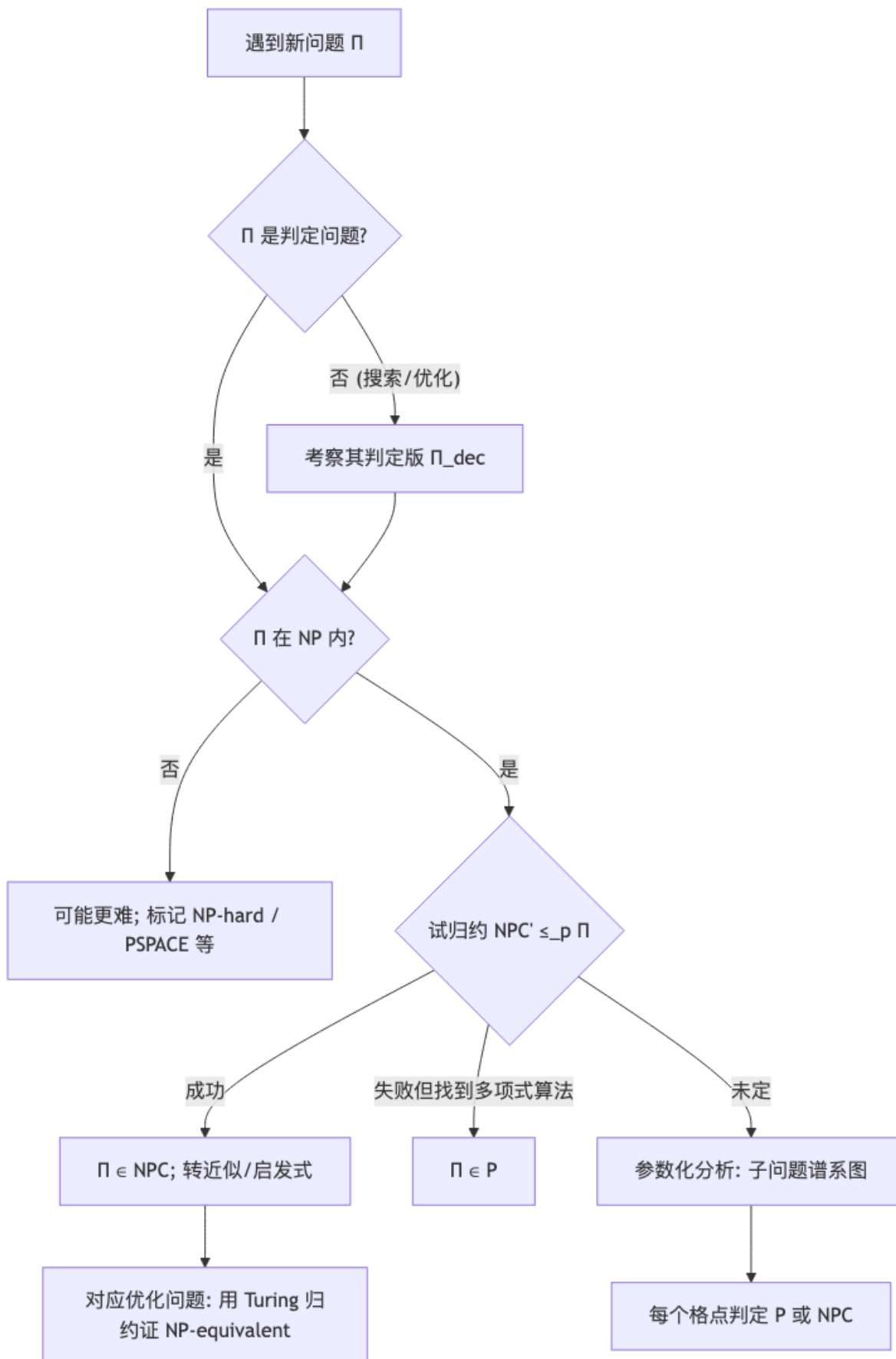
加上 MinLength 的  $O(\log m + \log d_{\max})$  次, 总调用数  $O(m^2 + \log d_{\max})$ , 为 TSO 实例规模 ( $O(m^2 + m \log d_{\max})$ ) 的多项式。

□

#### [TIP] 推广

同样的 Turing 归约模板可证: 基本六大 NPC 对应的优化问题 (TSO、最大独立集、最大团、最小 VC、最大 SAT 等) 全部是 NP-等价的。

## 6. 一图总结：NPC 理论的”研究操作手册”



## 7. 记号速查

| 符号 / 术语       | 含义                                                                    |
|---------------|-----------------------------------------------------------------------|
| $\leq_p$      | 多项式时间多对一归约 (Karp 归约)                                                  |
| $\propto_T$   | 多项式时间 Turing 归约 (Cook 归约)                                             |
| NP-hard       | 至少和某 NPC 一样难<br>( $\exists \Pi' \in \text{NPC}, \Pi' \propto_T \Pi$ ) |
| NP-easy       | 在 NPC 预言机下多项式可解<br>( $\Pi \propto_T \text{某 NPC}$ )                   |
| NP-equivalent | NP-hard $\wedge$ NP-easy                                              |
| 变元构件          | 强制布尔赋值二选一的图结构 (边 $(x_i, \neg x_i)$ )                                  |
| 析取式构件         | 强制每个子句至少一文字真的图结构 (三角形)                                                |
| 联络边           | 把变元构件和析取式构件挂钩                                                         |
| 限制法           | $\Pi_0$ 是 $\Pi$ 加约束/特例<br>$\Rightarrow \Pi_0 \leq_p \Pi$              |
| 局部替换法         | 机械规则改写实例 (如 $k$ -CNF $\rightarrow$ 3-CNF)                             |
| 构件设计法         | 设计 gadget 模拟原问题语义                                                     |

## 8. 期末复习要点

### [TIP] 必背 3 件套

1. VC / IS / CLIQUE 三者等价 + VC NPC 证明 (3-SAT 归约,  $K = n + 2m$  计数论证)。2. NPC 证明套路: 选源 + 三选一 (限制 / 局部替换 / 构件) + 双向证明。3. 优化问题 NPC-等价: Turing 归约 + 二分法求最优值 + 贪心重构具体解。

### [WARN] 易混淆

-  $\leq_p$  vs  $\propto_T$ : 前者只调用 1 次预言机 (判定问题专用); 后者可调用多次 (搜索/优化用)。- NP-hard 不等于 NPC: NP-hard 不要求  $\in \text{NP}$  (例: 停机问题是 NP-hard 但不在 NP);  $\text{NPC} = \text{NP-hard} \cap \text{NP}$ 。- “ $P \neq \text{NP}$ ” 假设下 NP 中存在非 P 非 NPC 的中间问题 (Ladner 定理)。

## 9. Anki 卡片 (建议导出)

| 正面             | 背面                                                                         |
|----------------|----------------------------------------------------------------------------|
| 顶点覆盖 (VC) 的定义? | $G = \langle V, E \rangle, V' \subseteq V$ 是 VC $\iff$ 每条边至少有一端 $\in V'$ 。 |

| 正面                           | 背面                                                                     |
|------------------------------|------------------------------------------------------------------------|
| VC、IS、CLIQUE 三者关系?           | $V'$ 是 VC $\iff V - V'$ 是 IS $\iff V - V'$ 是补图 $G^c$ 的 CLIQUE。         |
| 3-SAT $\leq_p$ VC 的 $K$ 取多少? | $K = n + 2m$ ( $n$ 变元数, $m$ 子句数)。                                      |
| NPC 证明的三大归约手段?               | 限制法、局部替换法、构件设计法。                                                       |
| Turing 归约和多项式变换 $\leq_p$ 区别? | $\leq_p$ 一次性翻译实例 + 判定; Turing 可调用预言机多次 + 用于搜索/优化。                      |
| NP-hard 的定义?                 | $\exists$ NPC 问题 $\Pi'$ 使 $\Pi' \propto_T \Pi$ 。                       |
| NP-equivalent 的两个条件?         | NP-hard $\wedge$ NP-easy。                                              |
| TSP 是 NP-equivalent 的证明思路?   | 引入 TSP, 用 $s$ 预言机二分求 $B^*$ , 再贪心重构最优巡回; 调用 $O(m^2 + \log d_{\max})$ 次。 |
| Ladner 定理 (条件性结论)?           | 若 $P \neq NP$ , 则 $NP \setminus (P \cup NPC) \neq \emptyset$ 。         |
| 7 大基本 NPC 问题?                | DHC、恰好覆盖、子集和、装箱、双机调度、0-1 背包、ILP (加上 3-SAT / VC / IS / CLIQUE / HC)。    |

# 期中复习 (Midterm Review)

## [TIP] 期中安排

– 考试时间: 第 9 周周三下午 (2 小时, 闭卷, 凭学生证) – 覆盖范围: Lec 1–8 (数学基础 → 平摊分析) – 题型: 阶排序 / 递推方程 / 算法分析 / 算法设计 (DP/贪心/分治/回溯/LP)

## 0. 考试范围

| 模块     | 重点                              |
|--------|---------------------------------|
| 数学基础   | 估计函数的阶、递推方程求解、求和技术              |
| 算法设计技术 | 分治、动态规划、贪心、回溯/分支限界、线性规划         |
| 算法分析技术 | 估计简单伪码的基本运算次数 (最坏/平均)、简单问题复杂度估计 |
| 平摊分析   | 三种方法                            |

## 1. 数学基础

### 1.1 阶 (Order) 的概念

- $f(n) = O(g(n))$ :  $\exists c, n_0, \forall n \geq n_0 : 0 \leq f \leq c g$
- $f(n) = \Theta(g(n))$ :  $f = O(g) \wedge g = O(f)$
- $f(n) = o(g(n))$ :  $\forall c, \exists n_0 : f < c g$

[WARN] 阶反映  $n \rightarrow \infty$  的大趋势, 允许忽略有限项. 但要 连续趋势, 不允许在不同阶之间抖动.

### 1.2 阶的高低

$$1 \prec \log \log n \prec \log n \prec (\log n)^k \prec n^{1/k} \prec n \prec n \log n \prec n^c \prec 2^n \prec 3^n \prec n! \prec n^n.$$

### 1.3 递推方程求解

- 迭代归纳法 (求和): 直接展开 + 累加;
- 递归树: 每层工作量求和;
- 主定理:  $T(n) = aT(n/b) + f(n)$ .

## 1.4 常用求和

| 求和                         | 结果                 |
|----------------------------|--------------------|
| $\sum_{k=1}^n k$           | $n(n+1)/2$         |
| $\sum_{k=0}^{n-1} aq^k$    | $a(1-q^n)/(1-q)$   |
| $\sum_{k=0}^{\infty} aq^k$ | $a/q$              |
| $H_n = \sum_{k=1}^n 1/k$   | $\Theta(\log n)$   |
| $\sum_{k=1}^n \log k$      | $\Theta(n \log n)$ |
| $\sum_k k \cdot 2^k$       | 差消法                |

## 1.5 主定理

$T(n) = aT(n/b) + f(n)$ , 设  $c^* = \log_b a$ :

1.  $f = O(n^{c^*-\epsilon}) \Rightarrow T = \Theta(n^{c^*})$
2.  $f = \Theta(n^{c^*}) \Rightarrow T = \Theta(n^{c^*} \log n)$
3.  $f = \Omega(n^{c^*+\epsilon}) \Rightarrow T = \Theta(f(n))$

扩展:  $f = \Theta(n^{c^*} \log^k n) \Rightarrow T = \Theta(n^{c^*} \log^{k+1} n)$ .

[WARN] 情形 3 必须验证正则性  $a f(n/b) \leq c f(n), c < 1$ . 情形之间存在“ $n^{\epsilon}$  gap,  $\log$  因子”的差距不属情形 1/3.

| 递推                             | 解                      |
|--------------------------------|------------------------|
| $T(n) = T(n/2) + 1$            | $\Theta(\log n)$       |
| $T(n) = 2T(n/2) + n$           | $\Theta(n \log n)$     |
| $T(n) = 3T(n/2) + n$           | $\Theta(n^{\log_2 3})$ |
| $T(n) = 7T(n/2) + n^2$         | $\Theta(n^{\log_2 7})$ |
| $T(n) = T(n-1) + n$            | $\Theta(n^2)$          |
| $T(n) = T(n/5) + T(7n/10) + n$ | $\Theta(n)$            |

## 2. 分治策略

### 2.1 适用条件

- 子问题与原问题 **同型**;
- 子问题 **独立** (无重叠).

### 2.2 设计步骤

1. 划分 (Divide);
2. 递归求解 (Conquer);
3. 合并 (Combine);
4. 写递推方程并求解.

## 2.3 经典问题

| 问题        | 递推                                | 解                   |
|-----------|-----------------------------------|---------------------|
| 二分检索      | $T(n) = T(n/2) + 1$               | $\Theta(\log n)$    |
| 归并排序      | $T(n) = 2T(n/2) + n$              | $\Theta(n \log n)$  |
| Karatsuba | $T(n) = 3T(n/2) + n$              | $\Theta(n^{1.585})$ |
| Strassen  | $T(n) = 7T(n/2) + n^2$            | $\Theta(n^{2.807})$ |
| 最近点对      | $T(n) = 2T(n/2) + n$              | $\Theta(n \log n)$  |
| Select    | $W(n) \leq W(n/5) + W(7n/10) + n$ | $\Theta(n)$         |

### [WARN] 分治改进途径

1. 代数变换: 减少子问题数 (Karatsuba 4→3, Strassen 8→7). 2. 预处理: 把常做的操作提到递归外 (最近点对预排序).

## 3. 动态规划

### 3.1 适用条件

- 优化问题、多阶段决策;
- 优化原则 (Bellman): 最优解的子序列也是子问题的最优;
- 子问题重叠.

### 3.2 设计步骤

1. 状态 + 边界;
2. 决策 / 优化函数;
3. 递推方程;
4. 自底向上填表;
5. 标记函数构造解.

### 3.3 经典问题

| 问题     | 状态                 | 时间                                  |
|--------|--------------------|-------------------------------------|
| 矩阵链    | $m[i, j]$ = 最少乘法   | $\Theta(n^3)$                       |
| LCS    | $C[i, j]$ = LCS 长度 | $\Theta(mn)$                        |
| 投资     | $F_k(x)$           | $\Theta(nm^2)$                      |
| 0/1 背包 | $V[i, w]$          | $\Theta(nW)$ 伪多项式                   |
| 最大子段和  | $f[i]$ Kadane      | $\Theta(n)$                         |
| OBST   | $e[i, j]$          | $\Theta(n^3)$ / Knuth $\Theta(n^2)$ |
| 编辑距离   | $d[i, j]$          | $\Theta(mn)$                        |

### 3.4 矩阵链 (递归 vs DP)

递归 (无备忘录):  $T(n) = \Omega(2^n)$ . DP (自底向上):  $\Theta(n^3)$ .

## 4. 贪心算法

### 4.1 适用条件

- 组合优化、多步判断;
- 贪心选择性质: 存在最优解, 第一步与算法相同;
- 优化子结构: 选完第一步后的剩余子问题也是优化问题.

### 4.2 正确性证明

- 数学归纳法: 对算法步数或问题规模.
- 交换论证 (Exchange Argument): 从任意最优解出发, 替换为贪心解, 目标不变差.

### 4.3 经典问题

| 问题          | 贪心策略             | 复杂度                      |
|-------------|------------------|--------------------------|
| 活动选择        | 按 $f_i$ 升序       | $O(n \log n)$            |
| 最小延迟调度      | EDF (按 $d_i$ 升序) | $O(n \log n)$            |
| Huffman 编码  | 合并两最小频率          | $O(n \log n)$            |
| Kruskal MST | 按权升序 + 并查集       | $O(m \log n)$            |
| Prim MST    | 顶点贪心 + 堆         | $O(m \log n)$            |
| Dijkstra    | 按距离贪心            | $O(n^2)$ 或 $O(m \log n)$ |
| 短作业优先       | 按 $t_i$ 升序       | $O(n \log n)$            |
| 分数背包        | 按 $v_i/w_i$ 降序   | $O(n \log n)$            |

## 5. 回溯 / 分支限界

### 5.1 适用条件

- 搜索问题;
- 多步判断;
- 多米诺性质: 前缀可行 = 部分解可扩展的必要条件.

### 5.2 设计要素

1. 解向量  $\langle x_1, \dots, x_n \rangle$ ;
2. 分支条件 (约束);
3. 代价函数 (优化问题);
4. 搜索策略 (DFS/BFS/优先函数);
5. 估计搜索树规模 (Monte Carlo).

### 5.3 经典问题

| 问题     | 搜索树     | 叶子数       |
|--------|---------|-----------|
| n 皇后   | $n$ -叉树 | $\leq n!$ |
| 0/1 背包 | 子集树     | $2^n$     |
| TSP    | 排列树     | $(n-1)!$  |
| 最大团    | 子集树     | $2^n$     |

| 问题  | 搜索树 | 叶子数  |
|-----|-----|------|
| 圆排列 | 排列树 | $n!$ |

## 5.4 分支限界

DFS + 代价函数下/上界, 维护活节点表 (常用优先队列, best-first).

## 6. 线性规划

### 6.1 标准形

$$\min c^T x, Ax = b, b \geq 0, x \geq 0.$$

化标准形:

- $\max \Rightarrow \min(-z)$ ;
- $\leq$  + 松弛  $y \geq 0$ ;
- $\geq$  - 剩余  $y \geq 0$ ;
- 自由 拆成  $x' - x''$ , 都  $\geq 0$ .

### 6.2 单纯形法

- 找 BFS;
- 计算检验数  $\bar{c}_j = c_j - c_B^T B^{-1} A_j$ ;
- 若  $\bar{c} \geq 0$ : 最优.
- 否则: 取  $\bar{c}_j < 0$  入基, 最小比率检验确定出基;
- 重复.

### 6.3 对偶定理

| 原始 max      | 对偶 min         |
|-------------|----------------|
| $Ax \leq b$ | $A^T y \geq c$ |
| $x \geq 0$  | $y \geq 0$     |

弱对偶  $c^T x \leq b^T y$ . 强对偶: 最优时等号.

### 6.4 ILP 分支限界

LP 松弛  $\rightarrow$  分支取整  $\rightarrow$  限界剪枝.

## 7. 平摊分析

### 7.1 三种方法

1. 聚集:  $\sum c_i / n$ ;
2. 记账法:  $\hat{c}_i$  多收的钱存数据结构, 不变量  $\geq 0$ ;
3. 势能法:  $\hat{c}_i = c_i + \Delta\Phi$ ,  $\sum \hat{c}_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$ .

## 7.2 经典数据结构

| 数据结构                       | 操作           | 平摊             |
|----------------------------|--------------|----------------|
| 栈 PUSH/POP/MULTIPOP        | 任意           | $O(1)$         |
| 二进制计数器 INCREMENT           | 反转           | $O(1)$         |
| 动态表 INSERT (倍增)            | 任意           | $O(1)$         |
| 动态表 INSERT/DELETE (1/4 收缩) | 任意           | $O(1)$         |
| 斐波那契堆                      | DECREASE-KEY | $O(1)$         |
| 并查集                        | UNION/FIND   | $O(\alpha(n))$ |

## 8. 解题模板

### 8.1 分治

1. 描述问题
2. 算法思想（一段话）
3. 划分子问题方式
4. 递归结束条件
5. 最坏情况递推方程 + 初值
6. 求解递推（主定理 / 迭代 / 递归树）
7. 复杂度

### 8.2 动态规划

1. 子问题边界（状态）
2. 优化函数定义
3. 递推方程 + 边界条件
4. 标记函数（用于追踪解）
5. 自底向上填表
6. 复杂度（时间 / 空间）

### 8.3 贪心

1. 描述贪心策略
2. 证明贪心选择性质（归纳 / 交换）
3. 证明优化子结构
4. 算法步骤
5. 复杂度

### 8.4 回溯

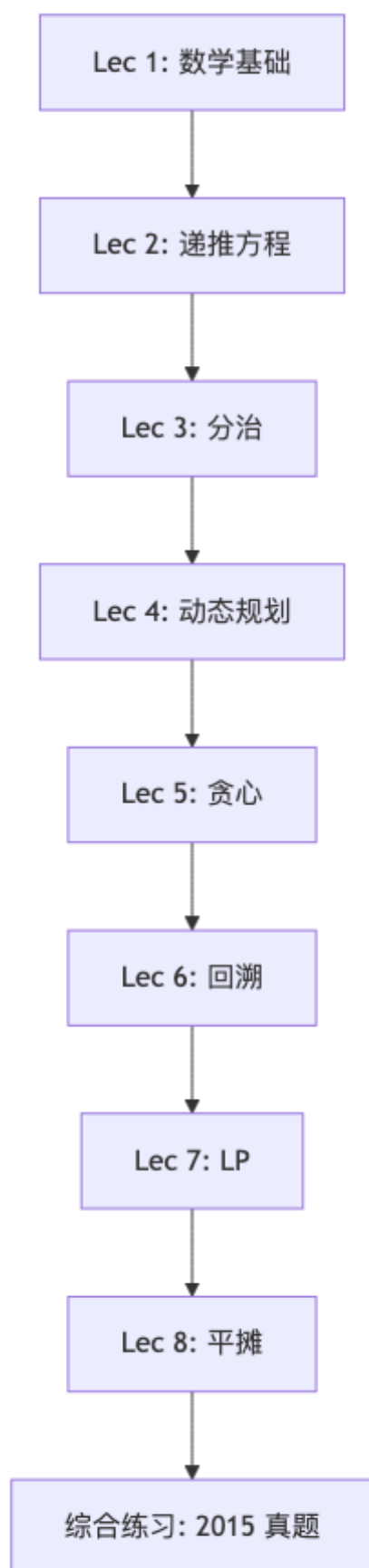
1. 解向量
2. 搜索树（子集 / 排列 / n-叉）
3. 多米诺性质（必要）
4. 约束条件
5. 代价函数（优化问题）
6. 算法步骤
7. 复杂度（最坏）

## 9. 常见易错点综合

### [WARN] 期中重点警示

1. **递推方程**: 边界  $T(1)$  必须写, 主定理三情形 + 扩展. 2. **DP 与贪心区分**: 贪心每步一个最优子问题; DP 多子问题取最优. 3. **正确性证明**: 贪心必证, DP 证优化原则. 4. **复杂度**: 时间 + 空间 都写. 5. **回溯**: 解向量 + 搜索树规模 + 多米诺性质. 6. **LP**: 标准形 + 单纯形 pivot 写清楚, 对偶方向. 7. **平摊**: 三种方法选一即可, 但势函数要合理.

## 10. 复习路线建议



# 2015 年算分期中试题—完整解答与复盘

## [TIP] 试题信息

– 考试时间: 2015 年 4 月 27 日 – 总分: 100 分, 共 7 题 (15/10/15/15/15/15/15) – 答题要求: 算法题先用一段话描述算法思想; DP 写递推方程/边界/标记函数; 贪心给证明; 回溯给解向量/搜索树/约束.

## 第一题 (15 分) 一阶排序

题目: 按阶递减顺序排列下列函数:

$n^{\log_2 1}$ ,  $\log_3 n$ ,  $\log_2^2 n$ ,  $2^n$ ,  $(\log n)^{\log \log n}$ ,  $n^2$ ,  $2^{\log^* n}$ ,  $\log(n!)$ ,  $\log(\log n)$ ,  $3^n$ ,  $n!$ ,  $n^{1/\log_2 n}$ ,  $n$ ,  $10 \log n$ .

### 思路

按 §1.2 的层次表逐一比较:

- $n! \prec 3^n \prec 2^n$  — 从 Stirling:  $n! \approx (n/e)^n$ , 比  $3^n$  更大.
- $n^2 \prec 2^n$ .
- $(\log n)^{\log \log n} = e^{(\log \log n)^2}$  增长比 polylog 快, 但比  $n^\varepsilon$  慢.
- $\log(n!) = \Theta(n \log n)$ .
- $n^{\log_2 1} = n^0 = 1$ .
- $n^{1/\log_2 n} = 2$  (恒等于 2).
- $\log_3 n = \Theta(\log n) = \Theta(10 \log n)$ .
- $2^{\log^* n}$  几乎是常数.
- $\log \log n$  增长极慢.

### 标准答案

由大到小:

$n! \succ 3^n \succ 2^n \succ n^2 \succ \log(n!) = \Theta(n \log n) \succ (\log n)^{\log \log n} \succ \log_2^2 n \succ \log_3 n = \Theta(10 \log n) \succ \log \log n$

(题目中具体函数与提供的图片对应可能略异, 但思路是按阶比较.)

## 复盘

### [WARN] 易错点

1.  $n^{\log_2 1} = n^0 = 1$  一别误读为  $n^{\log_2 n}$ . 2.  $n^{1/\log_2 n}$  是常数 2 一因  $\log_2(n^{1/\log_2 n}) = 1$ . 3.  $\log(n!) = \Theta(n \log n)$ , 与  $\log_2^2 n$  阶差很多. 4.  $\log^* n$  是迭代对数, 极慢.

## 第二题 (10 分) 一递推方程

题目: 求解递推方程

$$T(n) = 4T(n/2) + n^2 \log^k n, \quad T(1) = 1.$$

其中  $k$  是给定正整数.

### 思路

应用主定理:

- $a = 4, b = 2, c^* = \log_2 4 = 2$ .
- $f(n) = n^2 \log^k n = \Theta(n^{c^*} \log^k n)$ .

不属于主定理 3 个标准情形 (gap 是  $\log^k$  而非  $n^\varepsilon$ ), 但属于 **扩展情形**:

$$f(n) = \Theta(n^{c^*} \log^k n) \Rightarrow T(n) = \Theta(n^{c^*} \log^{k+1} n).$$

### 标准答案

$$T(n) = \Theta(n^2 \log^{k+1} n).$$

### 推导 (备用一递归树法)

每层工作量:

- 第 0 层:  $n^2 \log^k n$ ;
- 第 1 层:  $4 \cdot (n/2)^2 \log^k(n/2) = n^2 \log^k(n/2) = n^2(\log n - 1)^k$ ;
- 第  $i$  层:  $4^i \cdot (n/2^i)^2 \log^k(n/2^i) = n^2(\log n - i)^k$ .

总:

$$T(n) = \sum_{i=0}^{\log n} n^2(\log n - i)^k = n^2 \sum_{j=0}^{\log n} j^k = n^2 \cdot \Theta(\log^{k+1} n) = \Theta(n^2 \log^{k+1} n).$$

利用  $\sum_{j=1}^M j^k = \Theta(M^{k+1})$ .

## 复盘

### [WARN] 易错点

1. 主定理标准 3 情形不覆盖此题—必须用 **扩展情形** (Bentham 推广). 2. 递归树推导时, 注意  $\log(n/2^i) = \log n - i$  (整数  $i$ ). 3.  $T(1) = 1$  边界不能漏写, 但对阶不影响.

### 第三题 (15 分) — 递归算法分析

题目: 算法 XYZ:

```
XYZ(A[1..n]):  
    if n == 1: return A[1]  
    temp = XYZ(A[1..n-1])  
    if temp < A[n]: return temp  
    else: return A[n]
```

1. 算法 XYZ 输出什么?
2. 列出最坏情况  $W(n)$  的递推方程, 解出  $W(n)$ .
3. XYZ 最坏情况下是否是最优算法?

#### 解答

(1) 输出:  $A$  中 **最小** 实数. 因为递归从  $A[1..n-1]$  取得最小 temp, 然后与  $A[n]$  比较, 较小者返回.

[WARN] 容易看反

算法看似返回“max”, 因为 if temp <  $A[n]$  时返回 temp. 但仔细看: temp 较小时返回 temp (较小),  $A[n]$  较小时 (走 else) 返回  $A[n]$  (较小). 故返回 **最小**.

(2) 递推方程:

```
W(n) = W(n-1) + 1    # 每次 1 次比较  
W(1) = 0
```

迭代解:  $W(n) = n - 1$ .

(3) 最优性: 是最优. 因找最小问题至少需  $n - 1$  次比较 (输入  $n$  个数, 必须把  $n - 1$  个数都“淘汰”, 即每个非最小数必至少参与一次比较输给某更小数).

#### 复盘

[WARN] 易错点

1. 看清 if temp <  $A[n]$  then return temp 一是返回 **较小者** (temp 较小). 2. 最优性证明: 给出下界  $n - 1$  即可, 不需更复杂论证.

### 第四题 (15 分) — 近似中值

题目:  $A$  是  $n$  个数的序列. 如果  $x \in A$  满足: 小于  $x$  的数  $\geq n/4$ , 大于  $x$  的数  $\geq n/4$ , 则称  $x$  为  $A$  的 **近似中值**. 设计算法求近似中值, 给出时间复杂度.

#### 思路

算法 1 (基于 Select):

1. 用 Select 求  $a =$  第  $\lceil n/4 \rceil$  小的数,  $b =$  第  $\lceil 3n/4 \rceil + 1$  小的数
2. if  $a == b$ : return " 无解"
3. 用  $a, b$  划分  $A$ :  
     $A_1 = \{x : x < a\}$   
     $A_2 = \{x : x == a\}$   
     $A_3 = \{x : a < x < b\}$

```

    A_4 = {x : x == b}
    A_5 = {x : x > b}
4. if A_3 ≠ ∅: 返回 A_3 中任一元素
   else: return " 无解"

```

**正确性:** 若  $x \in A_3$ , 则  $x > a \geq$  “前  $\lceil n/4 \rceil$  小”, 故  $\geq n/4$  个数  $< x$ . 同理  $\geq n/4$  个数  $> x$ .

**时间复杂度:** Select 是  $O(n)$ , 划分  $O(n)$ . 总  $O(n)$ .

## 标准答案

时间  $O(n)$ . 算法步骤如上.

## 复盘

### [WARN] 易错点

1. 不能直接用排序—时间  $\Theta(n \log n)$ , 不达到  $O(n)$ . 2. 当  $a = b$  时确实可能无解 (例如  $n/2$  个数相等, 都恰好是分位点). 3. 必须用 BFPRT-Select 而非平均  $O(n)$  的快速选择, 才能严格  $O(n)$ .

## 第五题 (15 分) —文件副本放置 (DP)

**题目:** 服务器  $S = \{s_1, \dots, s_n\}$ , 副本放在  $s_i$  有存储代价  $c_i$ . 访问  $s_i$  时若有副本 0 代价, 否则顺序找  $s_{i+1}, \dots, s_j$  (第一个有副本的服务器), 访问代价  $j - i$ . 规定必须在  $s_n$  放副本. 每个  $s_i$  都发生 1 次访问. 求最小总代价.

## 思路

DP. 设  $\text{OPT}[j]$  = 副本放在  $s_j$ , 从  $s_1$  到  $s_j$  的最小代价.

设  $s_j$  前最近放副本的位置是  $s_i$ , 那么  $s_{i+1}, \dots, s_{j-1}$  访问代价之和为

$$\sum_{k=i+1}^{j-1} (j - k) = (j - i - 1) + (j - i - 2) + \dots + 1 = \frac{(j - i - 1)(j - i)}{2}.$$

记  $C(i + 1, j) = \frac{(j - i - 1)(j - i)}{2}$ . 递推:

$$\text{OPT}[j] = c_j + \min_{0 \leq i < j} \{ \text{OPT}[i] + C(i + 1, j) \}, \quad (*)$$

$$\text{OPT}[0] = 0.$$

**最优解:**  $\text{OPT}[n]$ .

**正确性:** 优化原则: 最优放置方案中, 若最后放副本的位置是  $s_n$ , 那么必有某个  $s_i$  是  $s_n$  前最近放副本的服务器; 由结构对称性,  $\text{OPT}[i] + C(i + 1, n) + c_n$  即从  $s_1$  到  $s_i$  子问题最优 +  $s_{i+1} \dots s_{n-1}$  访问代价 +  $c_n$ .

## 伪代码

```
PLACEMENT(c[1..n]):
    OPT[0] = 0
    for j = 1..n:
        OPT[j] = ∞
        for i = 0..j-1:
            val = OPT[i] + (j-i-1)(j-i)/2 + c[j]
            if val < OPT[j]:
                OPT[j] = val
                s[j] = i
    return OPT[n], s
```

# 标记函数

## 复杂度

- 时间:  $\Theta(n^2)$  (两层循环).
- 空间:  $\Theta(n)$ .

## 复盘

### [WARN] 易错点

1. 第一个副本可能放在  $s_1$  (此时  $\text{OPT}[1] = c_1$ , 没有访问代价). 2.  $C(i+1, j)$  是  $s_{i+1}$  到  $s_{j-1}$  的访问代价 (因  $s_j$  上访问无代价已计入). 3. 边界  $\text{OPT}[0]$ : 把哑节点  $s_0$  看作“前面无副本”状态, 然后  $\text{OPT}[i] + C(i+1, j)$  中  $i = 0$  表示前面没放; 此时  $s_1, \dots, s_{j-1}$  都查找找到  $s_j$ , 访问代价  $\sum_{k=1}^{j-1} (j-k)$  同样满足公式.

## 第六题 (15 分) — 软件许可证购买 (贪心)

题目: 公司买  $n$  个软件许可证, 每月最多 1 个. 第  $i$  个许可当前售价 1000 元, 但每月按  $r_i > 1$  因子指数增长 (第  $k$  个月售价  $r_i^k \cdot 1000$ ).  $r_i$  是给定正整数. 求购买顺序, 使总花费最少.

### 思路

贪心策略: 按  $r_i$  递减 排序, 依次购买. 即  $r_i$  增长率大的先买.

直观: 越往后买, 价格涨得越多. 故让涨得快的早买.

### 正确性证明 (交换论证)

设最优解中存在相邻 逆序:  $j$  在第  $t$  月买,  $i$  在第  $t+1$  月买, 但  $r_j < r_i$ .

交换  $i, j$  (即  $i$  在第  $t$  月,  $j$  在第  $t+1$  月), 新解  $S$ . 比较花费:

$$V(\text{OPT}) - V(S) = (r_i^{t+1} \cdot 1000 + r_j^t \cdot 1000) - (r_i^t \cdot 1000 + r_j^{t+1} \cdot 1000) = 1000 \cdot [r_i^t(r_i - 1) - r_j^t(r_j - 1)].$$

由  $r_i \geq r_j + 1 > 1$  且  $r_i^t \geq r_j^t$ ,  $r_i - 1 \geq r_j - 1$ , 故  $r_i^t(r_i - 1) \geq r_j^t(r_j - 1)$ , 即  $V(\text{OPT}) \geq V(S)$ . 交换后总花费不增. 经过有限次交换后得到按  $r$  递减的解.

## 算法

```
LICENSE-BUY(r[1..n]):  
  按 r 降序排序得  $r_{\{\sigma(1)\}} \geq r_{\{\sigma(2)\}} \geq \dots \geq r_{\{\sigma(n)\}}$   
  第 k 月购买软件  $\sigma(k)$ , 花费  $r_{\{\sigma(k)\}}^k * 1000$   
  return  $\sigma$ 
```

## 复杂度

排序  $O(n \log n)$ , 计算总花费  $O(n)$ . 总  $O(n \log n)$ .

## 复盘

### [WARN] 易错点

1. 排序方向: 降序, 不是升序. 2. 交换论证关键不等式:  $r_i^t(r_i - 1) \geq r_j^t(r_j - 1)$ . 3. 直觉判断: 越往后越贵, 让“涨价快的早买”; 正好是  $r_i$  大的先买.

## 第七题 (15 分) 一会议室无线路由器布置 (回溯)

题目:  $m \times n$  会议室阵列, 每间边长 10 米, 路由器只能放会议室中心, 覆盖半径 25 米. 求最少路由器使所有会议室中心有信号.

## 思路

搜索空间:  $mn$  维 0/1 向量  $\langle x_{11}, \dots, x_{mn} \rangle$ ,  $x_{ij} = 1$  表示放. 搜索树  $mn$  层.

优化目标: 最小化  $\sum x_{ij}$ .

约束: 每个会议室至少被某个  $x_{ij} = 1$  的路由器覆盖. 路由器  $(i, j)$  覆盖以它为中心、半径  $25/10 = 2.5$  个会议室 (欧氏距离), 即坐标  $(i', j')$  满足  $(i' - i)^2 + (j' - j)^2 \leq 6.25$ .

## 算法

初始: 令所有  $x_{ij} = 1$ , 共  $mn$  个路由器. 搜索: - 从  $(1, 1)$  开始, 左子树 = 拿走  $(1, 1)$ , 右子树 = 保留. - 进入左子树时检查是否所有会议室都被覆盖; 若有未覆盖则停止该分支.

```
BACKTRACK-ROUTERS(k):  
  if k > m*n: 记录当前 sum(x) 与最优解比较  
  else:  
    # 左子树: 尝试不放 (i, j)  
    if 拿走 x[i][j] 后所有会议室仍被覆盖:  
      x[i][j] = 0; BACKTRACK-ROUTERS(k+1); x[i][j] = 1  
    # 右子树: 保留 (i, j)  
    BACKTRACK-ROUTERS(k+1)
```

## 复杂度

最坏  $O(2^{mn})$ . 实际剪枝后远低.

## 多米诺性质

“拿走  $(i, j)$  后所有会议室仍被覆盖”是约束. 若部分解  $\{x_{i_1 j_1} = 0, \dots, x_{i_k j_k} = 0\}$  已使某会议室无覆盖, 则任何扩展都不能恢复覆盖 (因只能继续减少 0/1 中的 1, 而不能再加回). 故 **多米诺成立** (前缀可行 可扩展).

## 检查复杂度

每个会议室  $(i, j)$  被覆盖与否, 只与其周围常数个邻居有关 (半径 2.5, 即  $\sqrt{6.25} \leq 2.5$ , 邻居坐标差  $\leq 2$ ). 故检查覆盖性 = 常数时间; 总检查 =  $O(mn)$ .

最坏总:  $O(2^{mn} \cdot mn)$ , 通常简化为  $O(2^{mn})$ .

## 复盘

### [WARN] 易错点

1. 路由器只放在中心—距离比较的是 **会议室中心到中心**, 而不是房间边界. 2. 覆盖半径 25 米 = 2.5 房间, 因房间长 10 米. 3. 多米诺方向: 初始全 1 全可行, 拿走一些后仍可行 多米诺正确. 反过来初始全 0 不可行, 加回需依赖未来决策, 多米诺会反向. 4. 搜索树叶字数  $2^{mn}$ , 但实际剪枝大幅减少.

## 总结表

| 题号 | 类型   | 复杂度                        | 关键技巧      |
|----|------|----------------------------|-----------|
| 1  | 阶比较  | —                          | 函数阶层次     |
| 2  | 递推方程 | $\Theta(n^2 \log^{k+1} n)$ | 主定理扩展     |
| 3  | 算法分析 | $W(n) = n - 1$             | 最优性       |
| 4  | 算法设计 | $O(n)$                     | Select    |
| 5  | DP   | $\Theta(n^2)$              | OPT[j] 递推 |
| 6  | 贪心   | $O(n \log n)$              | 交换论证      |
| 7  | 回溯   | $O(2^{mn})$                | 子集树 + 多米诺 |

## 备考建议

### [TIP] 复习要点

1. 阶排序题: 每年都有, 函数阶层次表必须烂熟. 2. 递推方程: 主定理 3 情形 + 扩展. 不属于主定理时用递归树. 3. 算法分析: 读懂递归 / 迭代代码, 写出递推方程. 4. DP / 贪心 / 分治 / 回溯各出 1 题. 关键: 算法思想 + 状态/方程 + 证明 + 复杂度. 5. 答题模板见 midterm\_review.md §8.